

01_Fondamenti_Ingegneria_Software

Ingegneria del Software - Appunti Completi del Corso

Parte 1 - Introduzione all'Ingegneria del Software Moderna

1. Il Software nei Sistemi Moderni

Il software costituisce ormai la struttura portante dei sistemi moderni in innumerevoli settori (trasporti, comunicazioni, sanità, finanza, ecc.). Di conseguenza, quando un sistema software fallisce, gli effetti possono essere immediati e critici.

I principali impatti di un fallimento includono:

- Interruzione di servizi più o meno essenziali.
- Problemi legati alla sicurezza e all'esposizione indebita della privacy e dei dati sensibili.
- Perdite economiche e crollo della fiducia da parte del cliente o utente.
- Impossibilità di gestire e garantire la scalabilità del sistema durante i picchi di accesso (es. sistemi di prenotazione o piattaforme bancarie).

Approccio Ingegneristico

È necessario **un approccio ingegneristico** per progettare, costruire e mantenere sistemi complessi e duraturi.

2. Le Cause dei Fallimenti e la Necessità dell'Ingegneria

In quasi tutti i casi di fallimento, il problema non è mai riducibile a "una singola riga di codice sbagliata". Le cause principali risiedono in carenze metodologiche:

- Requisiti iniziali poco chiari, ambigui o incompleti.
- Decisioni progettuali e architetture prese in modo affrettato.
- Mancanza di test adeguati e di una corretta gestione del rischio.

Scopo dell'Ingegneria del Software

L'Ingegneria del Software nasce proprio per fornire metodologie che riducano la probabilità che questi scenari critici si verifichino (o per limitarne i danni qualora si presentassero).

3. Programmazione vs. Ingegneria del Software

La semplice programmazione (la stesura del codice) rappresenta solo una porzione limitata del lavoro complessivo. I piccoli progetti personali possono sembrare lineari, ma nei sistemi reali la complessità esplode a causa di numerosi fattori:

- Molti attori coinvolti con ruoli diversi (utenti, committenti, analisti, sviluppatori, tester, manutentori).
- Vincoli tecnici e organizzativi, uniti alla gestione di una grande mole di dati e all'interazione tra componenti eterogenei.
- Requisiti in continua evoluzione, cambiamenti tecnologici e molteplici contesti d'uso.

Per governare questa complessità entra in gioco una figura fondamentale come quella dell'analista, che funge da tramite tra gli stakeholder (che esprimono bisogni spesso vaghi) e il team tecnico. Il suo compito è tradurre visioni di business in requisiti tecnici precisi.

Che cos'è l'Ingegneria del Software

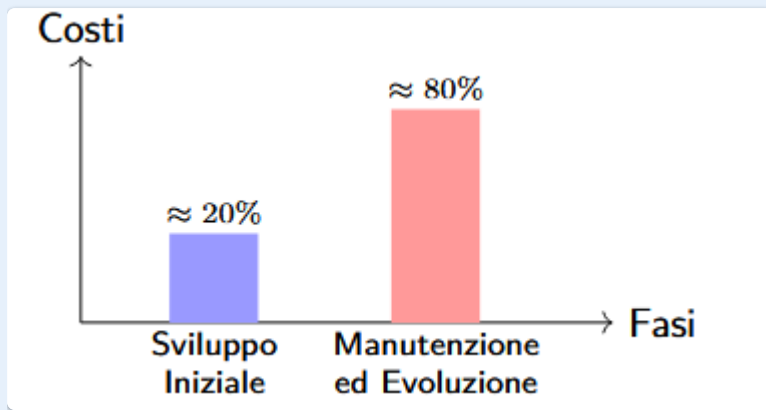
L'Ingegneria del Software è l'insieme di principi, metodi e strumenti volti a sviluppare e mantenere sistemi software che siano affidabili, utili ed economicamente sostenibili nel tempo.

L'attività di sviluppo è governata da quattro processi generici fondamentali:

1. **Specifica**: Comprendere esattamente cosa il software deve fare e identificare i vincoli che deve rispettare.
2. **Sviluppo**: Progettare l'architettura, le strutture interne del sistema e implementare (scrivere) il codice.
3. **Validazione**: Verificare e testare che il sistema sviluppato soddisfi realmente i requisiti attesi.
4. **Evoluzione**: Modificare e adattare il software per rispondere a nuove esigenze, correggere difetti e affrontare i cambiamenti del contesto operativo.

4. Analisi dei Costi e Ciclo di Vita

Costi

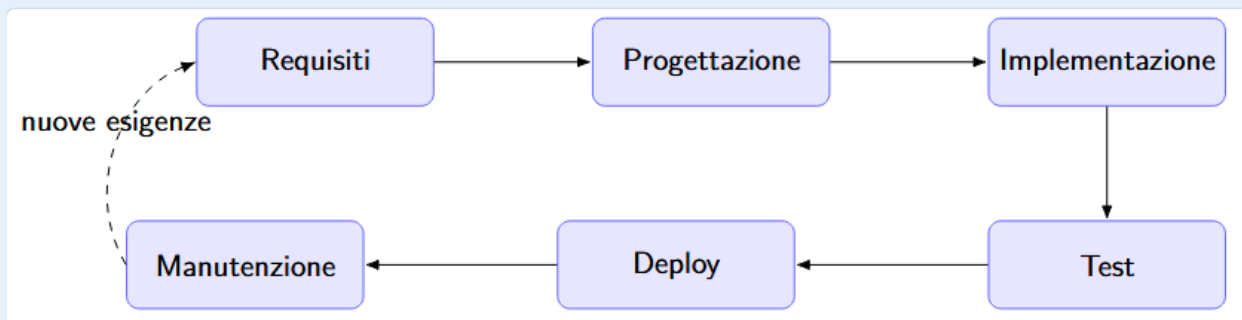


Generalmente, nei progetti reali, i costi si dividono in modo asimmetrico: circa il 20% delle risorse è assorbito dallo sviluppo della prima versione, mentre il restante 80% è destinato alla manutenzione, agli aggiornamenti e all'evoluzione del sistema.

La sostenibilità a lungo termine del software dipende drasticamente dalla **qualità delle scelte architetturali** effettuate nelle fasi iniziali. Un buon progetto facilita enormemente l'evoluzione futura, mentre un design fragile o frettoloso la rende costosa e altamente rischiosa.

Lo sviluppo necessita quindi di un percorso **rigoroso e ordinato** per evitare un lavoro caotico e incontrollabile. Il **ciclo di vita** del software fornisce questo quadro concettuale. Le sue fasi canoniche includono: raccolta dei requisiti, progettazione della soluzione, implementazione, verifica/test, rilascio (deploy) e manutenzione.

Ciclo della vita sinteticamente



Le attività iniziali seguono un flusso lineare: requisiti → progettazione → implementazione. Dopo il rilascio della prima versione, il processo abbandona la linearità per entrare in un ciclo continuo e iterativo fatto di test, rilascio e manutenzione. Ogni nuovo bugfix o modifica **riattiva** questo ciclo. Spesso, proprio le attività di manutenzione generano l'esigenza di nuovi requisiti, chiudendo il loop e riportando il processo alla fase iniziale.

5. Comprensione del Problema e Qualità

Comprensione del Problema

La costruzione del software deve partire da una reale comprensione del problema da risolvere e delle esigenze degli utenti. Prima di scrivere codice, occorre rispondere a domande fondamentali: Chi userà il sistema? Con quali obiettivi?

In quali contesti operativi? Con quali vincoli? Un'analisi iniziale affrettata o superficiale porterà a sviluppare un software poco utile, scarsamente adottato o estremamente costoso da correggere a posteriori. Dedicare il giusto tempo alla comprensione del dominio evita errori strutturali di fondo che impatterebbero su client, maintainer e amministratori.

Poiché emergono costantemente nuove informazioni ed esigenze, nessun progetto procede in linea retta. Il processo reale è intrinsecamente iterativo e adattivo.

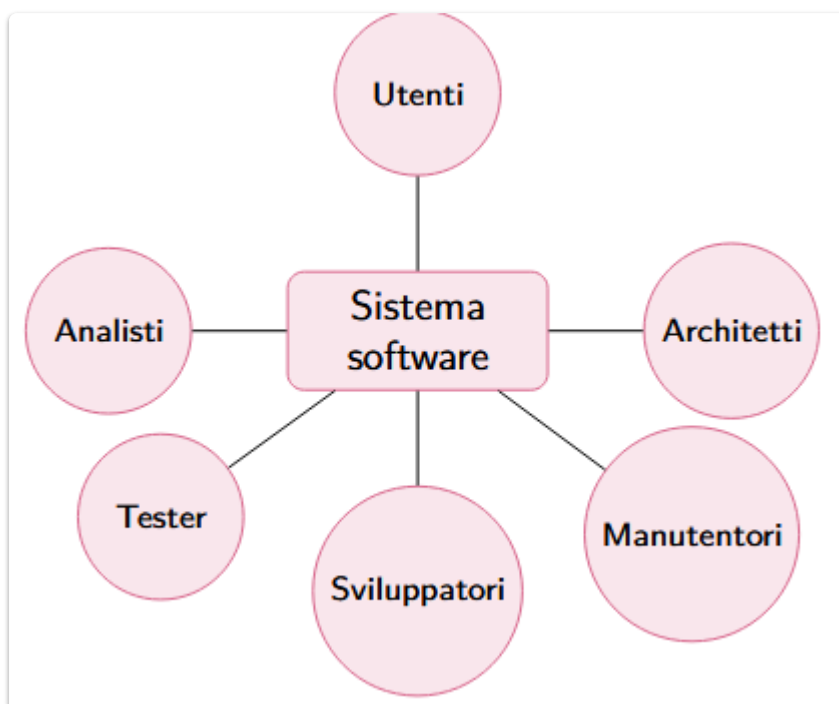
La qualità del prodotto si manifesta su due livelli interdipendenti:

1. **Qualità Esterna (Visibile)**: È quella percepita direttamente dall'utente e comprende l'esperienza d'uso (UX), le funzionalità offerte e le prestazioni.
2. **Qualità Interna (Invisibile)**: Riguarda l'architettura, la pulizia del codice, la testabilità, le dipendenze e la documentazione. Una scarsa qualità interna rende il sistema fragile e difficile da evolvere nel tempo, compromettendo infine anche la qualità esterna.

6. Figure nello Sviluppo

Un sistema software è il risultato del lavoro coordinato di molte figure in ruoli differenti e competenze specifiche. Ad esempio possiamo trovare:

- **Utenti e committenti** → definiscono bisogni e obiettivi;
- **Analisti e progettisti** → traducono le esigenze in modelli e architetture;
- **Sviluppatori e tester** → costruiscono e verificano il sistema;
- **Manutentori** → garantiscono la continuità del servizio nel tempo.



L'ingegneria del Software

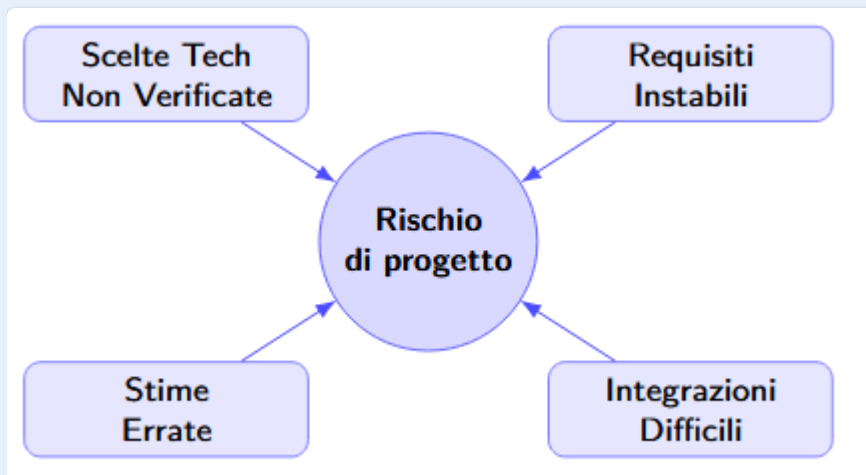
deve far **convivere in modo coerente** ogni ruolo.

7. Comunicazione e Gestione del Rischio

Spesso i difetti e i ritardi di un progetto non derivano da limiti puramente tecnici, ma da incomprensioni tra i vari attori coinvolti. Ogni figura (utente,

sviluppatore, architetto) osserva il sistema da una prospettiva differente e l'ingegneria del software deve farle convivere coerentemente.

Rischio



Esempi comuni di

fallimenti comunicativi:

- Requisiti fraintesi o espressi in modo ambiguo.
- Aspettative degli stakeholder non esplicitate chiaramente.
- Decisioni progettuali prese ma non documentate formalmente.
- Informazioni critiche che non vengono condivise con tutto il team.

Pertanto, una comunicazione efficace è da considerarsi parte integrante dell'ingegneria del software, tanto quanto il codice stesso.

La complessità (qualitativa e quantitativa) e le interazioni inaspettate sono il vero avversario del software. Senza metodo, ogni modifica diventa rischiosa e fa degenerare il sistema. L'Ingegneria del Software non elimina del tutto l'incertezza e gli imprevisti (es. requisiti instabili, stime errate, scelte tecnologiche premature), ma fornisce metodi e strumenti per:

- Prevedere con maggiore accuratezza tempi e costi di sviluppo.
- Valutare sistematicamente alternative architetture prima dell'implementazione.
- Rilevare e risolvere i problemi in fase embrionale.
- Mantenere il controllo sul progetto anche in presenza di forti cambiamenti.

Parte 2 - Processi Software

1. Introduzione ai Processi Software

Sviluppare software non significa semplicemente scrivere codice: lo sviluppo di un sistema software è un'attività collettiva che coinvolge più persone, ruoli e decisioni distribuite nel tempo. Anche con programmatori molto competenti, l'assenza di un processo condiviso porta facilmente a incomprensioni, duplicazione di funzionalità e a un'evoluzione incoerente del sistema.

Un **processo software** fornisce una struttura organizzativa comune per:

- Decidere come procedere (fasi, iterazioni, rilasci).
- Coordinarsi nel tempo (chi fa che cosa e quando).
- Verificare il lavoro in modo sistematico.

L'obiettivo principale non è aggiungere burocrazia, ma ridurre il caos, garantire che il progetto proceda in modo tracciabile e aumentare la prevedibilità dei risultati. In questo modo, la qualità e la conoscenza non dipendono dal singolo individuo, ma dal team.

2. Dal Ciclo di Vita ai Processi

È importante distinguere tra **ciclo di vita** del software e **processo** software:

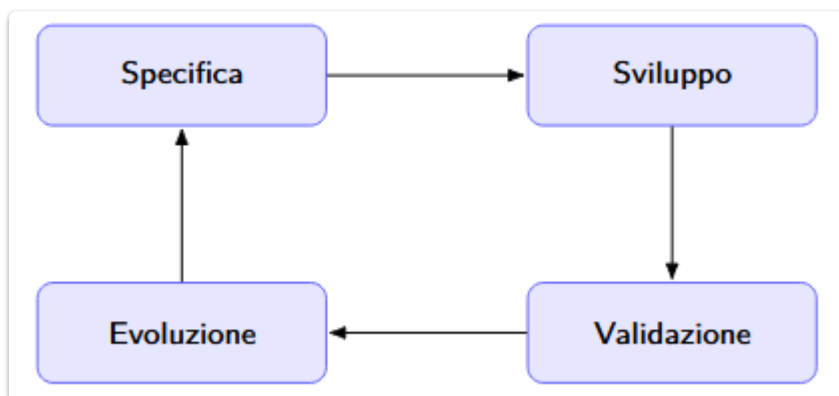
- **Ciclo di vita del software**: È una vista astratta delle fasi che qualunque progetto attraversa (requisiti, progettazione, implementazione, test, deploy e manutenzione).
- **Processi software**: Sono le strategie organizzative che stabiliscono come combinare le fasi del ciclo di vita. Rispondono a domande pratiche come: in quale ordine affrontare le attività, quanto approfondirle e quanto spesso tornare indietro a rivedere le decisioni.

La scelta del processo dipende sempre dal **contesto** e dal **livello di rischio e incertezza** del progetto. L'obiettivo è garantire che il progetto proceda in modo coerente, tracciabile e comprensibile per tutti i soggetti coinvolti.

Qualunque sia il processo adottato, si ritrovano quattro **attività fondamentali**:

1. **Specifica**: Capire cosa il sistema deve fare e quali vincoli rispettare.
2. **Sviluppo**: Progettare e implementare la soluzione.
3. **Validazione**: Verificare che il sistema soddisfi i requisiti e si comporti come atteso.
4. **Evoluzione**: Modificare e mantenere il sistema in risposta a nuove esigenze.

Le differenze tra i vari modelli derivano proprio da come queste attività vengono bilanciate, combinate e iterate nel tempo.



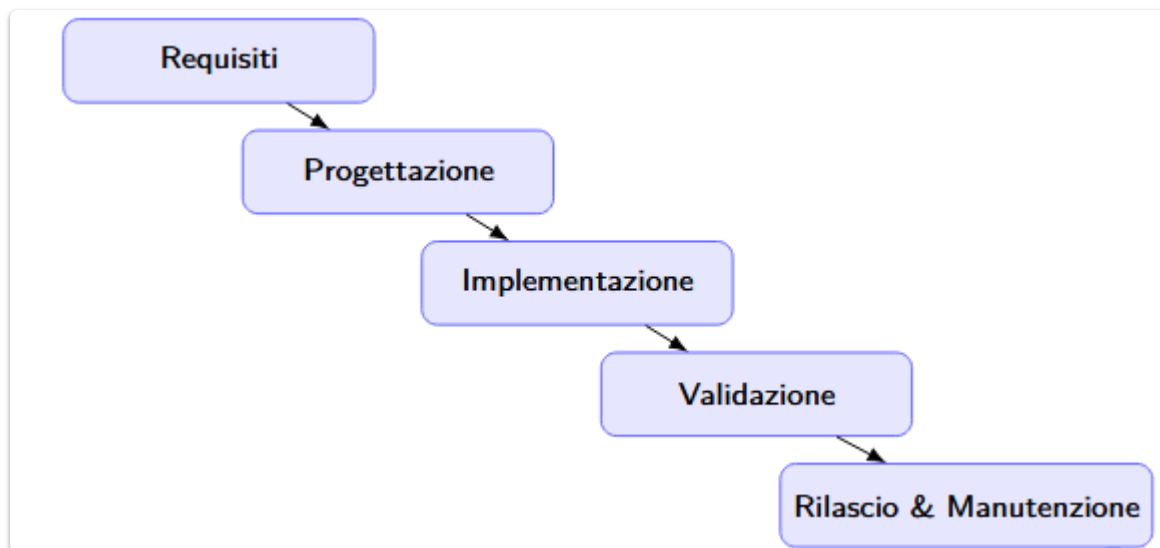
3. I Modelli Classici di Processo

Esistono tre modelli classici, che non sono schemi rigidi e mutuamente esclusivi; spesso i progetti reali ne combinano elementi adattandoli al contesto.

3.1 Modello Waterfall (a cascata)

Organizza lo sviluppo come una sequenza ordinata e lineare di fasi ben separate. Ogni fase produce gli input per la fase successiva (es. requisiti → progettazione → implementazione → validazione → rilascio).

- **Caratteristiche:** È un modello rigido che punta a minimizzare i ritorni indietro; si passa alla fase successiva solo quando la precedente è chiusa.
- **Pro:** Facile da comunicare, forte enfasi sulla documentazione, garantisce tracciabilità ed è ottimale per contesti stabili con obiettivi ben noti.
- **Contro:** Tornare indietro è molto costoso. Poiché gli utenti vedono il sistema completo solo alla fine, c'è un elevato rischio di scoprire troppo tardi mancanze o malintesi. Il *costo del cambiamento* cresce in modo non lineare man mano che si avanza nelle fasi.

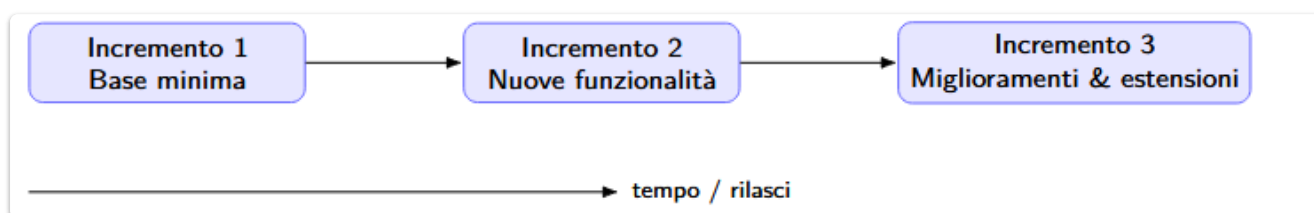


Contesti d'uso Waterfall

È particolarmente adatto quando i requisiti sono ben noti e il contesto cambia lentamente, ad esempio in alcuni ambiti industriali regolamentati.

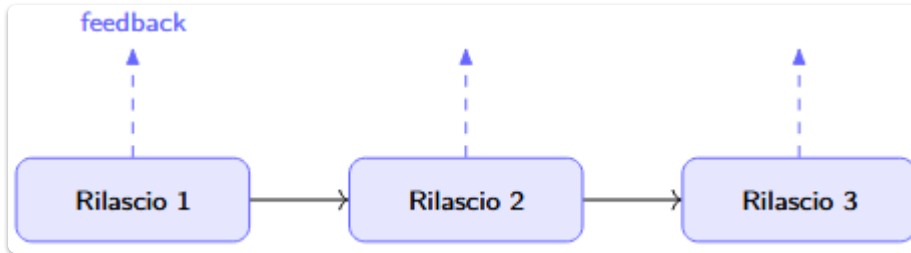
3.2 Modello Incrementale

Il sistema non viene progettato tutto insieme, ma costruito per passi successivi, crescendo attraverso piccoli rilasci progressivi.



- **Vantaggi:** Permette agli utenti di dare un feedback immediato dopo ogni rilascio. Questo consente di correggere la direzione prima, ridurre il rischio e

concentrarsi sulle funzionalità di maggior valore.



- **Attenzioni (Debito Tecnico):** Richiede comunque una visione architeturale complessiva. Se ogni incremento è visto come un progetto a sé, si accumula *debito tecnico* (es. codice duplicato, mancanza di test), che rende i cambiamenti futuri più fragili e costosi. Gli incrementi devono far parte di un disegno coerente.

3.3 Modello a Spirale

Rappresenta lo sviluppo come una serie di cicli successivi per affrontare incertezze. Ogni giro della spirale include: definizione degli obiettivi, analisi dei rischi, sviluppo parziale e valutazione. Accetta che non tutto sia chiaro fin dall'inizio. Costruendo prototipi (versioni semplificate) è possibile ridurre rischi specifici, come testare una tecnologia o un'interfaccia utente. Man mano che i cicli avanzano, il rischio diminuisce.



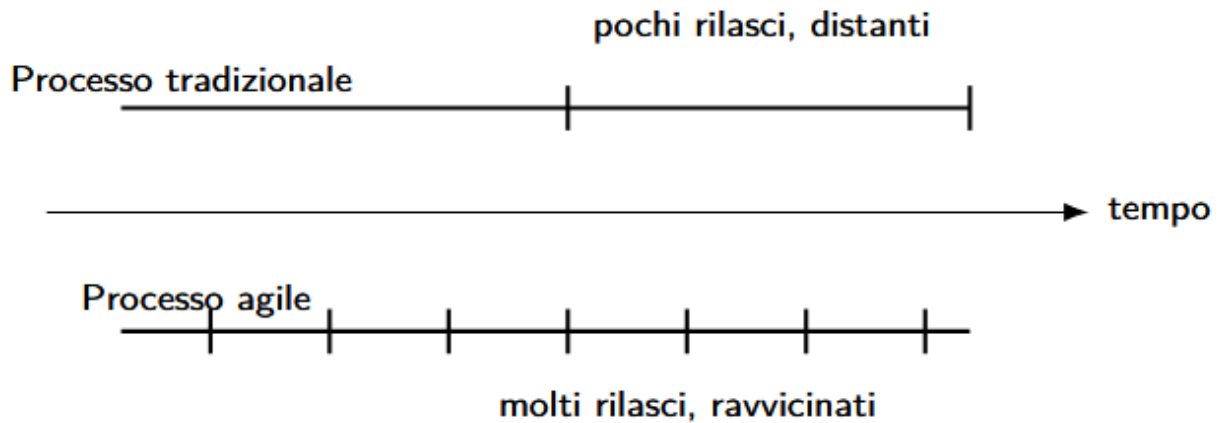
Confronto Modelli

- **Waterfall:** Sequenziale, fasi fisse. Requisiti stabili, documentazione forte. Poco flessibile.
- **Incrementale:** Rilasci progressivi. Feedback frequente, adattamento continuo. Più flessibile.
- **Spirale:** Cicli iterativi guidati dal rischio. Gestione esplicita dell'incertezza, uso di prototipi. Molto flessibile ma più costoso.

4. Metodi Agili e Scrum

I modelli tradizionali nascono per contesti stabili; tuttavia, oggi i progetti affrontano ambienti altamente dinamici, con richieste mutevoli e aspettative di aggiornamenti continui.

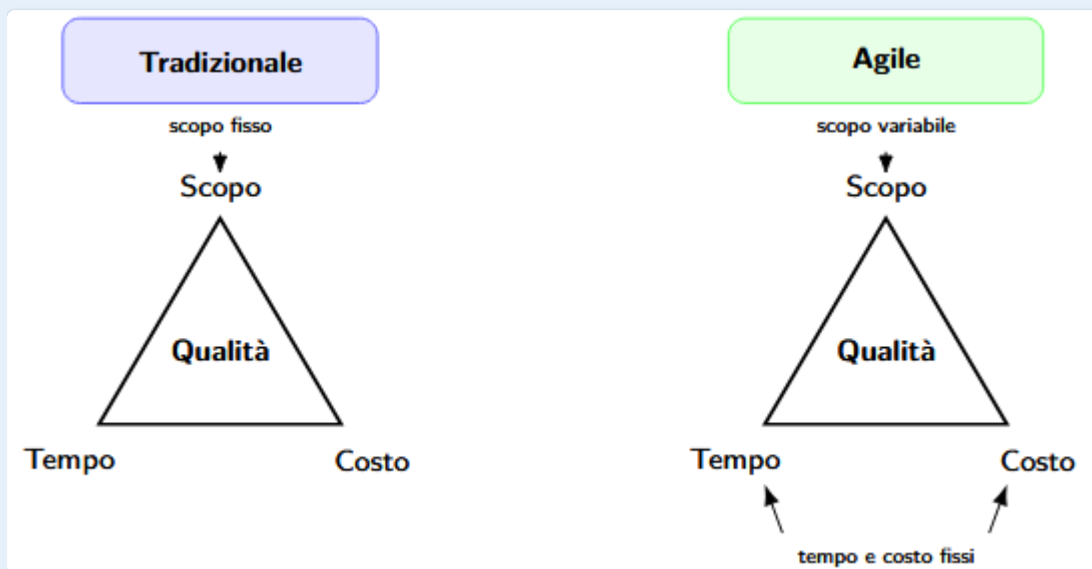
Differenza temporale (rilasci)



I processi tradizionali si basano su pochi rilasci distanti nel tempo. I processi agili introducono iterazioni brevi, con molti rilasci ravvicinati per ottenere feedback costanti e collaborazione continua.

Il focus dell'Agile è sulla capacità di adattamento: il software funzionante è la misura principale dell'avanzamento, e vengono valorizzate le persone, le interazioni e la collaborazione con gli stakeholder.

I Triangoli del Progetto



- **Tradizionale:** Tende a fissare lo **scopo** (tutte le funzionalità previste), lasciando variabili il **tempo** e il **costo**.
- **Agile:** Fissa **tempo** e **costo**, rendendo flessibile lo **scopo** (le funzionalità) in base alla priorità e al valore per l'utente.

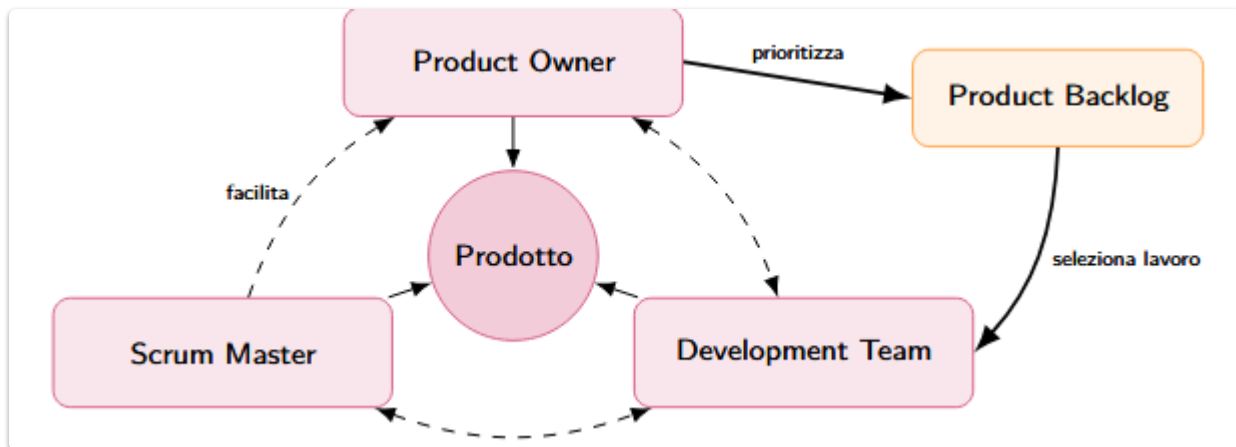
4.1 Scrum

Scrum è un framework logico/organizzativo che definisce come un team collabora, prende decisioni e rilascia valore in modo iterativo.

Si basa su tre ruoli essenziali:

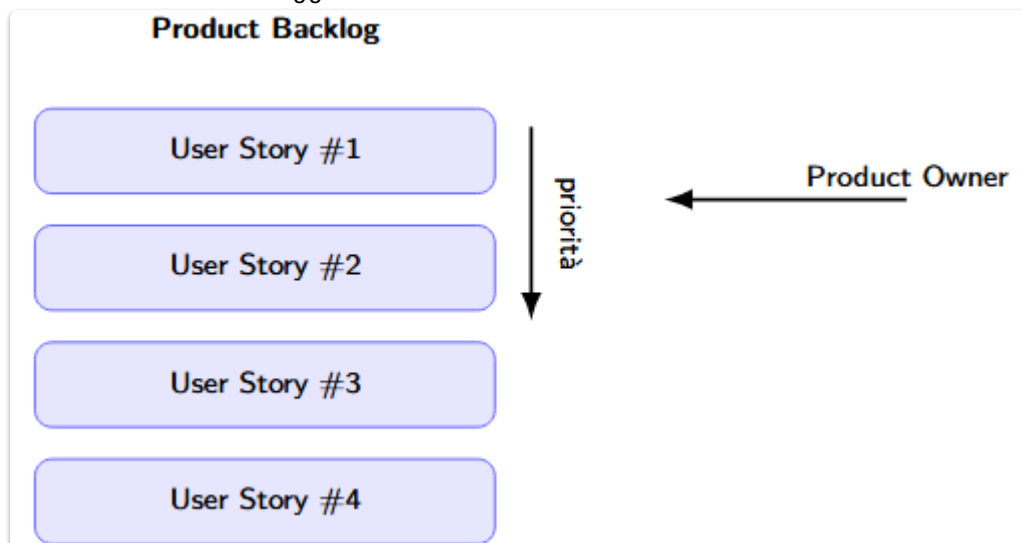
1. **Product Owner:** Rappresenta gli interessi degli utenti e degli stakeholder. Definisce la visione e stabilisce le priorità nel backlog.

2. **Scrum Master**: Facilita il lavoro del team, promuove le pratiche agili e aiuta a rimuovere gli ostacoli.
3. **Development Team**: Realizza concretamente gli incrementi, garantendo la qualità tecnica.

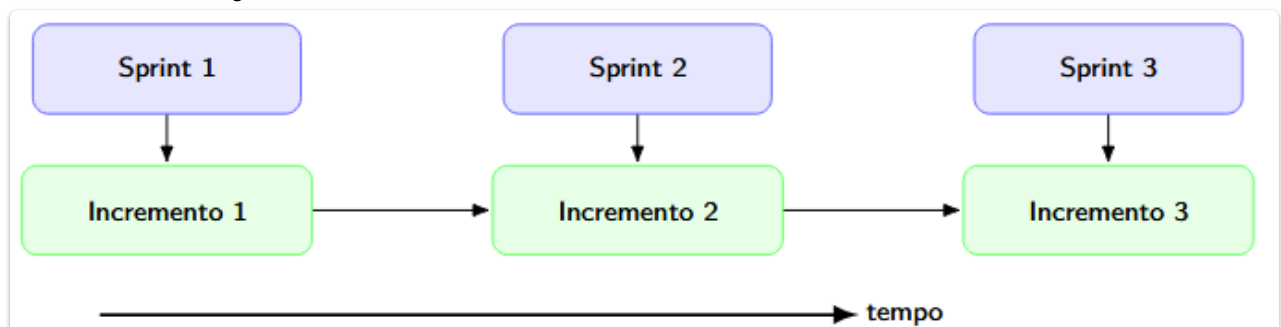


Gli artefatti principali:

- **Product Backlog**: Un elenco ordinato di funzionalità, miglioramenti e bug. È un artefatto "vivo" aggiornato continuamente in base ai feedback.



- **Sprint**: Intervalli di tempo brevi e fissi (es. 2 settimane) con obiettivi limitati ma chiari.
- **Incremento**: Alla fine di ogni sprint, il team deve poter mostrare un incremento funzionante del sistema, consentendo di valutare i progressi e coinvolgere gli stakeholder regolarmente.



Parte 3 - Requisiti Funzionali e Non Funzionali

La definizione dei requisiti è la base su cui poggia ogni scelta successiva del processo di sviluppo (progettazione, implementazione, test e verifiche, evoluzione futura).

Focus

La domanda principale in questa fase è sul "che cosa" deve fare il sistema.

1. Che cos'è un requisito?

Un requisito è una **descrizione chiara e condivisa di qualcosa che il sistema deve fare oppure di un vincolo che deve rispettare**. Rappresenta una vera e propria promessa tra chi richiede il sistema e chi lo realizza (Company and Customer).

Un buon requisito offre diversi vantaggi:

- Riduce le ambiguità e le interpretazioni personali.
- Permette di verificare in modo oggettivo se il sistema si comporta come atteso.
- Guida direttamente la progettazione dei test.

Il criterio guida fondamentale è che un requisito deve essere sempre verificabile e, quando possibile, anche misurabile.

2. Livelli di Dettaglio: Utente vs Sistema

Un requisito può essere espresso a due livelli di dettaglio:

- **Lato utente**: Utilizza un linguaggio semplice, orientato ai bisogni, agli obiettivi e al contesto. È comprensibile a chi utilizzerà il sistema.
- **Lato sistema**: Fornisce una descrizione più precisa, utile a sviluppatori e progettisti per implementare concretamente il sistema.

Esempio a due livelli: La necessità di base è la stessa, cambia solo il livello di precisione con cui la esprimiamo:

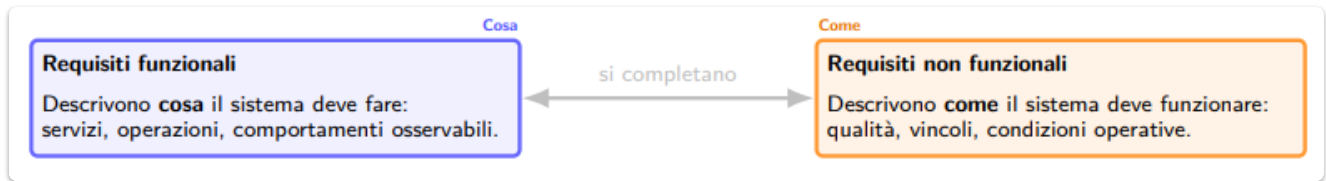
- **Versione lato utente**: «Il sistema deve permettere al docente di ottenere un riepilogo degli appelli d'esame.»
- **Versione lato sistema**: Il sistema genera un elenco contenente corso, data, iscritti ed esiti. Il riepilogo può essere filtrato per periodo, corso o docente. Il risultato è disponibile sia a video sia in formato scaricabile.

3. Le due grandi famiglie di requisiti

I requisiti si dividono in due grandi famiglie:

1. **Requisiti Funzionali**: Descrivono cosa il sistema deve fare, definendo servizi, operazioni e comportamenti osservabili dagli utenti. Definiscono le funzioni dirette del sistema.

2. **Requisiti Non Funzionali**: Descrivono come il sistema deve funzionare, quali vincoli deve rispettare e quale livello di qualità, limiti e contesto operativo deve garantire.



Requisiti Funzionali

I requisiti funzionali definiscono la trama del sistema: senza di essi, il sistema non potrebbe svolgere la sua attività principale. Riguardano azioni specifiche come inserire dati, consultare elenchi, o avviare procedure.

Il pericolo della vaghezza

Un requisito funzionale vago (es. "Il sistema deve permettere di cercare gli esami") non è verificabile. Non indica con chiarezza il risultato atteso e non guida la definizione dei test. Ambiguità di questo tipo si accumulano nel tempo generando debito tecnico. Prima o poi sarà inevitabile dover rivedere codice e documentazione per riallinearli a una definizione più precisa.

Requisiti Non Funzionali

I requisiti non funzionali non descrivono azioni specifiche, ma comportamenti e qualità attese. Riguardano aspetti trasversali fondamentali per la riuscita del progetto. Un sistema che "fa tutto" ma è lento o poco sicuro risulta comunque inadeguato.

Comprendono:

- **Prestazioni**: Tempi di risposta e capacità di gestire molti utenti contemporanei.
- **Sicurezza**: Protezione dei dati, autenticazione e autorizzazioni.
- **Affidabilità**: Continuità del servizio, recupero guasti e gestione degli errori.
- **Accessibilità e usabilità**: Chiarezza dell'interfaccia e riduzione delle barriere.
- **Compatibilità**: Integrazione con i sistemi già esistenti.
- **Rispetto di norme**: Regolamenti tecnici e gestione dei dati personali.

Esempi Pratici

- **Affidabilità**: Garantire un server sempre attivo, che prevenga crolli con troppi utenti o a seguito di cali di tensione.
- **Accessibilità/Usabilità**: Prevedere font adatti per utenti daltonici e assicurarsi che i template siano usabili.
- **Compatibilità**: Assicurarsi che il software sia compatibile con diverse applicazioni esterne.

Le Tre Famiglie dei Requisiti Non Funzionali

- **Legati al prodotto:** Qualità interne (prestazioni, tempi di risposta, consumo di risorse).
- **Legati all'organizzazione:** Vincoli del contesto di sviluppo/uso (strumenti, procedure interne, integrazioni).
- **Legati all'ambiente esterno:** Normative, standard e regolamenti esterni da rispettare.

Prodotto
Prestazioni, risposta, affidabilità

Organizzazione
Strumenti, procedure, integrazioni

Ambiente esterno
Normative, regolamenti, standard

4. Requisiti Misurabili

Un requisito è utile solo se verificabile in modo oggettivo. Obiettivi generali come "veloce" o "facile da usare" non sono verificabili direttamente. Occorre trasformarli in requisiti misurabili.

Ad esempio, la transizione da *"Il sistema deve essere veloce"* a *"Il sistema deve rispondere entro 2 secondi per almeno il 95% delle richieste"*. Questo evita interpretazioni diverse e guida in modo coerente le scelte architetturali.

5. Raccogliere i Requisiti

I requisiti non emergono spontaneamente; bisogna raccogliergli dialogando con gli stakeholder. Strumenti utili includono:

- Colloqui con gli utenti per comprendere bisogni e difficoltà.
- Osservazione delle attività reali senza filtri.
- Analisi di procedure e documenti esistenti.
- Prototipi e dimostrazioni per far emergere idee latenti.

Le esigenze si scoprono progressivamente, generando requisiti che all'inizio possono essere incompleti o in conflitto tra loro (es. "facile da usare" ma con "password complesse"). L'analisi serve proprio a chiarire, verificare la compatibilità e negoziare le priorità.

Una volta raccolti i bisogni, per rappresentarli in modo semplice e comprensibile si utilizzano le **User Story**.

6. User Stories

Una user story descrive un'esigenza dal punto di vista dell'utente, in modo semplice e privo di dettagli implementativi. Serve ad allineare utenti e sviluppatori per popolare un backlog comprensibile.

User Story vs Caso d'Uso

- **User story:** Frase breve orientata a chi-cosa-perché, utile per la gestione del backlog e la prioritizzazione dei bisogni.
- **Use case:** Descrizione strutturata e dettagliata del comportamento del sistema, usata per la modellazione formale e comportamentale (es. UML).

La forma tipica essenziale risponde a tre domande (chi, cosa, perché):

| «Come [tipo di utente], voglio [azione] per [obiettivo / beneficio].»

De facto, per una user story ben formulata, si deve specificare:

- **Chi**: l'attore che usa il sistema;
- **Cosa**: l'azione desiderata, espressa in modo semplice e osservabile;
- **Perché**: il valore/beneficio atteso per l'utente o per il processo.

Esempio: «Come docente, voglio esportare la lista iscritti per preparare il verbale.»

7. Criteri di Accettazione

Ogni user story deve essere accompagnata da criteri di accettazione: condizioni chiare che stabiliscono quando la funzionalità è davvero completata.

Essi servono a:

- Eliminare ambiguità sulle aspettative.
- Definire il confine tra "fatto" e "non fatto".
- Fornire una base solida per i test di verifica.

Mentre la user story racconta l'intenzione, i criteri ne specificano la verifica concreta. Possono usare forme strutturate come «Dato che... Quando... Allora...».

8. Requisiti, Backlog e Priorità

Le user stories diventano elementi del backlog, una lista ordinata di ciò che il sistema offrirà. È necessario stabilire delle priorità, affrontando prima le funzionalità che portano più valore, che affrontano rischi tecnici, o che sbloccano dipendenze. Il backlog è un artefatto vivo che cambia e si aggiorna.

Anche i requisiti non funzionali entrano nel backlog:

- Possono diventare **attività specifiche** (es. migliorare i tempi di risposta o potenziare la sicurezza).
- Possono rappresentare **vincoli trasversali** da rispettare in ogni funzionalità (es. normativa sui dati o compatibilità).

Ignorarli nelle prime fasi porta quasi sempre a interventi più costosi (*rework*) e rischiosi in seguito.

Parte 4 - Modellazione Concettuale

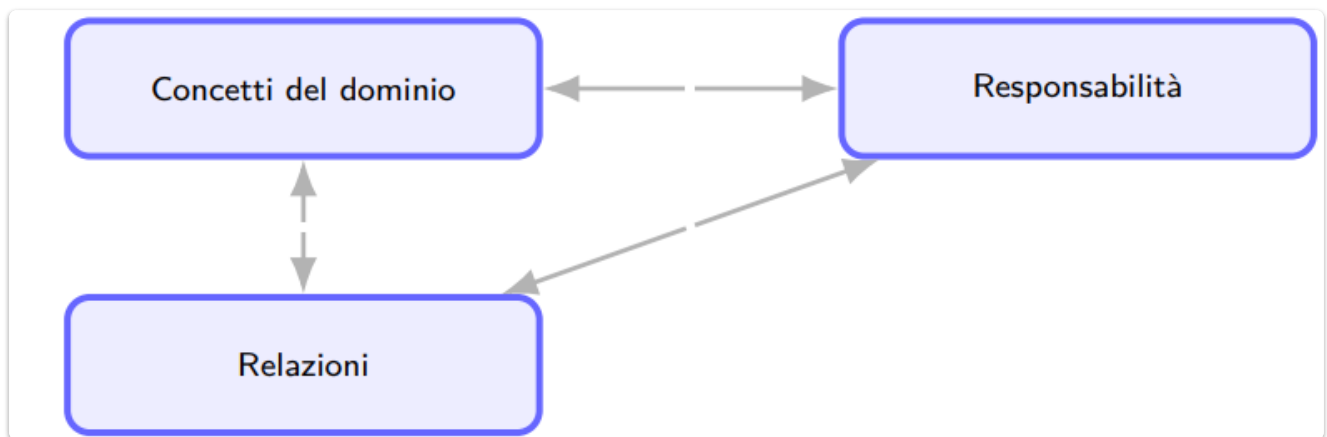
1. Introduzione alla Modellazione Concettuale

Il **dominio** rappresenta il mondo reale in cui il sistema si inserisce, ovvero l'insieme di persone, oggetti, regole e processi reali che il software deve rappresentare e supportare. La modellazione concettuale è l'attività che funge da ponte tra i requisiti e l'analisi strutturata, mirando a:

- Identificare i concetti importanti del dominio.
- Chiarire quali responsabilità ha ciascun concetto.
- Descrivere le relazioni tra questi concetti.

Questa fase descrive logicamente il problema, indipendentemente dalla soluzione tecnica (non si parla ancora di classi, metodi, tabelle del database o architetture).

I **concetti** sono elementi significativi del dominio, non vanno ancora pensati come classi, tabelle o dettagli implementativi. La modellazione concettuale ci aiuta a individuare i concetti davvero importanti come una sorta di guida.



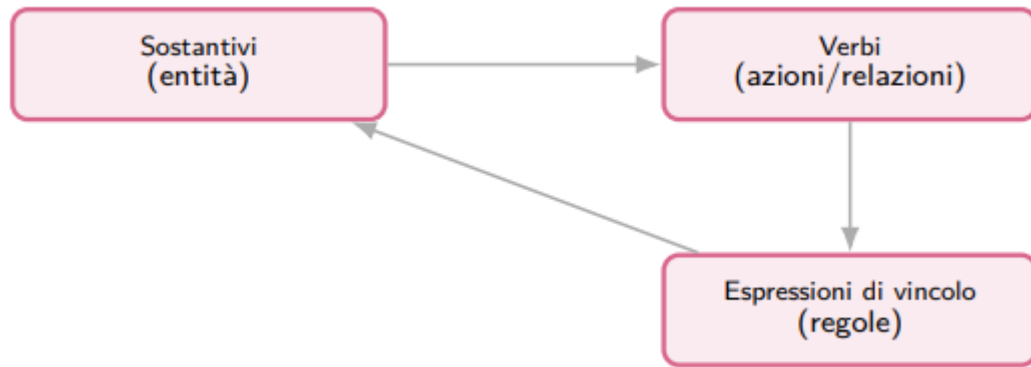
Lo scopo principale è costruire un vocabolario condiviso tra utenti, analisti e sviluppatori.

2. Dai Requisiti ai Concetti

I requisiti testuali scritti in precedenza contengono già molti suggerimenti sulla struttura del dominio. Leggendoli con attenzione, possiamo estrarre gli elementi fondamentali:

- **Sostantivi:** Suggeriscono le potenziali **entità** del dominio (es. studente, appello, corso, voto, verbale, carriera).
- **Verbi:** Suggeriscono le **azioni** o le **relazioni** tra le entità (es. iscriversi, registrare, consultare).
- **Espressioni di vincolo:** Nascondono le **regole** e i limiti del dominio (es. "solo se", "entro").

[i Guida all'estrazione](#)

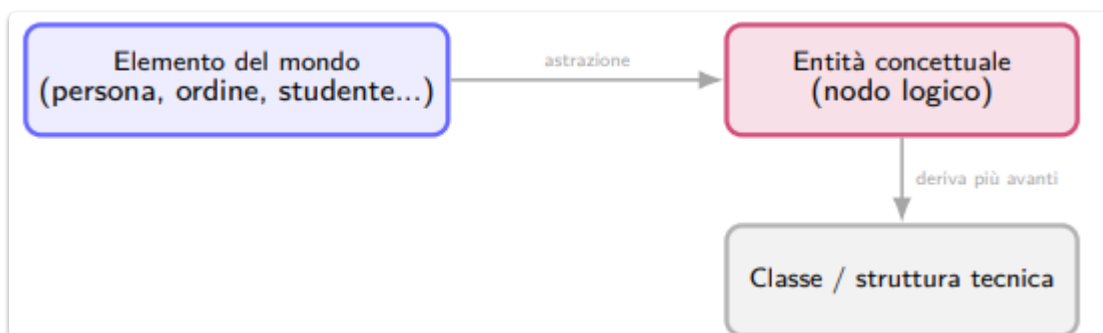


L'estrazione di sostantivi, verbi e vincoli non è una trasformazione meccanica, ma una guida pratica per la modellazione.

3. Le Entità Concettuali

Un'entità concettuale è un **nodo logico**, un **elemento del mondo** che vogliamo rappresentare in modo **esplicito** nel sistema. Esiste indipendentemente dalla tecnologia scelta e ha un significato riconoscibile anche dai non tecnici.

L'**errore da evitare** assolutamente in questa fase è confondere le entità concettuali (astrazioni del dominio) con le strutture di codice (classi) o le tabelle del database (implementazione). Mentre l'entità descrive "cosa" è rilevante per il business, la **classe** è la sua rappresentazione formale in un diagramma UML o nel codice sorgente.



L'idea non è "definire le classi", ma capire in modo chiaro:

- **Che cosa** rappresenta ciascuna di queste entità.
- **Perché** è utile distinguerle.
- **Che tipo** di informazioni ci aspettiamo che custodiscano.

Esempio applicato: UniManager

Nel caso di studio UniManager, le entità principali individuate sono:

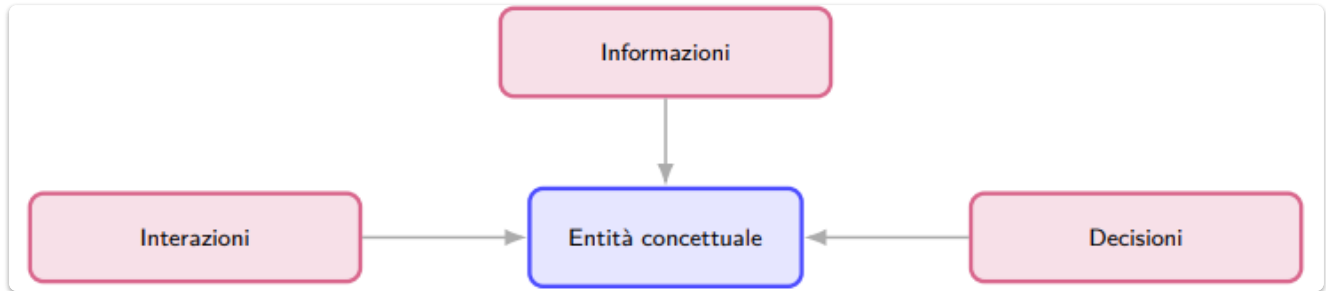
- **Studente**: Rappresenta la persona che segue i corsi e sostiene gli esami.
- **Corso**: Rappresenta l'insegnamento (nome, docente, CFU).
- **Appello**: Rappresenta un'occasione concreta per sostenere l'esame (data, ora, aula).
- **Iscrizione**: Rappresenta l'atto con cui uno studente si registra a uno specifico appello. Ha una propria data e stato, utile per tracciare la storia delle interazioni.

- **Voto:** Rappresenta l'esito ottenuto dallo studente in quell'appello. Non è un semplice attributo numerico dell'appello, ma ha uno stato proprio e può essere oggetto di statistiche o controlli.

4. Responsabilità delle Entità

A ogni entità associamo delle responsabilità specifiche:

- Quali **informazioni** custodisce.
- In quali **interazioni** è coinvolta.
- Quali **decisioni** del dominio passano da essa.



5. Relazioni tra le Entità

Le entità non vivono isolate: le relazioni tra i concetti rendono interessante un dominio. Si dividono in due famiglie principali:

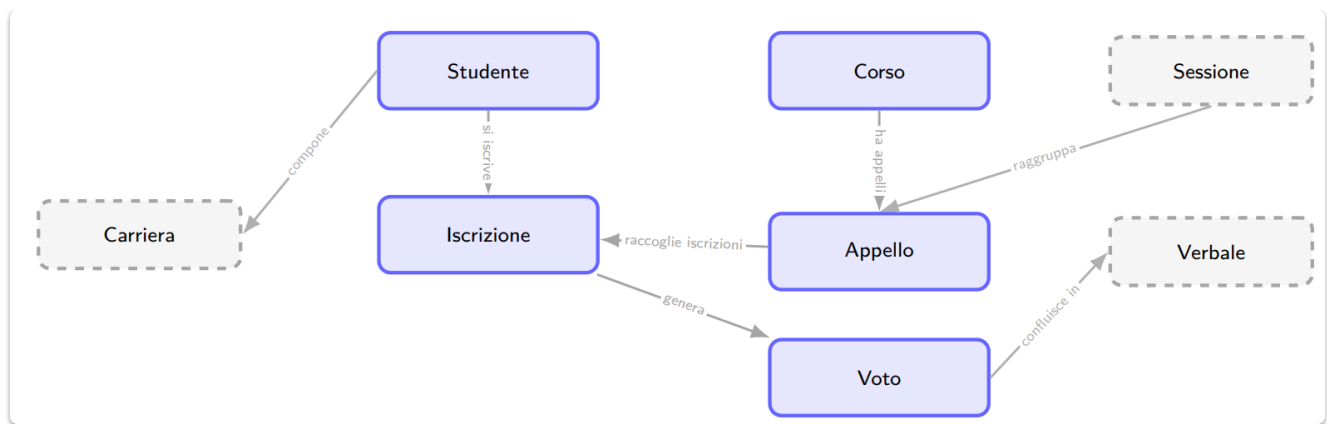
1. **Relazioni Strutturali:** Descrivono l'organizzazione stabile del dominio.
 - **Corso - Docente:** Un corso è tenuto da uno o più docenti.
 - **Corso - Appello:** Un corso ha uno o più appelli nel tempo.
 - **Iscrizione - Appello:** Ogni iscrizione si riferisce a uno specifico appello.
2. **Relazioni di Azione/Evento:** Descrivono ciò che avviene nel sistema.
 - **Studente → Iscrizione:** Crea una nuova iscrizione a un appello.
 - **Docente → Voto:** Registra o aggiorna l'esito.

Esempi pratici sulle Azioni

Queste relazioni sono più vicine al comportamento del sistema e includono tipicamente precondizioni, effetti e controlli. L'obiettivo principale qui è **capire chi interagisce con chi e in che modo**.

6. Mappa Concettuale

La mappa concettuale è la rappresentazione visiva e informale della vista strutturale del dominio. Utilizza nodi per le entità ed etichette testuali (in italiano, senza simboli complicati) per le relazioni. Essa è uno strumento vivo che può evolvere e raffinarsi man mano che emergono nuovi bisogni. Inoltre, è fondamentale definire i confini del sistema per evitare di attribuire al software responsabilità che non gli competono.



Prima di procedere con UML formale o la scrittura del codice, verifica:

- **Nomi chiari:** Comprensibili anche ai non tecnici (user friendly).
- **Relazioni esplicite:** Coerenti con i requisiti.
- **Niente dettagli tecnici:** Evitare accuratamente di pensare già a classi, DB o API.

Saltare la modellazione concettuale introduce vincoli tecnici prematuri, rischia fraintendimenti e rende le correzioni molto più costose nelle fasi avanzate. Investire tempo in questa fase riduce le ambiguità e consolida le basi per la progettazione architetturale successiva.

Parte 5 - UML e Use Case Diagram

1. Introduzione a UML

UML (Unified Modeling Language) è un linguaggio di modellazione standard condiviso, non un linguaggio di programmazione. Fornisce un insieme di diagrammi visivi per descrivere:

- La **struttura** statica del sistema.
- Il **comportamento** dinamico del software.
- Le **interazioni** tra le parti interne del sistema e gli attori esterni.

UML mette a disposizione diverse famiglie di diagrammi. I principali diagrammi trattati sono:

- **Use Case Diagram:** mostra la vista del sistema dal punto di vista degli attori esterni.
- **Modello di dominio:** è il modello concettuale del dominio, spesso rappresentato con un class diagram semplificato.
- **Class Diagram:** descrive la struttura statica del dominio, incluse classi, attributi e associazioni.
- **Sequence Diagram:** illustra le interazioni nel tempo tra oggetti e attori.
- **Activity Diagram:** rappresenta il flusso di attività, le decisioni e le responsabilità di processo.
- **Altri diagrammi** (come State Machine, Component, Deployment) fanno parte dello standard UML ma non sono oggetto principale di questo corso.

2. Use Case Diagram

Il diagramma dei casi d'uso è strettamente legato ai requisiti e alle user stories del sistema. Si concentra su chi usa il sistema e per quali obiettivi specifici.

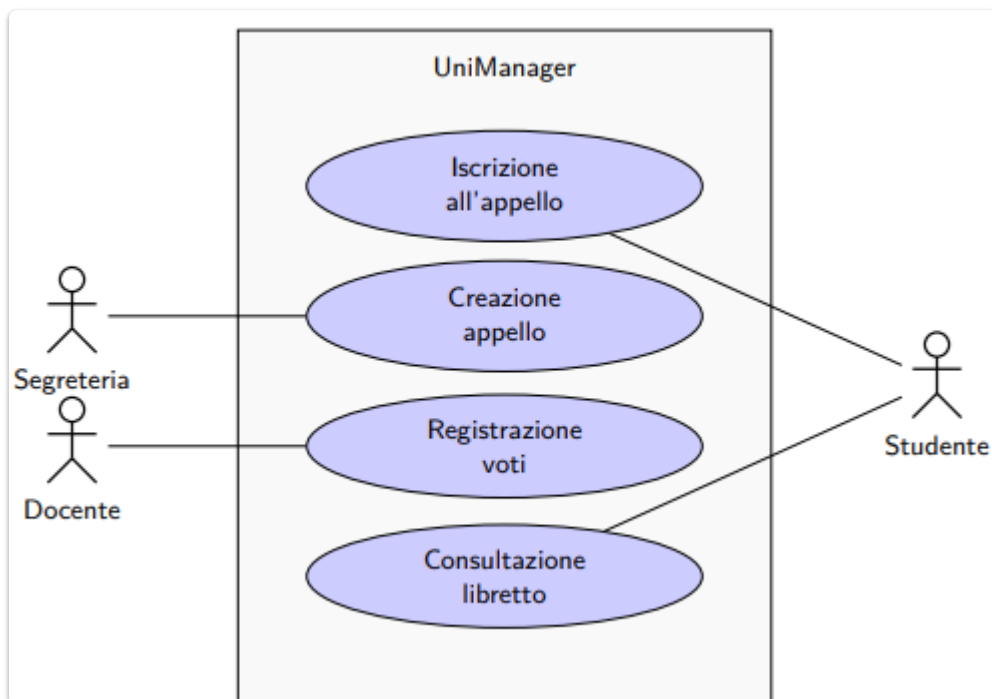
Nel diagramma dei casi d'uso, la separazione tra interno ed esterno è fondamentale:

- **Sistema:** Viene rappresentato graficamente da un rettangolo che racchiude i casi d'uso. Rappresenta il confine di ciò che il software deve fare.
- **Attori:** Sono ruoli esterni che interagiscono con il sistema, situati sempre al di fuori del rettangolo. Un attore può essere una persona (es. studente, docente), un altro sistema (es. sistema di autenticazione) o un dispositivo. Non indicano singole persone, ma una tipologia di utilizzatore.

2.1 Casi d'Uso

Un caso d'uso rappresenta un obiettivo di business significativo per un attore. Graficamente è rappresentato da un'ellisse contenente il nome dell'obiettivo.

- Non rappresenta una singola funzione tecnica (come una "query sul DB") o un singolo "click".
- Rappresenta un risultato di senso compiuto per l'utente, come ad esempio *"Iscrizione all'appello"* o *"Registrazione voti"* nel sistema UniManager.



2.2 Relazioni negli Use Case Diagram

Oltre all'associazione base (linea semplice) che collega un attore a un Use Case, esistono relazioni più complesse:

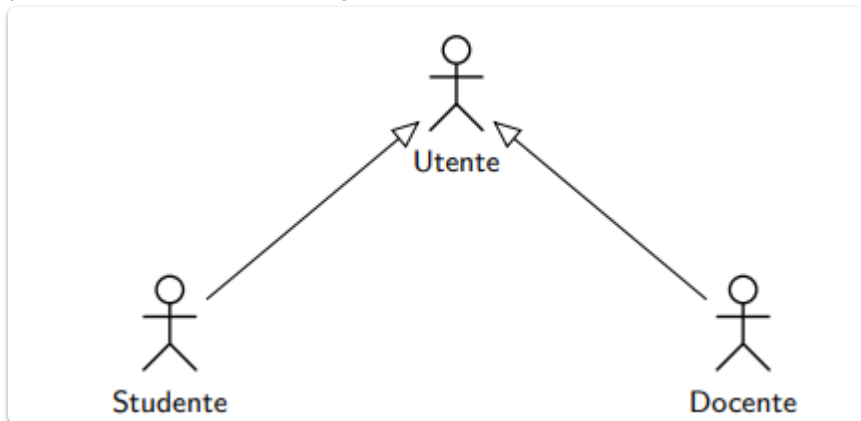
- **Include** (`<<include>>`): Indica un comportamento comune e obbligatorio. Il caso d'uso "incluso" è un pezzo riusabile che viene eseguito sempre come parte del caso d'uso principale (es. l'autenticazione per potersi iscrivere a un appello).



- **Extend** (`<<extend>>`): Rappresenta una variante opzionale. Il caso d'uso che "estende" aggiunge passi opzionali o condizioni particolari a un flusso normale, attivandosi solo in determinate circostanze (es. invio di una notifica opzionale dopo la registrazione di un voto).



- **Generalizzazione tra Attori**: Permette di creare una gerarchia per evitare ripetizioni. Un attore "padre" (es. Utente) raggruppa i casi d'uso comuni, mentre gli attori "figli" (es. Studente, Docente) ereditano le interazioni e aggiungono le proprie specificità. Viene indicata con una freccia triangolare vuota che punta verso l'attore generale.



2.3 Elementi Organizzativi e Descrittivi

- **Package**: Raggruppano i casi d'uso per aree funzionali, migliorando la leggibilità visiva in sistemi complessi senza alterarne il comportamento.
- **Vincoli e Note**: I vincoli (scritti tra parentesi graffe `{}`) esplicitano regole di business fondamentali, come scadenze o prerequisiti. Le note aggiungono commenti testuali per chiarire assunzioni altrimenti ambigue.
- **Descrizione Testuale**: I diagrammi offrono una sintesi visiva, ma i casi d'uso critici richiedono una scheda testuale dettagliata. Questa deve includere: precondizioni, postcondizioni, flusso principale dei passi "normali" ed eventuali flussi alternativi o eccezioni.

Parte 6 - Domain Model e Class Diagram

1. Che cos'è un Domain Model

Il Domain Model è una **rappresentazione concettuale** del dominio del problema. È una "fotografia" del mondo a cui il sistema software si riferisce e non contiene dettagli tecnici.

Esso mostra:

- I **concetti rilevanti** (entità, oggetti, ruoli).
- Le **relazioni** tra questi concetti.

- Gli **attributi essenziali** che li caratterizzano.

Non contenendo alcun dettaglio implementativo, nel Domain Model non si parla di database, API, interfacce grafiche, e non vi sono metodi o algoritmi.

Perché è importante?

Il Domain Model aiuta a capire il problema prima di pensare alla soluzione. Agisce come un linguaggio condiviso tra sviluppatori, analisti, domain expert e stakeholder non tecnici. Un modello chiaro facilita la transizione verso la progettazione.

2. Dalle Storie agli Oggetti del Dominio: La Tecnica

Per estrarre le entità concettuali dai requisiti o dai casi d'uso (es. *"Lo studente si iscrive a un appello"*), si utilizza una strategia basata sull'analisi testuale:

- **Evidenziare i sostantivi**: questi diventano i candidati concetti del modello.
- **Evidenziare i verbi**: questi diventano i candidati per le associazioni o le responsabilità.

Un concetto è considerato un buon candidato se è nominato spesso nei requisiti, se ha regole di business proprie o se è riconosciuto dagli stakeholder come un'entità reale del loro mondo (es. Studente, Corso, Appello).

3. Attributi Essenziali e Associazioni

Gli attributi descrivono informazioni importanti su un concetto. Essi dovrebbero essere pochi (seguendo il principio del minimalismo), stabili nel tempo, non derivabili e rilevanti per le regole di business.

Errori frequenti nella scelta degli attributi:

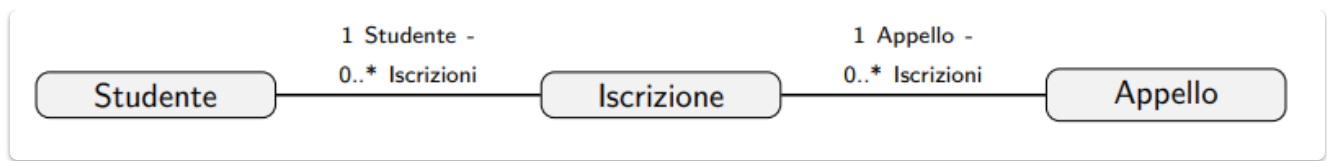
- Confondere il Domain Model con il database, inserendo ID tecnici, timestamp di audit o campi di log.
- Aggiungere attributi troppo tecnici, come token di sessione o flag di caching.
- Inserire **attributi derivabili**, ovvero dati che possono essere ottenuti da altri (es. numero di esami superati derivabile dalle iscrizioni, o media voti).

3.1 Associazioni e Molteplicità

Le associazioni rappresentano legami concettuali tra gli elementi del modello. La **molteplicità** esprime quante istanze di un concetto possono essere collegate a un altro, traducendo in diagramma i vincoli reali del dominio.

La lettura rapida delle molteplicità è la seguente:

- **1**: esattamente una.
- **0..1**: opzionale.
- **0..***: zero o molte.
- **1..***: almeno una.



4. Dal Domain Model al Class Diagram

Esiste una netta differenza tra i due diagrammi:

- **Domain Model**: È concettuale. Mostra i concetti del mondo reale, gli attributi essenziali per capire il problema e le relazioni concettuali tra entità.
- **Class Diagram**: Guarda verso il software. Introduce il concetto di classe, mostra gli attributi che saranno effettivamente memorizzati e le relazioni che guideranno la struttura del codice.

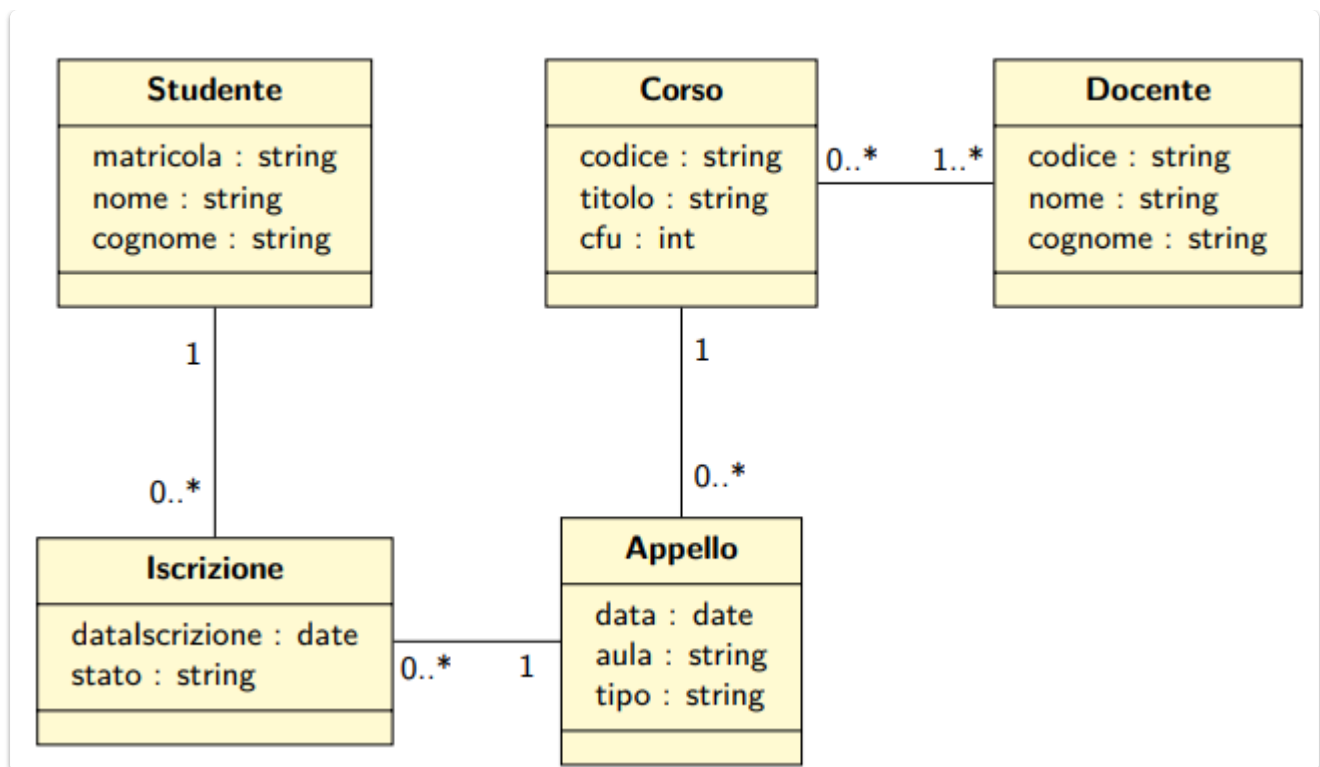
4.1 Checklist per il passaggio al Class Diagram

- **Cosa portare in questa fase**: I nomi chiari delle classi, gli attributi essenziali tipizzati (es. string, int) e le relazioni ispirate al Domain Model.
- **Cosa lasciare fuori in questa fase**: Metodi, logica di dettaglio, classi tecniche o di infrastruttura (Repository, DAO, Controller, helper) e dettagli legati ai framework.

5. Notazione UML Avanzata

Nei diagrammi di classe orientati al software si adottano notazioni specifiche:

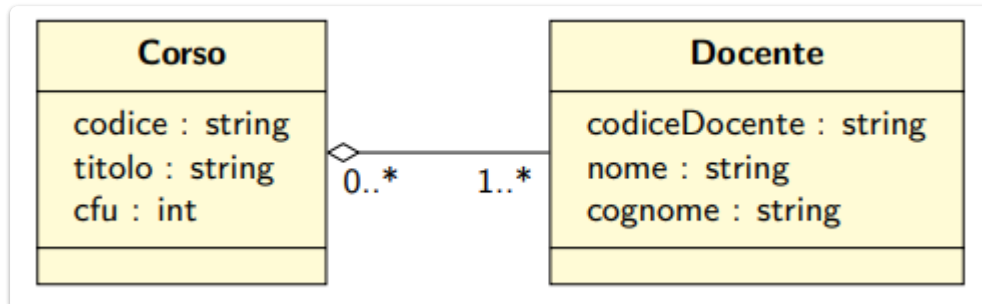
- **Visibilità degli attributi**:
 - **+** public: visibile a tutti.
 - **-** private: visibile solo all'oggetto stesso.
 - **#** protected: visibile a classe e sottoclassi.
 - **~** package: visibile all'interno del package.
- **Attributo derivato**: Un attributo che può essere calcolato a partire da altri dati. Si indica ponendo il simbolo **/** davanti al nome (es. **/eta** derivato da **dataNascita**).
- **Attributo statico (di classe)**: Sono attributi condivisi tra tutte le istanze della classe. Si indicano sottolineando il nome (es. **contaCorsi**).
- **Ereditarietà (Generalizzazione)**: Si indica con una freccia a triangolo vuoto che punta verso la superclasse. Le sottoclassi ereditano attributi e operazioni dalla superclasse.



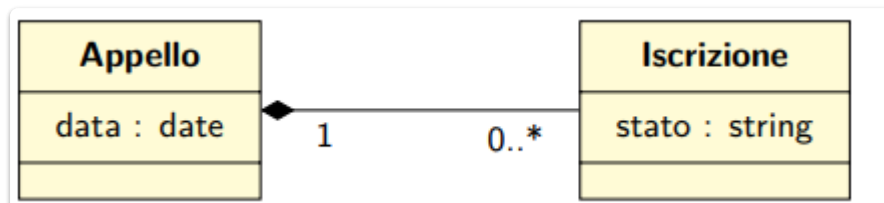
5.1 Aggregazione e Composizione

Rappresentano particolari tipi di associazione in cui esiste un rapporto tra un "tutto" e una "parte":

- **Aggregazione (rombo vuoto)**: La parte può esistere anche indipendentemente dal tutto. Es. Se si elimina un *Corso*, i *Docenti* associati continuano a esistere.



- **Composizione (rombo pieno)**: La parte dipende strettamente dal tutto per esistere (vincolo esistenziale). Es. Se si elimina un *Appello*, le *Iscrizioni* relative a quell'appello non hanno più senso e cessano di esistere.



Parte 7 - Sequence Diagram e Activity Diagram

Dal modello statico, che definisce la struttura del sistema (Domain Model e Class Diagram), si passa al livello dinamico per analizzare come gli oggetti collaborano durante uno scenario e in quale ordine avvengono azioni e decisioni. Per fare

questo, si utilizzano due viste complementari: il Sequence Diagram per le interazioni tra partecipanti e l'Activity Diagram per il flusso del processo.

1. Sequence Diagram

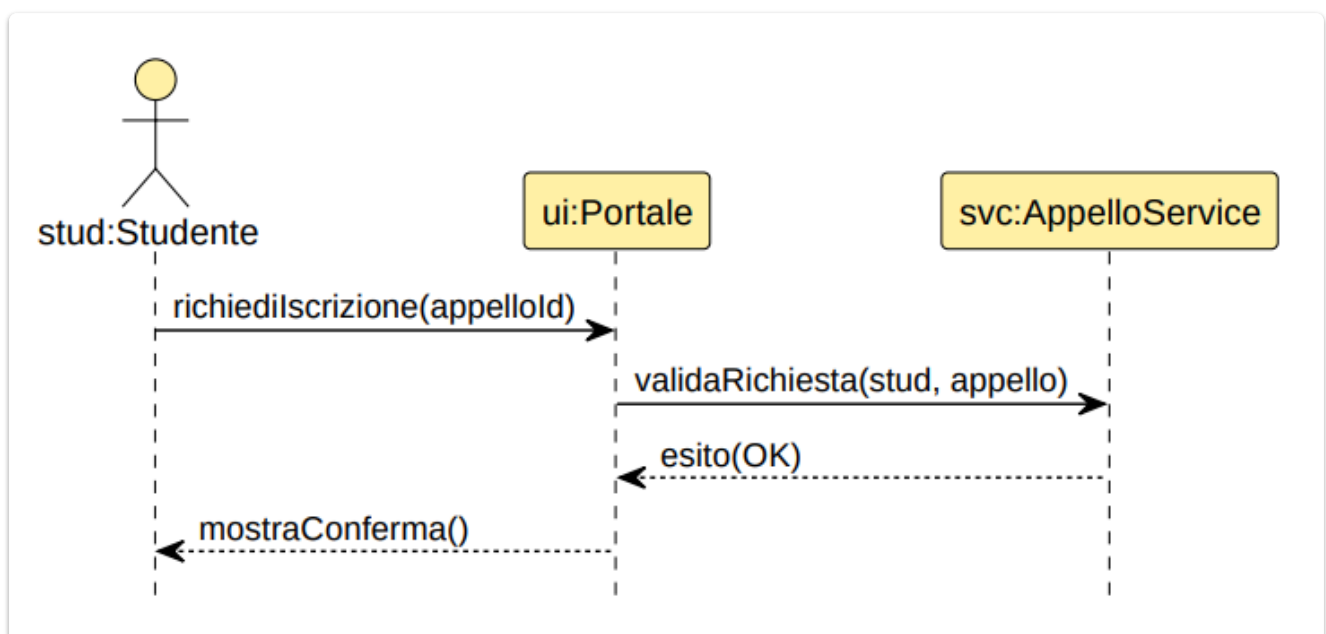
Il Sequence Diagram si utilizza per **chiarire come uno scenario viene eseguito nel tempo**. È particolarmente utile per scenari di caso d'uso con esito positivo e per le relative varianti di errore.

Risponde a tre quesiti operativi principali:

- **Chi interagisce con chi.**
- **In quale ordine** avvengono i messaggi.
- **Chi prende in carico ogni responsabilità.**

La lettura corretta di un Sequence Diagram segue due assi principali:

- **Verticale:** Il tempo scorre dall'alto verso il basso.
- **Orizzontale:** Mostra i diversi partecipanti coinvolti nello scenario.



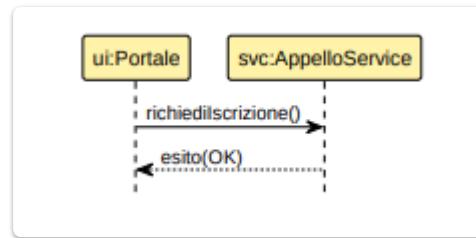
- **Quando usarlo:** Quando la logica e l'ordine dei messaggi sono rilevanti per il comportamento del sistema.
- **Errore da evitare:** Disegnarlo ovunque senza un reale valore informativo.

1.1 Componenti Principali del Sequence Diagram

- **Lifeline (Linea di Vita):** Ogni partecipante ha una lifeline che lo identifica nello scenario. L'obiettivo è rendere esplicito chi partecipa prima di dettagliare i messaggi.
 - Possiamo distinguere diversi ruoli con diverse responsabilità: attore, interfaccia e servizio.
- **Activation (Barra di Attivazione):** Indica il periodo in cui il partecipante sta effettivamente eseguendo del lavoro.
 - Serve a leggere meglio la durata logica e la nidificazione delle chiamate.

1.2 Tipologie di Messaggi

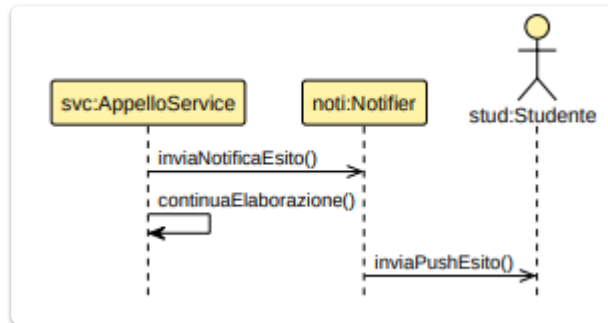
- **Messaggio Sincrono:** Il chiamante invia il messaggio e attende il completamento dell'operazione, creando una dipendenza temporale immediata. Rappresentato da una



freccia con punta piena.

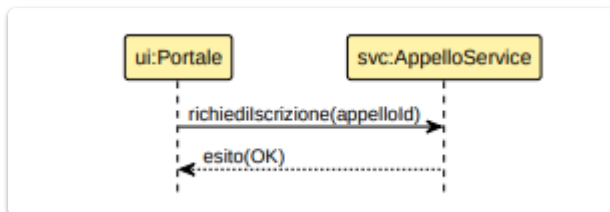
- **Messaggio Asincrono:** Il chiamante invia il messaggio ma prosegue la sua esecuzione senza bloccarsi (es. in integrazioni event-driven). Rappresentato da una freccia con punta aperta.

- **Nota:** Se si manda un messaggio asincrono, qualcuno a un certo punto deve

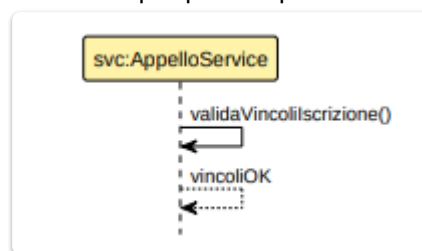


gestirne l'esito.

- **Return Message:** Freccia tratteggiata che rappresenta il risultato di una richiesta sincrona, utile se il valore di ritorno guida una decisione successiva.



- **Create / Destroy:** Mostrano rispettivamente quando un oggetto nasce durante l'interazione e quando termina il suo ciclo di vita temporaneo (cleanup esplicito con una 'X').
- **Self-call:** Un partecipante invoca una propria operazione interna per evidenziare



una sotto-fase significativa.

1.3 Frammenti (Fragments)

I frammenti permettono di modellare logiche complesse all'interno del Sequence Diagram:

- **Alt:** Modella rami alternativi mutuamente esclusivi (es. success/failure nello stesso scenario).
- **Opt:** Esprime un comportamento opzionale (simile a un *if* senza *else*), dove il flusso principale resta valido anche senza di esso.
- **Loop:** Rappresenta una ripetizione controllata da una condizione esplicita.
- **Break:** Interrompe il normale scenario in caso eccezionale; il resto del flusso principale non prosegue.

- **Par:** Descrive rami concorrenti eseguibili in parallelo, utile per discutere latenza e sincronizzazione.
 - **Critical:** Delimita una sezione che richiede mutua esclusione e non deve essere interrotta.
 - **Ref:** Rimanda a un'interazione definita altrove, riducendo la duplicazione di sotto-scenari ricorrenti.
 - **Assert:** Indica una sequenza di protocollo o una traccia che deve valere obbligatoriamente.
 - **Neg:** Rappresenta e rende esplicita una traccia vietata o un comportamento non ammesso.
 - **Strict vs Seq:** `strict` impone un ordine globale totale senza interleaving, mentre `seq` (comportamento di default) permette un ordine parziale preservando solo i vincoli locali.
-

2. Activity Diagram

A differenza del Sequence Diagram che analizza il dettaglio dei messaggi, l'Activity Diagram si focalizza sul flusso complessivo del processo. È utile per descrivere flussi di attività, regole decisionali, passi operativi end-to-end e parallelismi.

2.1 Struttura Base

- **Initial e Final Node:** Ogni attività parte da un nodo iniziale (cerchio nero pieno) e termina in un nodo finale (cerchio pieno con bordo), delimitando chiaramente il processo.
- **Action e Control Flow:** Le azioni (rettangoli arrotondati) sono i passi di lavoro, mentre il control flow (freccie piene) definisce l'ordine di esecuzione.

2.2 Decisioni e Concorrenza

- **Decision e Merge:** Il *decision* (rombo con un ingresso e più uscite) separa i rami operativi in base a *guardie* esplicite (condizioni logiche tra parentesi quadre, es. `[posti > 0]`). Il *merge* (rombo con più ingressi e un'uscita) ricompone il flusso.
- **Fork e Join:** Il *fork* (barra nera spessa con più uscite) apre esecuzioni concorrenti, mentre il *join* (barra con più ingressi) le sincronizza convergendo in un unico flusso.
- **Swimlane:** Partizioni grafiche che assegnano chiare responsabilità organizzative ai vari passi, indicando chi fa cosa (es. Studente vs Sistema).

2.3 Eventi, Segnali e Interruzioni

- **Send Signal:** L'attività emette un evento verso l'esterno e prosegue senza attendere una risposta immediata (integrazione event-driven).
- **Receive Signal:** Il flusso si interrompe restando in attesa di un evento esterno per poter proseguire.
- **Time Event:** Il processo reagisce a una transizione innescata dal tempo (scadenza temporale).
- **Interruptible Region:** Una regione del processo che può essere interrotta forzatamente da un evento (es. comando stop o annullamento utente).

3. Confronto Finale: Quando usare cosa?

- **Usa il Sequence Diagram:** Quando il rischio è distribuire male la logica e le responsabilità tra gli oggetti. È ideale per analizzare le interfacce e i messaggi precisi tra i componenti.
 - **Usa l'Activity Diagram:** Quando il rischio è perdere la visione complessiva e il flusso logico end-to-end del processo di business, comprese decisioni e parallelismi.
 - I due diagrammi sono complementari: vengono spesso usati insieme sullo stesso scenario per fornire una vista dinamica completa.
-
-

Parte 8 - Principi di Architettura Software

Nelle tre lezioni UML precedenti abbiamo usato cinque diagrammi per capire che cosa deve fare il sistema:

1. Use Case Diagram per funzionalità e attori;
2. Domain Model per i concetti del dominio;
3. Class Diagram per la struttura software;
4. Sequence Diagram per i messaggi nel tempo;
5. Activity Diagram per il flusso di processo.

Questa analisi ci dà una buona fotografia del comportamento e del dominio di UniManager. Il passo successivo è chiederci: come organizziamo il software in modo che questi elementi siano implementati in maniera ordinata, comprensibile e mantenibile?

L'architettura software è il ponte tra: l'analisi (requisiti, casi d'uso, modelli UML), e l'implementazione concreta (classi, moduli, codice). *Se l'analisi ci dice cosa deve succedere, l'architettura ci dice come organizzare il sistema perché quelle funzionalità possano essere realizzate in modo coerente.* Un'architettura chiara rende più semplice:

- assegnare il lavoro a più sviluppatori,
- prevedere l'impatto dei cambiamenti,
- discutere le scelte progettuali usando una mappa condivisa.



Parliamo di architettura in questo momento del corso perché:

- abbiamo già compreso il dominio di UniManager (gestione di corsi, appelli, iscrizioni e voti),
- abbiamo identificato gli attori e i casi d'uso principali di questo dominio (Studente, Docente, Segreteria),

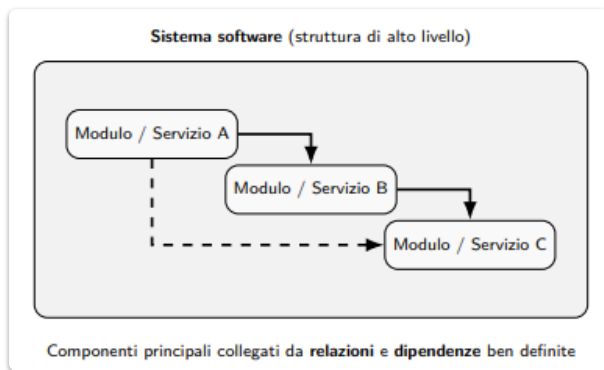
- abbiamo un modello concettuale stabile degli elementi centrali del dominio (Studente, Corso, Appello, Iscrizione).

È il momento di decidere come strutturare il codice in modo che rispecchi questa comprensione, evitando di arrivare all'implementazione con un insieme di file disorganizzati. Le scelte architetturali prese all'inizio sono difficili da cambiare e condizionano l'evoluzione futura del sistema: per questo è importante renderle esplicite.

Che cos'è l'architettura software? Possiamo definire l'architettura software come:

- la struttura di alto livello del sistema,
- la suddivisione in componenti principali (moduli, servizi, sottosistemi),
- le relazioni e le dipendenze tra questi componenti.

L'architettura non entra nei dettagli di ogni singolo metodo, ma definisce il contenitore in cui questi dettagli andranno a collocarsi. Una buona architettura aiuta chiunque, anche non autore del codice, a farsi un'idea del sistema “a colpo d'occhio”.



Possiamo considerare l'architettura software a due livelli:

1. **in piccolo:** riguarda la struttura interna di un singolo sistema applicativo, come UniManager. Ci chiediamo come suddividere il software in componenti, servizi, moduli.
2. **in grande:** riguarda sistemi più complessi che coinvolgono molte applicazioni, servizi distribuiti, componenti remoti o cloud. Qui le decisioni architetturali includono protocolli, distribuzione, scalabilità.

Nel corso ci concentreremo soprattutto sull'architettura in piccolo, perché è la base su cui costruire soluzioni più complesse. Il passaggio all'architettura “in grande” avviene spesso in modo naturale, man mano che un sistema cresce e integra componenti esterni.

≡ Esempio concreto di architettura “in grande”:

- una piattaforma e-commerce come Amazon, con sistemi distinti per catalogo, ordini, pagamenti, spedizioni e raccomandazioni,
- applicazioni dedicate per utenti finali e operatori (web, app mobile, app per venditori e logistica),
- servizi distribuiti e comunicazione tra sistemi tramite API/eventi.

L'architettura è anche l'insieme delle decisioni strutturali che:

- sono difficili da cambiare una volta implementate,
- influenzano molte parti del sistema,
- impattano su prestazioni, manutenibilità, riusabilità.

☰ Alcuni esempi di decisioni architetturali:

- separare o meno la logica di business dall'accesso ai dati,
- usare o meno livelli distinti per presentazione, logica, persistenza,
- organizzare il codice per funzionalità (Studenti, Corsi, ...) o per tipo di responsabilità (domain = entità e regole di business, service = logica applicativa/casi d'uso, repository = accesso ai dati e persistenza).

Queste decisioni definiscono confini e regole di dipendenza:

- chi può chiamare chi,
- dove può risiedere una regola di business,
- quali componenti possono conoscere dettagli tecnologici.

❗ In UniManager, per esempio, decidere che la CLI parli solo con i Service e non direttamente con i Repository è una decisione architetturale, non di dettaglio.

Progettare l'architettura: significa prendere una serie di decisioni che influenzano profondamente il sistema, per esempio:

- come suddividere il software in componenti,
- quale forma far assumere ai confini tra i componenti,
- come far comunicare i diversi moduli tra loro,
- quali dipendenze ammettere e quali vietare,
- quanto isolare il dominio dai dettagli tecnologici.

Ogni decisione architetturale ha un costo e un beneficio: l'obiettivo è trovare l'equilibrio che rende il sistema comprensibile, efficace e manutenibile.

L'architettura emerge quindi non come una singola scelta, ma come un insieme coerente di decisioni che lavorano insieme.

Architettura vs design di dettaglio: È utile distinguere l'architettura dal design di dettaglio:

- l'architettura definisce i grandi blocchi e le loro relazioni,
- il design di dettaglio decide come questi blocchi sono implementati internamente.

Le due dimensioni non sono in conflitto:

- l'architettura crea i confini (chi fa cosa),
- il design di dettaglio popola questi confini con classi, metodi e algoritmi specifici.

☰ Esempi tipici di design di dettaglio:

- firma dei metodi e scelta dei parametri (iscriviStudente(matricola, idAppello));
- struttura delle classi interne a un modulo e responsabilità dei singoli oggetti;
- algoritmo di ordinamento/ricerca adottato in un servizio;
- gestione puntuale di eccezioni, validazioni e messaggi di errore.

Strumento per il team: In un progetto reale, l'architettura non è solo un concetto tecnico, ma un strumento di comunicazione:

- permette a sviluppatori, progettisti e responsabili di parlare del sistema con un linguaggio comune,
- aiuta a spiegare rapidamente “come è fatto” il software a chi entra nel team.

Sapere in quale componente collocare una nuova funzionalità riduce discussioni inutili e favorisce un'evoluzione ordinata del codice. Un'architettura condivisa rende più semplice anche la revisione del codice, perché si può valutare se una modifica rispetta o meno i confini stabiliti.

Se non ci fosse l'architettura: Se l'architettura non viene pensata o viene lasciata “emergere” senza guida, il rischio è che il sistema si trasformi in una *“big ball of mud”*:

- file molto grandi e confusi,
- logica di business mescolata all'accesso ai dati,
- interfaccia utente che conosce dettagli interni del dominio.

In questo scenario ogni modifica è potenzialmente pericolosa, nessuno sa esattamente che cosa rompe cambiando una riga e si generano “zone del codice” che nessuno vuole toccare.

Obiettivi concreti dell'architettura: Una buona architettura si pone obiettivi molto pratici:

- ridurre la complessità percepita del sistema,
- facilitare i cambiamenti nel tempo, isolandoli dove possibile,
- favorire la testabilità, separando ciò che è facilmente verificabile da ciò che dipende dall'ambiente,
- supportare la crescita del progetto senza doverlo riscrivere da zero ogni anno.

Nel caso di UniManager, vogliamo che aggiungere una funzionalità (es. un nuovo tipo di report) non significhi toccare codice in dieci posti diversi.

Architettura e requisiti non funzionali: L'architettura non serve solo a organizzare il codice: è lo strumento principale per soddisfare molti requisiti non funzionali. Aspetti come:

- prestazioni (tempi di risposta, carico),
- affidabilità e tolleranza ai guasti,
- scalabilità,
- sicurezza,
- manutenibilità ed estendibilità,

dipendono molto più dalle scelte architettureali che dal codice delle singole classi

Un'architettura ben pensata permette di prevedere come il sistema si comporterà sotto stress e come potrà evolvere in futuro. Per questo motivo, le decisioni architettureali vengono prese molto presto nel progetto e orientano molte attività successive.

Decisioni architettureali e trade-off: In architettura non esistono scelte “gratis”: quasi ogni decisione porta vantaggi su alcuni aspetti e costi su altri.

☰ Esempi tipici di trade-off:

- più livelli e confini → maggiore manutenibilità, ma maggiore complessità iniziale;
- moduli molto indipendenti → migliore evoluzione, ma più interfacce da progettare;
- accesso ai dati centralizzato → coerenza, ma possibile collo di bottiglia.

Per questo una decisione architettureale va sempre motivata rispetto a:

- requisiti prioritari del progetto,
- vincoli tecnici e organizzativi,
- costo di implementazione e di evoluzione.

Documentare le decisioni architettureali: Anche in progetti didattici conviene rendere esplicite le decisioni architettureali, evitando che restino solo “nella testa” di chi le ha proposte. Un formato minimo efficace può contenere:

- **contesto:** quale problema dobbiamo risolvere;
- **decisione:** quale scelta adottiamo;
- **alternative:** quali opzioni abbiamo scartato e perché;
- **conseguenze:** effetti attesi, rischi, impatti.

Buona pratica operativa:

- una decisione per documento, titolo chiaro e data;
- aggiornamento del documento quando la decisione cambia.

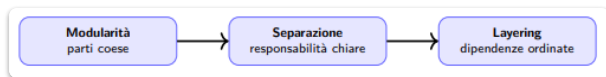
☰ Esempio in UniManager:

“i comandi CLI invocano solo i Service, senza accesso diretto ai Repository”. Questo riduce ambiguità nel team e facilita revisioni future.

Tre principi fondamentali: Per lavorare sull'architettura useremo come filo conduttore tre principi semplici ma molto potenti:

- **modularità:** dividere il sistema in parti coese,
- **separazione delle responsabilità:** evitare componenti “tuttofare”,
- **layering:** organizzare il sistema in livelli con ruoli distinti.

Questi principi non sono legati a un linguaggio specifico, e valgono anche oltre l'OS, come criterio generale di organizzazione.



Vocabolario operativo: CLI/UI, Service, Domain, Repository: Fissiamo i quattro concetti che useremo nel resto del corso:

- **CLI/UI (Command-Line Interface / User Interface):** riceve input utente e mostra output, senza contenere regole di business;
- **Service:** coordina i casi d'uso applicativi (sequenza dei passi, validazioni applicative, gestione esiti/errore) e orchestra le interazioni tra Domain e Repository;
- **Domain:** rappresenta entità e regole del dominio (vincoli, invarianti, significato dei dati), indipendenti da UI e persistenza;
- **Repository:** si occupa di accesso ai dati e persistenza (lettura/scrittura).

Regola guida:

- la **regola di business** vive tra Service e Domain;
- CLI/UI e Repository sono livelli di interazione, non sede della logica di dominio.



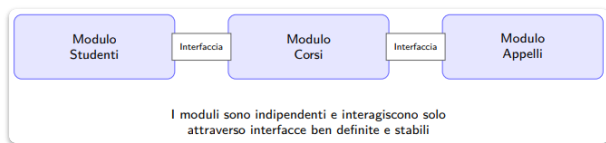
≡ Esempio di una decisione in UniManager

- Problema: dove collocare la regola "uno studente non può iscriversi due volte allo stesso appello"?
- Possibili scelte:
 - nella CLI/UI (scelta errata: vincolo di dominio in livello di presentazione),
 - nel Repository (scelta parziale: verifica tecnica sui dati ma non governo del caso d'uso),
 - in Service + Domain (scelta coerente con le responsabilità architetturali).
- Decisione architetturale coerente:
 - la CLI/UI invia la richiesta di iscrizione al Service;
 - il Service applica la regola di business usando concetti e vincoli di Domain;
 - il Repository viene usato solo per leggere/scrivere lo stato persistente;
 - l'esito (OK/errore) torna alla CLI/UI senza esporre dettagli di persistenza.

Primo principio- modularità: La modularità consiste nel dividere il sistema in parti che:

- hanno un ruolo chiaro,

- comunicano tramite interfacce ben definite,
- sono ragionabili e testabili quasi indipendentemente.

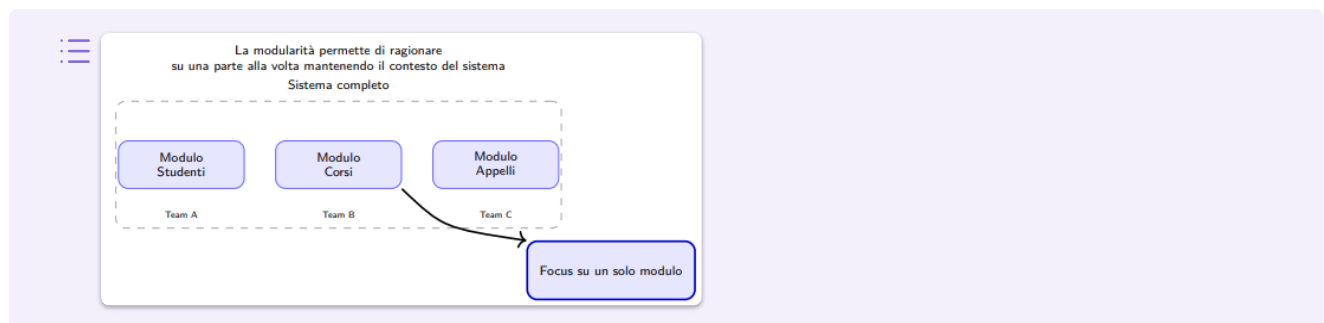


Perché la modularità è essenziale? La modularità riduce la porzione di sistema da analizzare per prendere una decisione progettuale:

- possiamo concentrarci su un modulo alla volta,
- il resto del sistema viene visto come un “servizio” con alcune funzioni disponibili.

Un sistema modulare:

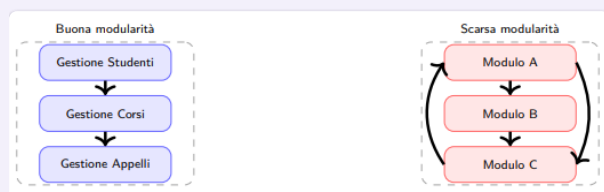
- si presta meglio al lavoro in team (ogni gruppo segue un modulo),
- facilita il riuso (un modulo può essere sfruttato in altri contesti),
- rende più prevedibili gli impatti di ogni modifica.



Segnali di buona e cattiva modularità:

- **Buona modularità:** nomi dei moduli intuitivi e stabili nel tempo, interfacce semplici (pochi metodi, significativi), dipendenze chiare, idealmente in una sola direzione.
- **Scarsa modularità:** moduli che cambiano continuamente scopo, componenti che conoscono dettagli interni l'uno dell'altro, dipendenze incrociate difficili da districare.

Example

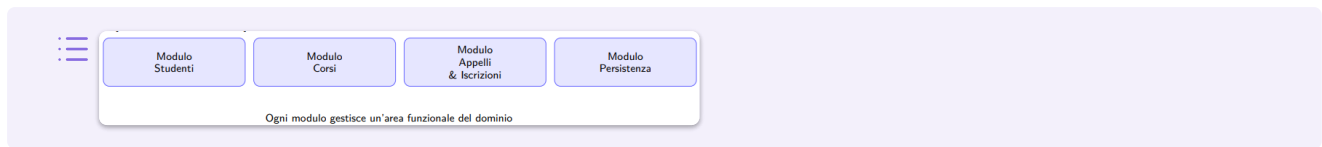


Modularità funzionale in UniManager: Guardando ai requisiti di UniManager, emergono in modo naturale alcune aree funzionali:

- gestione degli Studenti,
- gestione dei Corsi,
- gestione degli Appelli e delle Iscrizioni,

- gestione della persistenza delle informazioni.

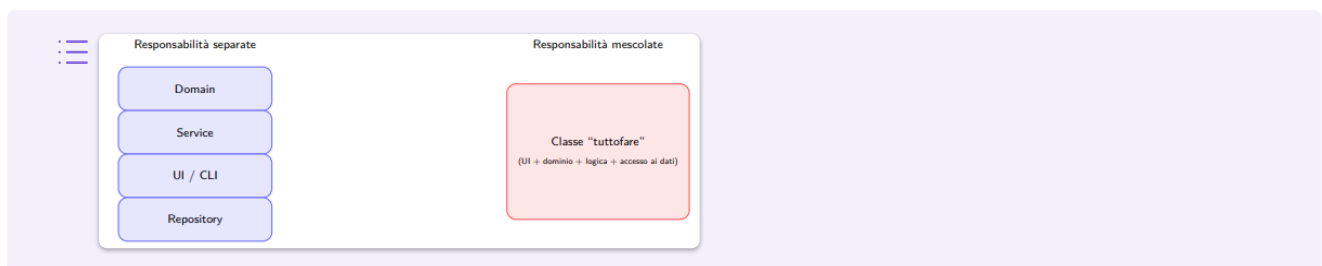
Possiamo quindi immaginare di avere, almeno concettualmente, un modulo per ciascuna di queste aree, che si occupa di quella parte del dominio. Questa è una prima forma di modularità, che poi andremo a raffinare con il principio della separazione delle responsabilità.



Secondo principio- separazione delle responsabilità: La separazione delle responsabilità invita a evitare componenti che fanno “un po' di tutto”. Ogni parte del sistema dovrebbe avere una responsabilità principale ben riconoscibile:

- rappresentare dati del dominio,
- applicare regole di business,
- comunicare con l'utente,
- accedere al database.

Quando questi ruoli si mescolano, il codice diventa più difficile da capire, da testare e da modificare.



Perché separare le responsabilità: Una separazione chiara delle responsabilità offre diversi vantaggi:

- rende più semplice capire “a chi compete” un certo cambiamento,
- riduce il rischio di introdurre effetti collaterali non desiderati,
- permette di sostituire una parte (es. la persistenza) mantenendo le altre stabili.

Al contrario, una classe o un modulo che gestisce:

- logica del dominio,
- interfaccia utente,
- accesso ai file

è destinato a diventare un punto critico del sistema.

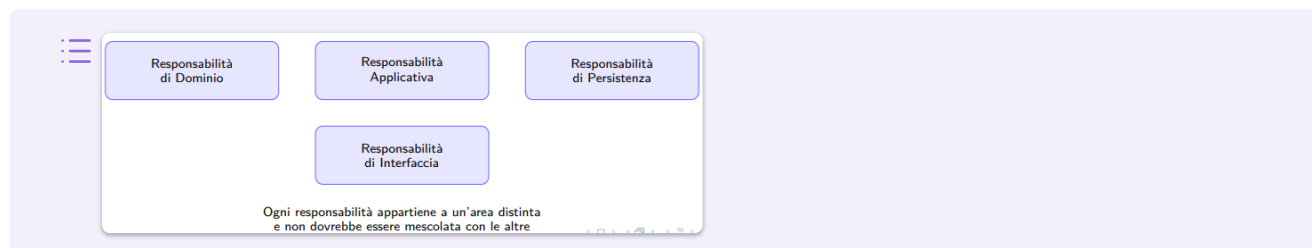
Tipi di responsabilità in UniManager: In questa slide adottiamo una vista per responsabilità tecniche (non per aree funzionali):

- è una prospettiva diversa da “modularità funzionale” (Studenti, Corsi, Appelli),
- serve a chiarire chi fa cosa tra i componenti tecnici.

In UniManager possiamo individuare almeno quattro tipi di responsabilità:

- dominio: rappresentare entità e regole del mondo universitario,
- logica applicativa: orchestrare i casi d'uso,
- persistenza: leggere e scrivere dati,
- interfaccia: interagire con l'utente.

Separare queste responsabilità ci aiuta a progettare componenti più puliti e facili da testare.



Separazione in pratica- entità, servizi, repository, CLI: Possiamo tradurre queste responsabilità in quattro famiglie di componenti:

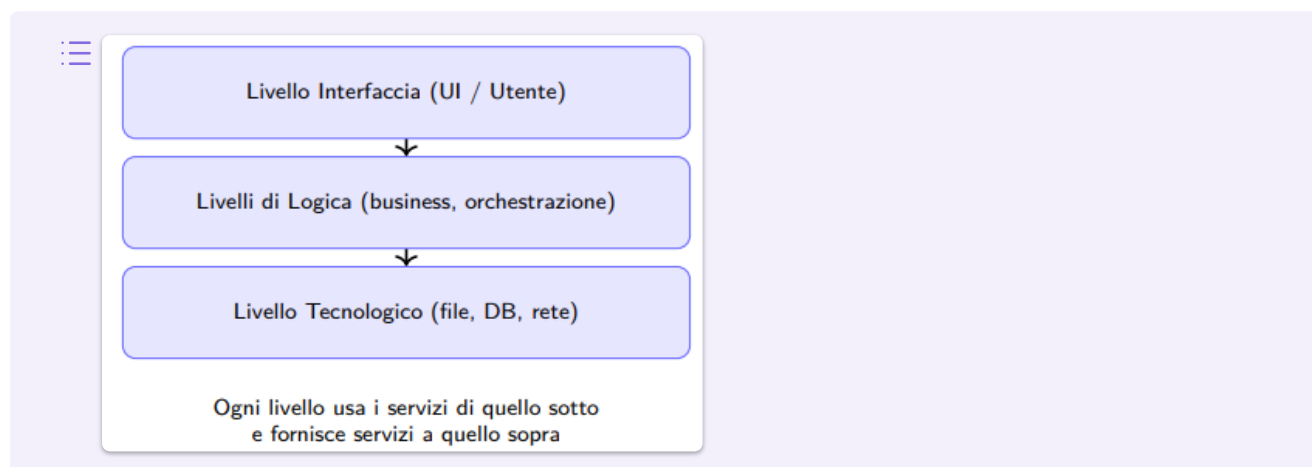
- *entità del dominio* (es. Studente, Corso, Appello), che contengono dati e regole del dominio;
- *servizi applicativi* (es. AppelloService), che implementano le operazioni richieste dai casi d'uso;
- *repository* (es. StudenteRepository), che si occupano di salvare e recuperare i dati;
- *componenti CLI/UI*, che gestiscono input e output verso l'utente.

In questo modo, ogni famiglia ha una responsabilità precisa e riconoscibile.

Terzo principio- layering: Il terzo principio riguarda il layering, cioè l'idea di organizzare il sistema in livelli di astrazione:

- un livello più vicino all'utente (interfaccia),
- livelli intermedi di logica,
- un livello più vicino alle tecnologie di base (file, database, rete).

Ogni livello offre servizi al livello superiore e sfrutta i servizi del livello inferiore, secondo regole di dipendenza ben precise.



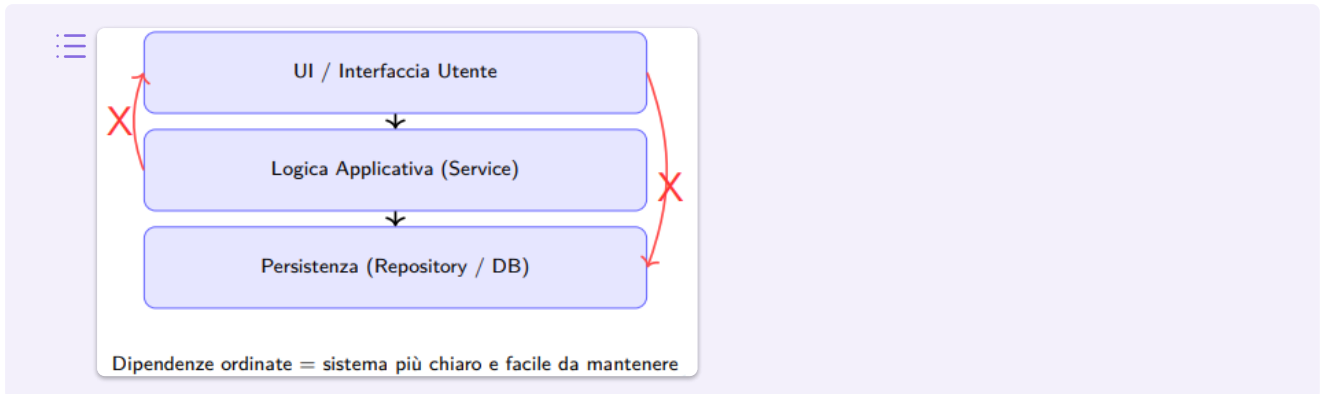
Perché usare i layer: I layer servono a tenere sotto controllo le dipendenze:

- il codice dell'interfaccia utente non dovrebbe conoscere i dettagli del database,

- la logica di business non dovrebbe dipendere da come i dati sono memorizzati.

I benefici principali sono:

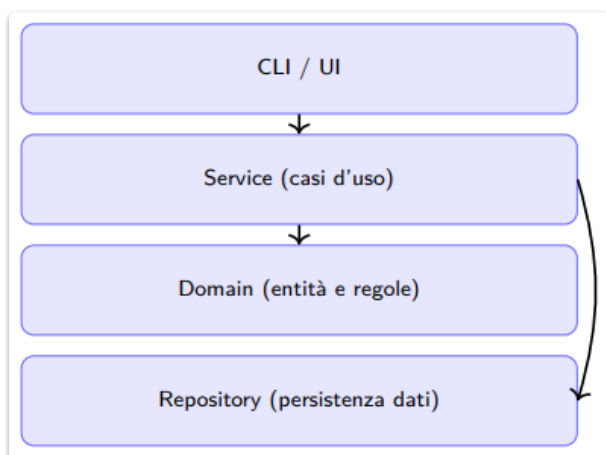
- maggiore chiarezza nella struttura,
- confini più netti per i test,
- facilità nel sostituire un livello (es. cambiare DB) mantenendo stabili gli altri.



Un layering semplice per UniManager: Per UniManager possiamo immaginare quattro livelli principali:

- **CLI/UI:** comandi testuali o schermate che l'utente utilizza;
- **Service:** implementazione dei casi d'uso;
- **Domain:** entità e regole del dominio universitario;
- **Repository:** accesso ai dati su file o database.

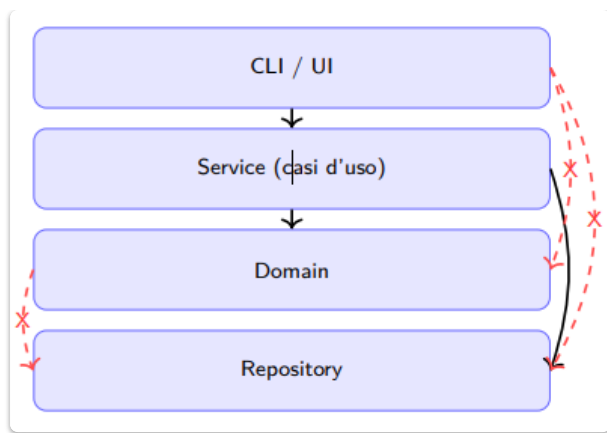
Questi livelli formano una “pila” dove ciascuno ha un ruolo distinto.



Regole di comunicazione tra i layer: Per mantenere l'architettura pulita, è utile stabilire alcune regole:

- la CLI/UI parla solo con il livello Service,
- i Service parlano con il Domain e i Repository,
- i Repository non devono dipendere dall'interfaccia utente,
- il Domain non dovrebbe conoscere dettagli di persistenza o di interfaccia.

In questo modo, le dipendenze hanno una direzione chiara e il dominio rimane al centro dell'architettura.



Checklist rapida di qualità architetturale: Domande operative da porre al modello architetturale:

- ogni componente ha una responsabilità principale chiara?
- le dipendenze rispettano i confini tra layer?
- una modifica tipica richiede tocchi localizzati o diffusi?
- il dominio resta isolato da UI e persistenza?
- i test possono concentrarsi su un livello alla volta?

Se molte risposte sono negative, l'architettura va rifinita prima di ampliare il codice.

UniManager- una prima suddivisione in pacchetti: Possiamo tradurre questi livelli in una prima proposta di struttura del progetto:

- cli, per l'interfaccia testuale o grafica,
- service, per i servizi applicativi,
- domain, per le entità e le regole del dominio,
- repository, per l'accesso ai dati.

Questa suddivisione ha il vantaggio di:

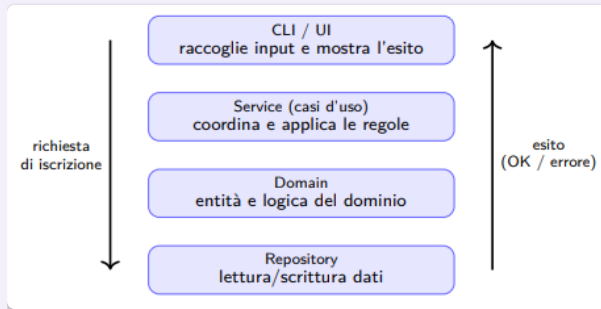
- rendere subito visibile la separazione delle responsabilità,
- dare una mappa chiara del sistema fin dai primi commit.



≡ Iscrizione a un appello attraverso i layer. Immaginiamo l'operazione "Studente si iscrive a un appello":

- la CLI raccoglie l'input (matricola, appello) e invoca il servizio opportuno;
- il Service verifica le regole (esistenza di studente e appello, vincoli sulle iscrizioni) usando il Domain;
- il Service chiede al Repository di aggiornare la persistenza;
- il risultato torna verso la CLI, che mostra all'utente conferma o eventuali errori.

La responsabilità di ciascun passo è chiara e localizzata nel layer corretto.



Parte 9 - Stili Architetaturali

Nella lezione precedente abbiamo visto:

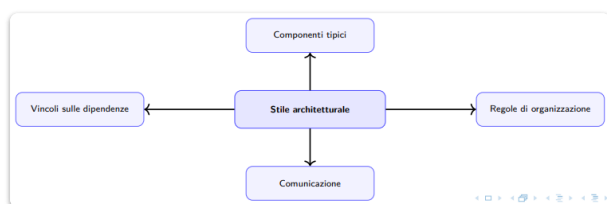
- che cos'è l'architettura software,
- perché è importante pensarla esplicitamente,
- tre principi chiave: modularità, separazione delle responsabilità, layering.

Oggi facciamo un passo avanti: vediamo come questi principi possono essere combinati in forme ricorrenti, cioè in stili architetaturali. Obiettivo: dare un vocabolario per riconoscere le strutture tipiche dei sistemi e per discutere le scelte senza basarsi solo su tecnologie o mode del momento.

Che cos'è uno stile architetaturale? Uno stile architetaturale descrive:

- un insieme di componenti tipici,
- le regole di organizzazione (es. livelli, ruoli),
- le modalità di comunicazione tra i componenti,
- i vincoli sulle dipendenze.

Non è una tecnologia o un framework, è un modello concettuale che può essere implementato in molti modi diversi. Serve per ragionare in termini di forme globali del sistema, non di singole classi.



Perché esistono più stili? I sistemi software devono soddisfare esigenze molto diverse: interattività, prestazioni, scalabilità, affidabilità, integrazione con altri sistemi, ... Ogni stile è nato per rispondere meglio a alcuni di questi bisogni. Non esiste uno stile "migliore in assoluto": ogni scelta comporta vantaggi e svantaggi, capire questi compromessi è parte del mestiere dell'ingegnere del software.

In questa lezione ci concentriamo su quattro stili molto diffusi:

1. **Layered**: organizzazione a livelli (es. presentazione, logica, dati).
2. **Client-server**: separazione tra chi chiede un servizio e chi lo fornisce.

3. **Repository:** più componenti che condividono una base dati comune.
4. **Service-based:** sistema composto da servizi indipendenti che espongono interfacce.

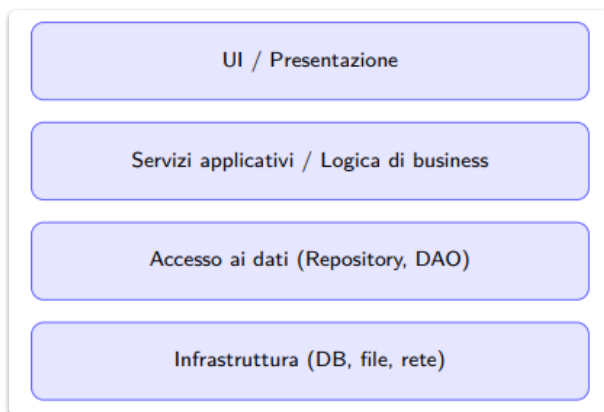
Architettura layered: Lo stile layered organizza il sistema in livelli di astrazione:

- ogni livello ha un ruolo specifico,
- i livelli comunicano tra loro secondo regole definite.

Tipicamente si distinguono livelli come:

- presentazione verso l'utente,
- logica di business / servizi applicativi,
- accesso ai dati e infrastruttura di base.

L'idea chiave è tenere separati i diversi tipi di responsabilità e controllare le dipendenze tra le parti del sistema. In pratica, layered risponde bene alla domanda: dove deve vivere una certa logica, per evitare che UI, regole di business e persistenza si mescolino. Non impone una tecnologia specifica: è uno schema organizzativo che può essere applicato in modo simile in stack diversi.



Ogni layer offre servizi a quello sopra e usa i servizi di quello sotto. Le dipendenze seguono una direzione prevalente: dall'alto verso il basso.

Layered- vantaggi:

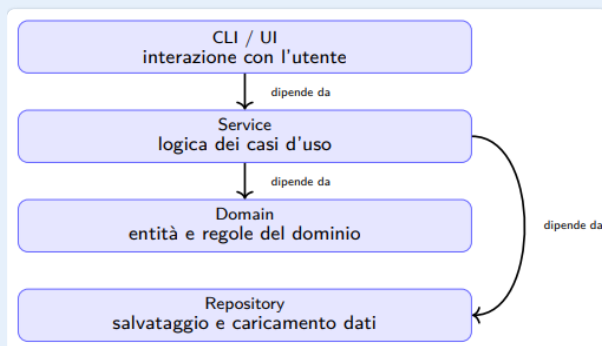
1. **Chiarezza strutturale:**
 - è facile spiegare “come è fatto” il sistema,
 - i confini tra responsabilità sono visibili.
2. **Facilità di manutenzione:**
 - cambiare il database può richiedere modifiche solo nel layer di accesso ai dati,
 - cambiare UI dovrebbe toccare principalmente il layer di presentazione.
3. **Testabilità:**
 - si possono testare i servizi senza coinvolgere né UI né DB reali (mock).
4. **Evoluzione controllata:**
 - una modifica in un layer tende a propagarsi meno agli altri, se le interfacce restano stabili.
5. **Onboarding del team:**
 - la struttura a livelli aiuta a capire rapidamente dove collocare nuova logica.

Layered: limiti e compromessi:

1. *Possibile overhead:*
 - le chiamate attraversano più layer, con qualche costo in termini di prestazioni.
2. *Rischio di rigidità:*
 - se i confini tra layer sono troppo rigidi, diventa difficile gestire casi "trasversali".
3. *Necessità di disciplina:*
 - se i layer iniziano a "bucarsi" (UI che accede direttamente al DB), la struttura si degrada in fretta.
4. *Rischio di stratificazione artificiale:*
 - in sistemi piccoli, troppi layer possono introdurre complessità non necessaria.
5. *Possibile duplicazione di modelli:*
 - spesso gli stessi dati attraversano più layer con trasformazioni ripetitive.

i Per UniManager possiamo riprendere il layering già introdotto:

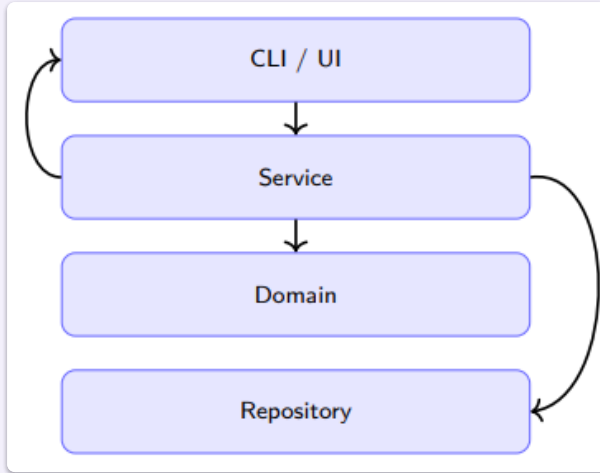
- CLI / UI: interazione con l'utente (comandi, menu).
- Service: implementazione dei casi d'uso (iscrizione, creazione appelli, ...).
- Domain: entità e regole del dominio universitario.
- Repository: accesso ai dati su file o database.



≡ Iscrizione a un appello nei layer

- Scenario «Studente si iscrive a un appello»:
 1. CLI/UI chiede matricola e appello e invoca AppelloService.
 2. Service controlla i vincoli usando oggetti del Domain.
 3. Service chiede al Repository di aggiornare la persistenza.
 4. CLI/UI mostra il risultato (successo o errore).

- Il flusso attraversa i layer in modo ordinato, con responsabilità chiare



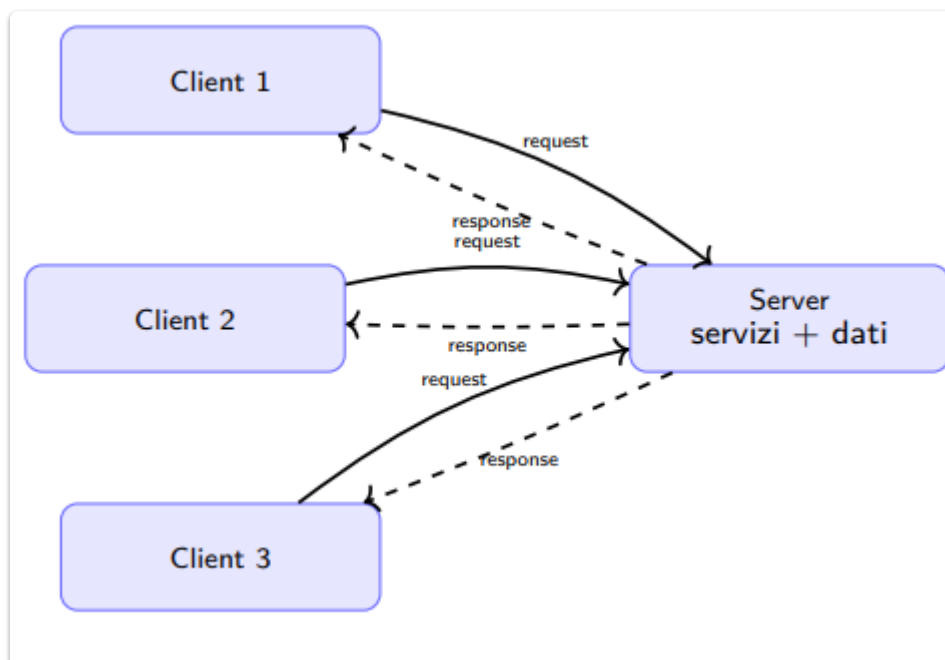
Architettura client-server: Lo stile client-server separa:

- il client, che richiede servizi,
- il server, che fornisce servizi.

Tipicamente:

- il server è centrale, mantiene dati e logica,
- i client sono distribuiti, interagiscono con il server via rete.

È il modello concettuale alla base di molte applicazioni web e sistemi distribuiti. La separazione client/server rende esplicito un confine importante: l'esperienza utente sta lato client, mentre regole e dati condivisi stanno lato server. Questo stile è spesso il primo passo quando un sistema locale deve diventare multiutente.



I client inviano richieste (request), il server risponde (response).

Client-serverà- vantaggi:

- **Centralizzazione:**
 - dati e logica critici restano concentrati sul server,
 - più facile aggiornare e controllare il sistema.

- **Multi-client:**
 - lo stesso server può servire molti client diversi (desktop, web, mobile).
- **Sicurezza:**
 - le regole di accesso ai dati sono gestite lato server.
- **Governance semplificata:**
 - aggiornamenti di logica e policy avvengono centralmente, senza distribuire patch su ogni client.
- **Riuso dei servizi:**
 - più canali (web, mobile, desktop) possono sfruttare lo stesso backend.

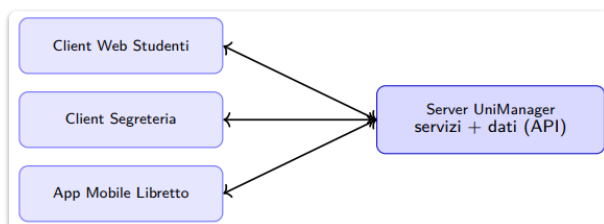
Client-server- limiti e compromessi:

- **Single point of failure:**
 - se il server è giù, tutti i client sono bloccati.
- **Scalabilità:**
 - se il numero di client cresce molto, il server può diventare un collo di bottiglia.
- **Complessità di rete:**
 - bisogna gestire latenza, errori di comunicazione, autenticazione, ...
- **Dipendenza dalla connettività:**
 - in assenza di rete, il client può perdere funzionalità critiche.
- **Scelte di stato/sessione:**
 - la gestione dello stato lato server o lato client impatta scalabilità e semplicità.

Al momento, UniManager è pensato come applicazione locale (es. CLI). Potremmo però immaginare un'evoluzione:

- un server UniManager che espone servizi (API),
- diversi client:
 - applicazione web per gli studenti,
 - applicazione interna per la segreteria,
 - app mobile per consultare il libretto.

In questo scenario, lo stile client-server diventa il modello concettuale di riferimento.

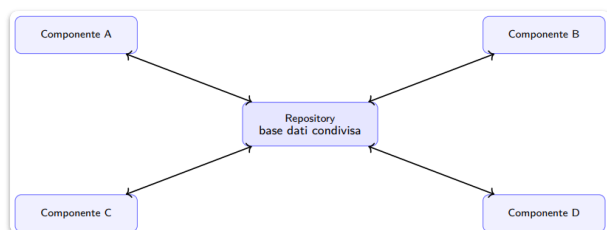


Architettura repository: Nello stile repository, più componenti condividono un'unica base dati comune. Il repository è il “cuore” dove risiedono le informazioni condivise. Intorno ad esso ci sono componenti che:

- elaborano dati,
- offrono funzionalità,
- reagiscono ai cambiamenti nel repository.

La priorità di questo stile è la coerenza informativa: più moduli diversi lavorano sullo stesso patrimonio dati. È adatto quando il valore principale del sistema sta nei dati condivisi più che nel comportamento di un singolo componente.

Repository: Nello stile repository, tutti i componenti ruotano attorno a un'unica base dati condivisa. Ogni componente legge e aggiorna il repository, ma rimane indipendente dagli altri.



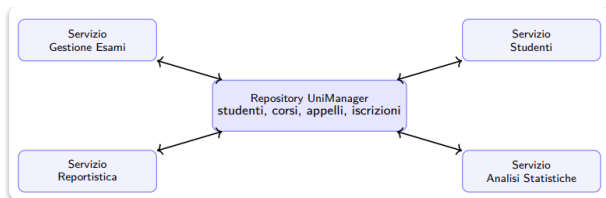
Repository- vantaggi:

- **Consistenza dei dati:**
 - una sola fonte di verità per le informazioni condivise.
- **Condivisione:**
 - sottosistemi diversi possono cooperare accedendo agli stessi dati.
- **Evoluzione dei componenti:**
 - è possibile sostituire o aggiornare i componenti attorno al repository mantenendo stabile la base dati (entro certi limiti).
- **Tracciabilità:**
 - avere un punto dati centrale facilita audit, report e analisi trasversali.
- **Integrazione:**
 - nuovi componenti possono essere aggiunti collegandosi al repository condiviso.

Repository- limiti e compromessi:

- **Punto critico di fallimento:**
 - se il repository si blocca o viene corrotto, l'intero sistema è in difficoltà.
- **Scalabilità e prestazioni:**
 - molte richieste concorrenti possono creare colli di bottiglia.
- **Dipendenza forte dal modello dati:**
 - cambiare lo schema del repository può impattare molti componenti.
- **Accoppiamento semantico:**
 - componenti nominalmente separati possono diventare dipendenti dagli stessi dettagli di schema.
- **Migrazioni delicate:**
 - evolvere il repository richiede coordinamento tra molti moduli e finestre di rilascio ben pianificate.

In UniManager abbiamo concettualmente una base dati universitaria: studenti, corsi, appelli, iscrizioni, voti, ... Per ora la implementiamo in modo semplice (file / strutture in memoria), ma concettualmente è un repository informativo condiviso. In una versione più evoluta: servizi diversi (es. gestione esami, reportistica, analisi statistiche) potrebbero accedere allo stesso repository.



Architettura service-based: Nello stile service-based, il sistema è costituito da servizi:

- ciascun servizio è un componente indipendente,
- espone una interfaccia ben definita,
- può essere sviluppato, distribuito e aggiornato separatamente.

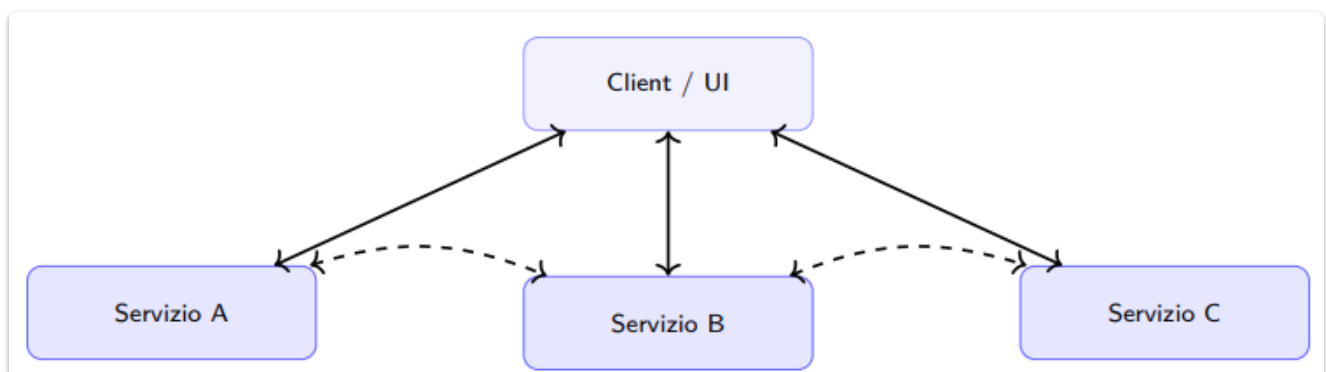
I servizi comunicano tra loro tramite chiamate di rete (es. API). Ogni servizio presidia una responsabilità funzionale chiara: questo riduce l'impatto dei cambiamenti e favorisce l'evoluzione incrementale. L'attenzione si sposta dai confini tra classi ai contratti tra servizi (endpoint, payload, versioni).

Da client-server a service-based: Lo stile service-based può essere visto come una evoluzione del client-server:

- invece di un unico server "monolitico", abbiamo più servizi specializzati,
- ogni servizio può avere i propri dati e responsabilità.

In UniManager:

- servizio StudenteService per la gestione studenti,
- servizio AppelloService per appelli e voti,
- servizio ReportingService per report e statistiche.



Service-based:

- Il client/UI parla con più servizi indipendenti, ciascuno con una responsabilità ben definita.
- I servizi possono anche collaborare tra loro tramite chiamate di rete (linee tratteggiate).

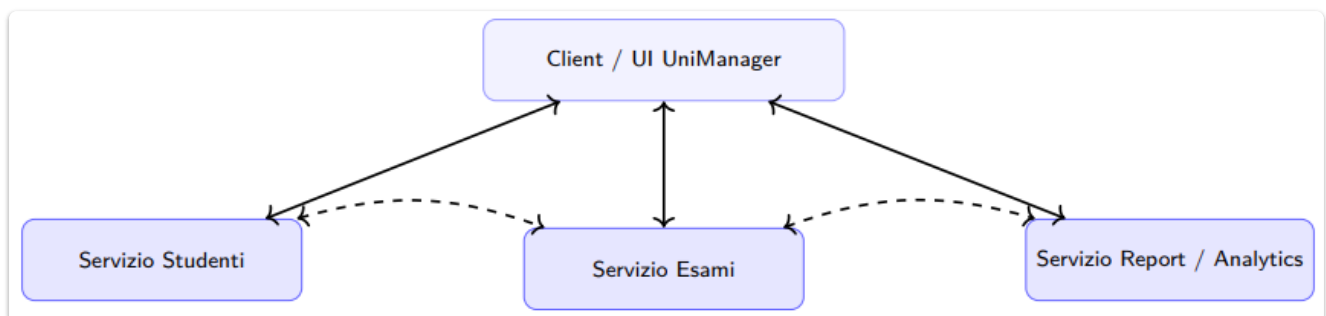
Service-based- vantaggi:

- **Scalabilità:**
 - ogni servizio può essere scalato indipendentemente (più istanze del solo servizio critico).
- **Autonomia dei team:**

- team diversi possono sviluppare e rilasciare servizi separati.
- *Evoluzione tecnologica:*
 - servizi diversi possono usare tecnologie diverse, purché rispettino i contratti di interfaccia.
- *Isolamento dei guasti:*
 - un problema in un servizio può avere impatto limitato, se i fallback sono progettati bene.
- *Rilasci mirati:*
 - è possibile aggiornare un singolo servizio senza rilasciare l'intero sistema.

Service-based- limiti e compromessi:

- *Maggiore complessità complessiva:*
 - bisogna gestire comunicazione tra servizi, fallimenti parziali, versioni delle API.
- *Osservabilità e debugging più difficili:*
 - un flusso di richiesta può attraversare molti servizi diversi.
- *Coerenza dei dati:*
 - se ogni servizio ha il proprio storage, garantire consistenza diventa più complesso.
- *Transazioni distribuite:*
 - operazioni che coinvolgono più servizi richiedono sequenze coordinate e azioni di annullamento in caso di errore.
- *Costo operativo:*
 - servono strumenti maturi per monitoraggio, tracing, deployment e gestione API.



Per il corso manteniamo UniManager semplice e monolitico (layered). Ma possiamo usare il modello service-based per:

- discutere come dividere il sistema in aree funzionali indipendenti,
- ragionare su possibili evoluzioni future (es. servizi separati per studenti, esami, analytics),
- introdurre l'idea di contratti tra componenti (interfacce ben definite).

Stili a confronto Ogni stile risponde meglio a una domanda diversa:

- *Layered:* come separare responsabilità interne e tenere ordinate le dipendenze?
- *Client-server:* come distribuire il sistema tra client e server condiviso?
- *Repository:* come far cooperare componenti diversi sugli stessi dati?
- *Service-based:* come far evolvere parti del sistema in modo indipendente?

Sintesi operativa:

- priorità a manutenibilità interna e chiarezza: *Layered*;

- priorità a multiutente via rete: *Client-server*;
- priorità a consistenza e patrimonio dati comune: *Repository*;
- priorità a scalabilità organizzativa e rilasci indipendenti: *Service-based*.

Nei sistemi reali la scelta è spesso ibrida, ma i confini devono restare espliciti.

Ogni stile ha impatti diversi su:

- *prestazioni*,
- *scalabilità*,
- *affidabilità*,
- *manutenibilità*,
- *evolutività*.

Example

- layered aiuta molto la manutenibilità e la testabilità;
- client-server e service-based introducono nuove sfide di prestazioni e affidabilità di rete;
- repository facilita la consistenza, ma crea possibili colli di bottiglia.

Come scegliere uno stile (o più di uno): Nella pratica, i sistemi reali sono spesso una combinazione di stili:

- un'applicazione web può essere client-server e internamente layered,
- più servizi possono condividere un repository o avere ognuno il proprio.

Alcune domande utili:

- Quanto è importante la manutenibilità interna? (layered)
- Il sistema è distribuito o locale? (client-server / service-based)
- I dati condivisi sono il vero cuore del sistema? (repository)
- Ci sono team diversi che devono lavorare in modo indipendente? (service-based)

Sintesi per UniManager in questa fase:

- base layered con repository centrale dei dati,
- client-server e service-based come possibili evoluzioni successive.

Parte 10 - Principi SOLID

Finora ci siamo concentrati su:

- principi architetturali generali (modularità, separazione delle responsabilità, layering),
- stili architetturali (layered, client-server, repository, service-based).

Oggi scendiamo di livello: come progettare le classi e le loro dipendenze, in modo coerente con l'architettura scelta. L'obiettivo è quello di collegare le decisioni locali (sulle classi) alla struttura globale del sistema

Perché servono principi di design? Il codice non deve solo funzionare oggi: deve poter evolvere senza degradarsi. Quando il progetto cresce, emergono problemi ricorrenti:

- classi con troppe responsabilità e confini poco chiari;
- dipendenze rigide che propagano gli effetti di ogni modifica;
- regressioni frequenti e test costosi da mantenere.

In queste condizioni, il costo principale non è scrivere nuova logica, ma capire e cambiare quella esistente. I principi di design (come **SOLID**) servono come criteri operativi per ridurre accoppiamento, aumentare coesione e migliorare testabilità.

Che cos'è SOLID? SOLID è un acronimo che indica cinque principi:

1. **Single Responsibility Principle (SRP)**: principio di responsabilità singola
2. **Open/Closed Principle (OCP)**: principio aperto/chiuso
3. **Liskov Substitution Principle (LSP)**: principio di sostituzione di Liskov
4. **Interface Segregation Principle (ISP)**: principio di segregazione delle interfacce
5. **Dependency Inversion Principle (DIP)**: principio di inversione delle dipendenze

Non sono leggi assolute, ma linee guida per prendere decisioni migliori.

SRP – Single Responsibility Principle: Una classe dovrebbe avere un solo motivo per cambiare. Per “motivo per cambiare” si intende una famiglia omogenea di modifiche (stesso tipo di responsabilità). Quando una classe mescola responsabilità diverse:

- aumenta l'accoppiamento interno;
- crescono effetti collaterali e regressioni;
- test e manutenzione diventano più costosi.

Esempio in UniManager:

- una classe che contiene insieme regole d'esame, persistenza e messaggi UI viola SRP;
- queste responsabilità vanno separate in componenti distinti

In UniManager conviene distinguere almeno queste responsabilità:

- business: regole del dominio (vincoli su appelli, prerequisiti, esiti);
- persistenza: lettura/scrittura su file o database;
- presentazione: messaggi verso CLI/UI e formati di output;
- cross-cutting: log, audit, monitoraggio.

Criterio pratico per rilevare una violazione SRP: “quante ragioni indipendenti ho per modificare questa classe?”. Se la risposta è più di una, serve separare. L'obiettivo è quello di avere classi piccole ma coese, con confini chiari e test più mirati.

```

1  class GestoreEsami {
2
3      public void creaAppello(Corso corso, LocalDate data) {
4          // (1) Logica di business:
5          // controlla sovrapposizioni, scadenze, ecc.
6          // ...
7
8          // (2) Persistenza:
9          // salva l'appello su file
10         salvaSuFile(corso, data);
11
12         // (3) Presentazione:
13         // mostra un messaggio a video
14         stampaReportCreazione(corso, data);
15     }
16
17     private void salvaSuFile(Corso corso, LocalDate data) {
18         // codice di I/O su file...
19     }
20
21     private void stampaReportCreazione(Corso corso, LocalDate data) {
22         // System.out.println(...);
23     }
24 }

```

GestoreEsami mescola: *business (regole sugli appelli)*, persistenza (salvataggio su file), * presentazione (stampa su console). Cambia il formato di salvataggio? ⇒ modifico GestoreEsami. Cambia il modo di comunicare con l'utente? ⇒ modifico ancora GestoreEsami. Risultato: classe fragile, difficile da testare e da capire.

```

1  class ServizioEsami {
2
3      private ArchivioEsamiSuFile archivio;
4
5      public ServizioEsami(ArchivioEsamiSuFile archivio) {
6          this.archivio = archivio;
7      }
8
9      public void creaAppello(Corso corso, LocalDate data) {
10         // (1) Solo logica di business:
11         // controlli sulle regole degli appelli
12         // (niente I/O qui dentro!)
13
14         if (archivio.esisteAppelloPer(corso, data)) {
15             throw new IllegalStateException("Appello gia' esistente");
16         }
17
18         Appello appello = new Appello(corso, data);
19         archivio.salvaAppello(appello);
20     }
21 }

```

```

1  class ArchivioEsamiSuFile {
2
3      public void salvaAppello(Appello appello) {
4          // I/O su file...
5      }
6
7      public boolean esisteAppelloPer(Corso corso, LocalDate data) {
8          // controlla nel file...
9          return false;
10     }
11 }

```

Prima:

- UI → GestoreEsami (regole + salvataggio su file + stampa).
- Un'unica classe con responsabilità miste (business, persistenza, presentazione).

Dopo:

- UI → ServizioEsami (solo regole di business),
- ServizioEsami delega a ArchivioEsamiSuFile la persistenza,
- la UI (CLI) si occupa dei messaggi verso l'utente (stampe, errori, conferme).

Effetti:

- ogni classe ha un motivo principale per cambiare,
- il comportamento si capisce per "blocchi" separati,
- il design è pronto per introdurre OCP e DIP.

OCP – Open/Closed Principle: Il software dovrebbe essere aperto all'estensione, ma chiuso alla modifica. Interpretazione operativa:

- nuovo comportamento = aggiungere nuove classi/composizioni;
- codice stabile = evitare modifiche continue agli stessi metodi centrali.

OCP non significa "mai modificare nulla", ma ridurre le modifiche invasive nelle parti già consolidate.

Esempio in UniManager: aggiungere un nuovo criterio di ordinamento senza riscrivere ServizioEsami.

OCP nasce a causa di un problema ricorrente: il requisito resta lo stesso, ma cambiano le varianti (nuovi casi, nuove regole, nuove modalità). Se ogni variante richiede di riaprire lo stesso componente centrale:

- aumenta il rischio di regressioni;
- i test da rieseguire crescono a ogni modifica;
- quel componente diventa un collo di bottiglia del team.

OCP nasce per gestire la variabilità in modo più stabile:

- la variabilità va isolata in estensioni dedicate;
- il nucleo resta stabile e riusabile.

```
1  class ServizioEsami {
2
3      public List<Appello> ordina(List<Appello> appelli, String criterio) {
4
5          if (criterio.equals("data")) {
6              Collections.sort(appelli, new Comparator<Appello>() {
7                  public int compare(Appello a, Appello b) {
8                      return a.getData().compareTo(b.getData());
9                  }
10             });
11
12         } else if (criterio.equals("corso")) {
13             Collections.sort(appelli, new Comparator<Appello>() {
14                 public int compare(Appello a, Appello b) {
15                     return a.getCorso().getNome().compareTo(b.getCorso().getNome())
16                 }
17             });
18
19         } else if (criterio.equals("iscritti")) {
20             Collections.sort(appelli,
21                 new Comparator<Appello>() {
22                     public int compare(Appello a, Appello b) {
23                         return a.getNumIscritti() - b.getNumIscritti();
24                     }
25                 });
26         }
27
28         return appelli;
29     }
30 }
```

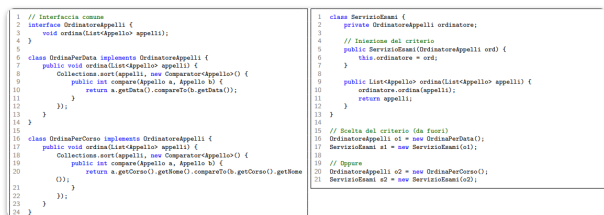
Metodo "monolitico" con tre comportamenti diversi. Ogni nuovo criterio richiede di:

- modificare ServizioEsami,
- aggiungere un nuovo blocco if/else. Questo viola OCP:
- il codice non è chiuso alla modifica,
- cresce nel tempo e diventa fragile. La logica di ordinamento è tutta concentrata qui, difficile da testare in isolamento.

Perché questa versione viola OCP? Ogni nuovo criterio di ordinamento richiede:

- di modificare ServizioEsami,
- di aggiungere un altro blocco if/else,
- di rischiare di rompere casi già funzionanti.

Criterio diagnostico: se una nuova variante obbliga a riaprire sempre lo stesso metodo, il design non è OCP. Il metodo cresce continuamente: code smell di variabilità concentrata.



```

1 // Interfaccia comune
2 interface OrdinatorAppelli {
3     void ordina(List<Appello> appelli);
4 }
5
6 class OrdinaPerData implements OrdinatorAppelli {
7     public void ordina(List<Appello> appelli) {
8         Collections.sort(appelli, new Comparator<Appello>() {
9             public int compare(Appello a, Appello b) {
10                 return a.getData().compareTo(b.getData());
11             }
12         });
13     }
14 }
15
16 class OrdinaPerCorso implements OrdinatorAppelli {
17     public void ordina(List<Appello> appelli) {
18         Collections.sort(appelli, new Comparator<Appello>() {
19             public int compare(Appello a, Appello b) {
20                 return a.getCorso().compareTo(b.getCorso());
21             }
22         });
23     }
24 }
25
26
27 class ServizioEsami {
28     private OrdinatorAppelli ordinatore;
29
30     // Iniezione del criterio
31     public ServizioEsami(OrdinatorAppelli ord) {
32         this.ordinatore = ord;
33     }
34
35     public List<Appello> ordina(List<Appello> appelli) {
36         ordinatore.ordina(appelli);
37         return appelli;
38     }
39 }
40
41 // Sostituisce il criterio (da fuori)
42 OrdinatorAppelli o1 = new OrdinaPerData();
43 ServizioEsami s1 = new ServizioEsami(o1);
44
45 // Oppure
46 OrdinatorAppelli o2 = new OrdinaPerCorso();
47 ServizioEsami s2 = new ServizioEsami(o2);

```

Separiamo il criterio di ordinamento dal servizio. OrdinatorAppelli descrive l'operazione in astratto. Ogni criterio (per data, per corso, per iscritti, . . .) diventa:

- una classe separata,
- aggiungibile senza modificare codice esistente.

ServizioEsami non cambia più:

- riceve un ordinatore dall'esterno,
- lo usa senza sapere come funziona.

Questo rispetta pienamente OCP:

- aperto a nuovi criteri: basta aggiungere una classe;
- chiuso alle modifiche: nessun if/else da aggiornare.

Condizioni operative: La soluzione resta OCP solo se il punto di estensione è stabile:

- interfaccia chiara (OrdinatorAppelli),
- servizio dipendente dall'astrazione, non dai criteri concreti.

Per introdurre una nuova variante:

- si aggiunge una classe (OrdinaPer...),
- si seleziona il criterio in composizione/iniezione,
- ServizioEsami resta invariato.

Checklist rapida OCP:

- una nuova variante richiede solo codice nuovo?
- i test esistenti restano validi senza riscritture massive?
- il comportamento variabile è confinato in componenti dedicati?

LSP – Liskov Substitution Principle: Le sottoclassi dovrebbero poter sostituire le superclassi senza cambiare il comportamento atteso del programma. In pratica:

- il codice client deve poter usare la sottoclasse al posto della superclasse senza accorgersene;
- stessa chiamata, stesso significato, stesso tipo di esito.

Segnali tipici di violazione:

- la sottoclasse rifiuta casi che la superclasse accetta;
- la sottoclasse introduce errori/eccezioni inattese;
- il client deve aggiungere controlli sul tipo concreto per evitare bug.

In molti sistemi OO usiamo ereditarietà e polimorfismo per riuso. Senza LSP, il polimorfismo diventa fragile:

- il chiamante si aspetta un comportamento della superclasse,
- alcune sottoclassi rompono quelle attese.

Effetti pratici:

- if speciali per distinguere sottoclassi,
- bug difficili da vedere in revisione,
- test incompleti perché dipendono dal tipo concreto.

```
1  class VerificatoreIscrizione {
2      public boolean puoIscrivere(Stucente s, Appello a) {
3          // Contratto atteso:
4          // false se non puo', true se puo'
5          return s.haPrerequisitiPer(a);
6      }
7  }
8
9  class VerificatoreSoloInCorso extends VerificatoreIscrizione {
10     @Override
11     public boolean puoIscrivere(Stucente s, Appello a) {
12         // Introduce un comportamento non previsto dalla superclasse
13         if (!a.isAperto()) {
14             // Eccezione non prevista dal comportamento della superclasse
15             throw new IllegalStateException("Appello chiuso");
16         }
17         return super.puoIscrivere(s, a) && s.isInCorso();
18     }
19 }
```

La

superclasse promette un contratto semplice:

- input valido,
- output booleano. La sottoclasse introduce un comportamento incompatibile: lancia eccezioni inattese. Un chiamante polimorfico può rompersi pur rispettando il contratto della base.

Perché questa versione viola LSP? La sottoclasse VerificatoreSoloInCorso restringe il contratto:

- aggiunge una preconditione implicita (appello aperto),
- cambia la modalità di errore (eccezione invece di false).

Chi usa il tipo base non è più protetto: serve conoscere il tipo concreto per evitare crash. Se serve un comportamento diverso, è meglio modellare una strategia, non una sottoclasse incompatibile.

```

1  interface VerificatoreIscrizione {
2      boolean puoIscrivere(Studente s, Appello a);
3  }
4
5  class VerificatoreStandard implements VerificatoreIscrizione {
6      public boolean puoIscrivere(Studente s, Appello a) {
7          // Stessa gestione del caso appello chiuso
8          if (!a.isAperto()) {
9              return false;
10         }
11
12         // Restituisce sempre boolean coerente con l'interfaccia
13         return s.haPrerequisitiPer(a);
14     }
15 }
16
17 class VerificatoreSoloInCorso implements VerificatoreIscrizione {
18     public boolean puoIscrivere(Studente s, Appello a) {
19         // Stessa gestione del caso appello chiuso
20         if (!a.isAperto()) {
21             return false;
22         }
23
24         // Restituisce sempre boolean coerente con l'interfaccia
25         return s.haPrerequisitiPer(a) && s.isInCorso();
26     }
27 }

```

Tutte le

implementazioni rispettano lo stesso contratto:

- nessuna eccezione inattesa,
- risultato sempre booleano. Le varianti cambiano la regola, non il tipo di comportamento osservabile. Il chiamante resta stabile e sostituibile.

La soluzione resta LSP solo se il contratto è uniforme:

- stessi input ammessi (niente precondizioni più forti nelle sottoclassi),
- stesso tipo di output/errore osservabile dal client.

Le varianti devono cambiare la regola, non il significato del comportamento atteso.
Checklist rapida LSP:

- il client funziona uguale con qualunque implementazione?
- servono if sul tipo concreto per evitare crash?
- una sottoclasse introduce eccezioni o vincoli non previsti?

ISP – Interface Segregation Principle: È meglio avere più interfacce piccole e specifiche che un'unica interfaccia “gigante” che fa di tutto. In pratica:

- ogni client dovrebbe vedere solo le operazioni del proprio ruolo;
- le funzionalità non pertinenti vanno in interfacce separate.

Violazioni tipiche:

- interfacce con molti metodi opzionali,
- classi client che devono implementare metodi “vuoti” o fittizi.

Problema ricorrente:

- un'unica interfaccia viene usata da ruoli diversi (studente, docente, segreteria),
- col tempo accumula metodi eterogenei.

Effetti pratici:

- i client dipendono da operazioni che non usano;
- compaiono metodi vuoti o `UnsupportedOperationException`;
- una modifica locale impatta molti componenti non coinvolti.

ISP nasce per separare le interfacce per ruolo/scenario e ridurre l'accoppiamento inutile.

```

1  interface PortaleDidattico {
2      void iscriviAppello(String matricola, String idAppello);
3      void consultaLibretto(String matricola);
4      void registraVoto(String idAppello, String matricola, int voto);
5      void creaAppello(String idCorso, LocalDate data);
6      void generaReportCorso(String idCorso);
7  }
8
9  class ClientStudente implements PortaleDidattico {
10     public void iscriviAppello(String m, String a) {
11         /* ok */
12     }
13
14     public void consultaLibretto(String m) {
15         /* ok */
16     }
17
18     public void registraVoto(String a, String m, int v) {
19         throw new UnsupportedOperationException();
20     }
21
22     public void creaAppello(String c, LocalDate d) {
23         throw new UnsupportedOperationException();
24     }
25
26     public void generaReportCorso(String c) {
27         throw new UnsupportedOperationException();
28     }
29 }

```

`ClientStudente` dipende da metodi solo docenti/segreteria. Implementa metodi non pertinenti con eccezioni fittizie. Ogni cambiamento dell'interfaccia impatta tutti i client.

La violazione nasce da un problema di modellazione: `PortaleDidattico` mescola responsabilità di ruoli diversi in un unico contratto. Effetto sui client:

- ogni client dipende anche da metodi che non dovrebbe conoscere;
- compaiono implementazioni fittizie (`UnsupportedOperationException`).

Conseguenza pratica: modifiche pensate per un ruolo impattano anche client di altri ruoli. Correzione: separare l'interfaccia per responsabilità funzionali (Studente, Docente, Segreteria).

```

1 interface FunzioniStudiante {
2     void iscriviAppello(String matricola, String idAppello);
3     void consultaLibretto(String matricola);
4 }
5
6 interface FunzioniDocente {
7     void registraVoto(String idAppello, String matricola, int voto);
8     void creaAppello(String idCorso, LocalDate data);
9 }
10
11 interface FunzioniSegreteria {
12     void generaReportCorso(String idCorso);
13 }
14
15 class ClientStudiante implements FunzioniStudiante {
16     public void iscriviAppello(String m, String a) { /* ... */ }
17
18     public void consultaLibretto(String m) { /* ... */ }
19 }
20
21 class ClientDocente implements FunzioniDocente {
22     public void registraVoto(String idAppello, String m, int voto) { /* ... */ }
23
24     public void creaAppello(String idCorso, LocalDate data) { /* ... */ }
25 }
26
27 class ClientSegreteria implements FunzioniSegreteria {
28     public void generaReportCorso(String idCorso) { /* ... */ }
29 }

```

Ogni

client dipende solo da ciò che usa davvero. I contratti sono più piccoli e chiari. Le evoluzioni su un ruolo non forzano modifiche sugli altri.

La soluzione resta ISP se le interfacce sono coerenti con i ruoli:

- interfacce distinte per Studente, Docente, Segreteria;
- ogni client implementa solo metodi pertinenti.

Quando aggiungi una funzionalità:

- modifichi solo l'interfaccia del ruolo coinvolto;
- eviti impatti trasversali sui client non interessati.

Checklist rapida ISP:

- compaiono metodi vuoti o UnsupportedOperationException?
- un client dipende da operazioni che non usa?
- una modifica locale forza cambi in client di altri ruoli

DIP – Dependency Inversion Principle: I moduli di alto livello e quelli di basso livello dovrebbero dipendere da astrazioni; i dettagli concreti dovrebbero dipendere da tali astrazioni. In pratica:

- alto livello = logica applicativa/business, più astratta (es. ServizioEsami);
- basso livello = dettagli tecnici, più concreti (es. ArchivioEsamiSuFile);
- entrambi si collegano tramite un'astrazione comune (es. ArchivioEsami).

Effetto: il business resta stabile anche se cambiano tecnologie e componenti infrastrutturali.

Problema ricorrente:

- i servizi applicativi creano direttamente classi infrastrutturali (new Archivio..., new ClientHttp...);
- quindi il livello alto dipende direttamente dal livello basso.

Effetti pratici:

- cambiare tecnologia richiede modifiche nel business;
- i test diventano lenti e fragili perché dipendono da risorse reali;
- cresce l'accoppiamento tra livelli che dovrebbero restare separati.

DIP nasce per invertire questa dipendenza:

- il business dipende da interfacce stabili,
- i dettagli concreti si agganciano a quelle interfacce.

Mappa dei ruoli (in UniManager):

- alto livello: ServizioEsami (regole applicative);
- basso livello: ArchivioEsamiSuFile (I/O su file);
- astrazione: ArchivioEsami.

Regola DIP: ServizioEsami deve dipendere da ArchivioEsami, non da ArchivioEsamiSuFile. Se dipende dal dettaglio concreto: cambiare persistenza (file, DB, memoria) richiede modifiche nel servizio. DIP mantiene pulito il confine tra business e infrastruttura.

```
1  class ArchivioEsamiSuFile {
2      private Path path;
3
4      public ArchivioEsamiSuFile(Path path) {
5          this.path = path;
6      }
7
8      public void salvaAppello(Appello appello) {
9          // I/O su file...
10     }
11 }
12
13 // Servizio che dipende da una classe concreta
14 class ServizioEsami {
15
16     private ArchivioEsamiSuFile archivio;
17
18     public ServizioEsami() {
19         // Dipendenza creata direttamente qui
20         this.archivio = new ArchivioEsamiSuFile(Paths.get("esami.txt"));
21     }
22
23     public void creaAppello(Corso corso, LocalDate data) {
24         // Logica di business (semplificata)
25         Appello appello = new Appello(corso, data);
26
27         // Persistenza: usa sempre il file esami.txt
28         archivio.salvaAppello(appello);
29     }
30 }
```

ServizioEsami dipende da ArchivioEsamiSuFile, una classe concreta. La scelta del file ("esami.txt") è codificata dentro il servizio. Problemi:

- per usare un database devo modificare ServizioEsami;
- per fare test devo scrivere davvero sul file;
- la logica di business è legata a un dettaglio di infrastruttura. Qui DIP è violato: un modulo di alto livello dipende da un modulo di basso livello.

Nel nostro esempio:

- alto livello: ServizioEsami;
- basso livello: ArchivioEsamiSuFile.

Violazione: l'alto livello dipende direttamente dal basso livello concreto. Il servizio crea direttamente la dipendenza: `new ArchivioEsamiSuFile(...)` dentro il costruttore. Effetto: ogni variazione infrastrutturale entra nel codice business. Criterio diagnostico: se per cambiare tecnologia devo modificare un servizio di dominio/applicazione, DIP non è rispettato.

```

1  // Astrazione nel dominio
2  interface ArchivioEsami {
3      void salvaAppello(Appello appello);
4      boolean esisteAppelloPer(Corso corso, LocalDate data);
5  }
6
7  // Implementazione concreta su file
8  class ArchivioEsamiSuFile implements ArchivioEsami {
9      public ArchivioEsamiSuFile(Path path) { /* salva path */}
10
11     @Override
12     public void salvaAppello(Appello appello) {
13         // I/O su file...
14     }
15
16     @Override
17     public boolean esisteAppelloPer(Corso corso, LocalDate data) {
18         // controlla nel file...
19         return false;
20     }
21 }
22
23 // Ora il servizio dipende dall'astrazione
24 class ServizioEsami {
25     private ArchivioEsami archivio;
26     public ServizioEsami(ArchivioEsami archivio) {
27         this.archivio = archivio;
28     }
29     public void creaAppello(Corso corso, LocalDate data) {
30         Appello appello = new Appello(corso, data);
31         archivio.salvaAppello(appello);
32     }
33 }

```

```

1  // Composizione: scelta concreta fuori dal servizio
2  Path path = Paths.get("esami.txt");
3  ArchivioEsami arch = new ArchivioEsamiSuFile(path);
4  ServizioEsami srv = new ServizioEsami(arch);
5

```

Spostiamo

l'interfaccia ArchivioEsami nel dominio (modulo più stabile). ServizioEsami dipende solo da ArchivioEsami, non da classi concrete. Le implementazioni concrete (SuFile, SuDb, InMemoria, . . .) dipendono dall'astrazione, non il contrario. Abbiamo invertito la dipendenza:

- il modulo di alto livello (service) non conosce più i dettagli di persistenza;

- i dettagli dipendono dall'astrazione comune.

La soluzione resta DIP se:

- il servizio dipende da ArchivioEsami (astrazione), non da ArchivioEsamiSuFile;
- la scelta dell'implementazione avviene all'esterno (composizione/iniezione).

Impatto diretto sui test:

- si può usare un'implementazione finta:
 - ArchivioEsamiInMemoria,
 - oppure un mock creato con un framework di test.
- i test non richiedono file reali, database o configurazioni esterne.

Checklist rapida DIP:

- il servizio conosce classi concrete infrastrutturali?
- la dipendenza concreta viene istanziata dentro il servizio?
- è possibile testare il servizio senza risorse esterne?

I principi SOLID: come lavorano insieme:

- **SRP**: mantiene coese le classi (una responsabilità principale per classe).
- **OCP**: consente nuove varianti senza modificare continuamente il nucleo stabile.
- **LSP**: rende affidabile la sostituzione tra superclasse e sottoclasse.
- **ISP**: evita interfacce troppo ampie e dipendenze inutili lato client.
- **DIP**: separa business e infrastruttura tramite astrazioni.

Insieme, questi principi supportano:

- manutenibilità,
- estendibilità,
- testabilità.

Applicare SOLID non significa:

- creare mille micro-classi inutili,
- astrarre tutto "per principio".

Significa:

- riconoscere code smell (classi sovraccariche, dipendenze rigide),
- avere strumenti concettuali per migliorare il design,
- trovare un equilibrio tra semplicità e flessibilità.

SOLID e refactoring: Spesso non si progetta "perfettamente" al primo colpo: SOLID aiuta a migliorare in modo incrementale. Workflow tipico di refactoring guidato da SOLID:

- individuare code smell e punti di accoppiamento eccessivo;
- chiarire responsabilità (SRP) e interfacce (ISP);
- isolare variabilità (OCP) e dipendenze infrastrutturali (DIP);
- verificare sostituibilità corretta tra tipi correlati (LSP).

Obiettivo: migliorare il design senza riscrivere tutto da zero.

Mini-checklist per UniManager:

1. SRP:
 - ci sono classi che fanno sia business che I/O?
 - posso separare persistenza, log, presentazione?
 2. OCP:
 - ci sono grandi if/switch su tipo o modalità?
 - posso spostare logiche variabili in classi dedicate, evitando di toccare sempre lo stesso metodo?
 3. LSP:
 - una sottoclasse può sostituire la superclasse senza rompere il client?
 - compaiono controlli sul tipo concreto o eccezioni inattese?
 4. ISP:
 - i client dipendono da metodi che non usano?
 - ci sono metodi vuoti o UnsupportedOperationException?
 5. DIP:
 - i servizi dipendono da classi concrete di repository?
 - posso introdurre interfacce nel dominio e implementazioni nei layer esterni?
-

Parte 11 - Design Patterns Essenziali: Strategy e Factory Method

Nella lezione precedente abbiamo visto tutti e cinque i principi SOLID. Questi principi ci dicono come dovrebbe essere un buon design:

- classi con ruoli chiari;
- codice estendibile senza modificare quello già scritto;
- dipendenze dirette da astrazioni e non da dettagli concreti.

Domande naturali:

- quali soluzioni concrete possiamo usare per mettere in pratica questi principi?
- come riconoscere schemi ricorrenti di buona progettazione?

In questa lezione vedremo due pattern che realizzano nella pratica questi principi:

- **Strategy** soprattutto a supporto di OCP,
- **Factory Method** molto legato a **DIP** (e a OCP sulla creazione).

Design pattern: I design pattern nascono da problemi che si ripetono:

- varianti di comportamento gestite con `if/else` crescenti;
- creazione di oggetti sparsa in molti punti del codice.

Un pattern fornisce:

- una struttura riusabile;
- un linguaggio condiviso nel team;
- una soluzione coerente con principi come OCP e DIP.

Criterio pratico:

- usare un pattern quando semplifica evoluzione e manutenzione;
- evitarlo quando aggiunge astrazione senza variabilità reale.

Oggi ci concentriamo su due pattern fondamentali e trasversali:

- **Strategy**: rendere intercambiabili i comportamenti (es. diversi criteri di ordinamento degli appelli in UniManager), in linea con OCP (nuovi comportamenti senza modificare il contesto).
- **Factory Method**: centralizzare e strutturare la creazione degli oggetti (es. creazione di diverse tipologie di ExamSession in UniManager), aiutando a rispettare DIP (dipendere da ExamSession, non dalle sue sottoclassi).

L'idea non è memorizzare definizioni, ma:

- riconoscere quando un problema è adatto a questi pattern;
- saperli usare per mantenere il codice pulito, estensibile e coerente con i principi SOLID visti nella lezione precedente.

Strategy: contesto tipico:

- una classe deve eseguire un certo comportamento;
- esistono varie alternative per implementarlo;
- vorremmo poter cambiare questo comportamento senza modificare la classe.

Strategy propone di:

- estrarre il comportamento in un'interfaccia separata;
- avere più strategie concrete intercambiabili;
- far sì che il contesto usi sempre l'interfaccia, non le implementazioni concrete.

Effetto:

- Strategy permette di iniettare comportamenti diversi senza toccare il codice del contesto.
- È uno strumento concreto per applicare OCP e ridurre dipendenze rigide verso implementazioni specifiche.

 grandi if/else per i comportamenti

Nell'esempio di UniManager vogliamo ordinare gli appelli:

- per data;
- per corso;
- per numero di iscritti.

```

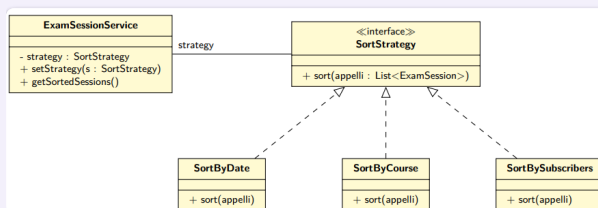
1  if (criterio == DATA) {
2      sortByDate(appelli);
3  } else if (criterio == CORSO) {
4      sortByCourse(appelli);
5  } else if (criterio == ISCRITTI) {
6      sortBySubscribers(appelli);
7  } else {
8      // ...
9  }

```

Problemi: ogni volta che vogliamo aggiungere un nuovo criterio dobbiamo modificare il metodo: il codice non è pensato per essere esteso facilmente. Il metodo, inoltre, cresce sempre di più ed è difficile da mantenere e testare.

Idea:

- definire un'interfaccia SortStrategy;
- avere implementazioni diverse per ogni criterio;
- lasciare che un ExamSessionService usi sempre una SortStrategy.



```

1 interface SortStrategy {
2     void sort(List<ExamSession> sessions);
3 }
4 class SortByDate implements SortStrategy {
5     @Override
6     public void sort(List<ExamSession> sessions) {
7         Collections.sort(sessions, new Comparator<ExamSession>() {
8             @Override
9             public int compare(ExamSession s1, ExamSession s2) {
10                return s1.getDate().compareTo(s2.getDate());
11            }
12        });
13    }
14 }
15 }

1 class SortByCourse implements SortStrategy {
2     @Override
3     public void sort(List<ExamSession> sessions) {
4         Collections.sort(sessions, new Comparator<ExamSession>() {
5             @Override
6             public int compare(ExamSession s1, ExamSession s2) {
7                 return s1.getCourse().compareTo(s2.getCourse());
8             }
9         });
10    }
11 }
12 }

14 class SortBySubscribers implements SortStrategy {
15     @Override
16     public void sort(List<ExamSession> sessions) {
17         Collections.sort(sessions, new Comparator<ExamSession>() {
18             @Override
19             public int compare(ExamSession s1, ExamSession s2) {
20                 return Integer.compare(
21                     s1.getNumberOfSubscribers(),
22                     s2.getNumberOfSubscribers());
23             }
24         });
25    }
26 }
27 }

```

```

1 class ExamSessionService {
2
3     private final List<ExamSession> sessions;
4     private SortStrategy strategy;
5
6     ExamSessionService(List<ExamSession> sessions) {
7         this.sessions = sessions;
8         this.strategy = new SortByDate(); // default
9     }
10
11     public void setSortStrategy(SortStrategy strategy) {
12         this.strategy = strategy;
13     }
14
15     public List<ExamSession> getSortedSessions() {
16         strategy.sort(sessions);
17         return new ArrayList<ExamSession>(sessions);
18     }
19 }

```

Il contesto non conosce le implementazioni concrete, usa sempre e solo l'interfaccia SortStrategy.

Strategie intercambiabili a runtime:

```

1 ExamSessionService service = new ExamSessionService(allSessions);
2
3 // ordinamento di default (per data)
4 List<ExamSession> byDate = service.getSortedSessions();
5
6 // cambio criterio a runtime
7 service.setSortStrategy(new SortByCourse());
8 List<ExamSession> byCourse = service.getSortedSessions();
9
10 service.setSortStrategy(new SortBySubscribers());
11 List<ExamSession> bySubscribers = service.getSortedSessions();

```

Il codice che usa ExamSessionService può:

- cambiare la strategia in base a una scelta dell'utente;
- selezionare la strategia da configurazione;
- sostituire le strategie senza modificare il servizio.

Secondo esempio Strategy: politiche di ammissione:

```
1 interface PoliticaAmmissione {
2     boolean ammesso(Studiante s, Appello a);
3 }
4
5 class PoliticaStandard implements PoliticaAmmissione {
6     public boolean ammesso(Studiante s, Appello a) {
7         return s.isApto() && s.hasPrerequisitiPer(a);
8     }
9 }
10
11 class PoliticaConMediaMinima implements PoliticaAmmissione {
12     public boolean ammesso(Studiante s, Appello a) {
13         return s.isApto()
14             && s.hasPrerequisitiPer(a)
15             && s.getMedia() >= 28.0;
16     }
17 }
```

```
1 class IscrizioneService {
2     private PoliticaAmmissione politica;
3
4     IscrizioneService(PoliticaAmmissione p) {
5         this.politica = p;
6     }
7
8     boolean puoiIscrivere(Studiante s, Appello a) {
9         return politica.ammesso(s, a);
10    }
11 }
```

```
1 PoliticaAmmissione p = new PoliticaConMediaMinima();
2 IscrizioneService srv = new IscrizioneService(p);
```

- Il contesto cambia politica senza modificare il servizio.
- Nuova regola: nuova classe, non nuovo `if/else`.

Terzo esempio Strategy: costo spedizione (e-commerce):

```
1 interface ShippingStrategy {
2     BigDecimal costoSpedizione(Ordine ordine);
3 }
4
5 class SpedizioneStandard implements ShippingStrategy {
6     public BigDecimal costoSpedizione(Ordine o) {
7         return new BigDecimal("5.90");
8     }
9 }
10
11 class SpedizioneEspress implements ShippingStrategy {
12     public BigDecimal costoSpedizione(Ordine o) {
13         return new BigDecimal("12.90");
14     }
15 }
```

```
1 class CheckoutService {
2     private ShippingStrategy shipping;
3
4     CheckoutService(ShippingStrategy shipping) {
5         this.shipping = shipping;
6     }
7
8     BigDecimal totale(Ordine ordine) {
9         return ordine.subtotale()
10             .add(shipping.costoSpedizione(ordine));
11    }
12 }
```

```
1 ShippingStrategy s = new SpedizioneEspress();
2 CheckoutService checkout = new CheckoutService(s);
```

- Cambia la politica di spedizione, non il CheckoutService.
- Stessa struttura del pattern, dominio completamente diverso.

Strategy: vantaggi principali (collegamento a SOLID):

- **OCP (Open/Closed):**
 - per aggiungere un nuovo criterio basta creare una nuova SortStrategy;
 - il contesto non si modifica: si estende senza toccare codice esistente.
- **DIP (Dependency Inversion):**
 - il contesto dipende dall'interfaccia SortStrategy, non dalle classi concrete;
 - riduce dipendenze rigide e rende la sostituzione delle strategie più semplice.
- **SRP (Single Responsibility):**
 - il contesto non deve "conoscere" tutti i criteri di ordinamento;
 - ogni criterio è confinato in una classe dedicata con un compito chiaro.
- **Testabilità:**
 - strategie testabili in isolamento; nel contesto si possono usare strategie fittizie.

Strategy: errori tipici (rischi su SOLID):

- **Introdurre Strategy senza varianti reali:**
 - aggiunge classi senza beneficio: aumenta complessità senza migliorare OCP/DIP.
- **Strategie troppo grandi o "onnivore":**
 - una Strategy dovrebbe incapsulare un comportamento specifico;
 - se dentro finisce "mezza applicazione", si perde SRP.
- **Contesto che continua a fare if/else sulle strategie:**

- es. controlli sul tipo concreto (instanceof) o switch sul criterio;
- si indebolisce OCP e spesso anche DIP.
- *Dipendenze concrete nelle strategie:*
 - se ogni Strategy istanzia direttamente servizi o repository concreti, si rischia una violazione di DIP (dipendenze dai dettagli).

Con Strategy abbiamo visto come rendere flessibili i comportamenti di un oggetto:

- criteri di ordinamento intercambiabili;
- possibilità di cambiare logica senza toccare il codice del servizio.

Ma in un sistema reale esiste un'altra fonte comune di rigidità: la creazione degli oggetti. Spesso troviamo:

- molti new sparsi per il codice;
- istanze create in modi diversi e difficili da uniformare;
- dipendenze concrete "nascoste" dentro i costruttori.

Domanda successiva naturale: possiamo rendere flessibile anche il modo in cui gli oggetti vengono creati?

Factory Method: Definiamo:

- un prodotto astratto (es. ExamSession);
- una o più sottoclassi concrete (es. WrittenSession, OralSession);
- una classe Creator che espone un metodo di fabbrica (il Factory Method) per creare il prodotto.

Il codice che usa il Creator:

- lavora solo con il tipo astratto;
- non conosce i dettagli delle classi concrete.

Effetto:

- la decisione su quale sottoclasse istanziare è concentrata nel Factory Method;
- il resto del codice può dipendere solo dal tipo astratto: questo va nella direzione di DIP e OCP.

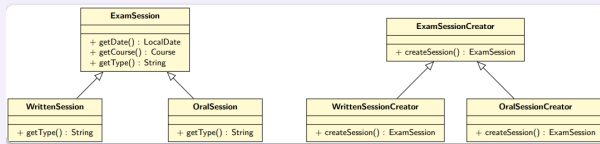
UniManager: creazione di ExamSession

Immaginiamo di avere varie tipologie di appello:

- WrittenSession (scritto);
- OralSession (orale);
- MixedSession (scritto + orale, non mostrata nei diagrammi per brevità).
- Senza Factory Method:
 - il codice farebbe new WrittenSession(...) o new OralSession(...) in molti punti diversi;
 - cambiare il modo in cui creiamo le sessioni richiederebbe modifiche in tante classi.

Con Factory Method:

- concentriamo la creazione in un creator (o pochi creator);
- il resto del sistema dipende solo da ExamSession.



Factory Method in Java: prodotto astratto:

```

1 abstract class ExamSession {
2
3     private final Course course;
4     private final LocalDate date;
5
6     protected ExamSession(Course course, LocalDate date) {
7         this.course = course;
8         this.date = date;
9     }
10
11     public Course getCourse() {
12         return course;
13     }
14
15     public LocalDate getDate() {
16         return date;
17     }
18
19     public abstract String getType();
20 }
21
22 class WrittenSession extends ExamSession {
23
24     public WrittenSession(Course course, LocalDate date) {
25         super(course, date);
26     }
27
28     @Override
29     public String getType() {
30         return "written";
31     }
32 }
33
34 class OralSession extends ExamSession {
35
36     public OralSession(Course course, LocalDate date) {
37         super(course, date);
38     }
39
40     @Override
41     public String getType() {
42         return "oral";
43     }
44 }
  
```

- ExamSession è il tipo astratto comune usato dal client.
- WrittenSession/OralSession cambiano il dettaglio concreto ma rispettano lo stesso contratto.
- Il client lavora sul tipo astratto, non sulla sottoclasse concreta.

Factory Method in Java: Creator e ConcreteCreator

```

1 abstract class ExamSessionCreator {
2
3     protected abstract ExamSession createSession(Course course, LocalDate date);
4
5     // logica comune che usa la sessione creata
6     public ExamSession schedule(Course course, LocalDate date) {
7
8         // Chiama pollinifon: implementata nel concrete creator
9         ExamSession session = createSession(course, date);
10
11         // eventuale logica comune:
12         // ad esempio: log, notifica...
13         return session;
14     }
15 }
16
17 class WrittenSessionCreator extends ExamSessionCreator {
18
19     @Override
20     protected ExamSession createSession(Course course, LocalDate date) {
21
22         return new WrittenSession(course, date);
23     }
24 }
25
26 class OralSessionCreator extends ExamSessionCreator {
27
28     @Override
29     protected ExamSession createSession(Course course, LocalDate date) {
30
31         return new OralSession(course, date);
32     }
33 }
  
```

- ExamSessionCreator definisce il flusso comune e delega il punto variabile a createSession().
- I concrete creator (WrittenSessionCreator, OralSessionCreator) decidono quale prodotto concreto istanziare.
- La creazione resta estendibile: nuovo tipo di appello = nuovo creator, senza modificare il client.

Uso del Factory method

```

1 class ExamManagementService {
2
3     private final ExamSessionCreator creator;
4
5     ExamManagementService(ExamSessionCreator creator) {
6         this.creator = creator;
7     }
8
9     public ExamSession createExamSession(Course course, LocalDate date) {
10
11         // il servizio non conosce
12         // WrittenSession / OralSession
13         return creator.schedule(course, date);
14     }
15 }
16
17 // configurazione dell'applicazione
18 ExamSessionCreator creator = new WrittenSessionCreator();
19 ExamManagementService service = new ExamManagementService(creator);
20 // in futuro:
21 // creator = new OralSessionCreator();
  
```

- Per cambiare il tipo di sessione basta fornire un ExamSessionCreator diverso.
- Nessuna modifica al codice di ExamManagementService.

Secondo esempio Factory Method: nuova tipologia di appello

```

1 class MixedSession extends ExamSession {
2
3     public MixedSession(Course c, LocalDate d) {
4         super(c, d);
5     }
6
7     @Override
8     public String getType() {
9         return "mixed";
10    }
11 }
12
13 class MixedSessionCreator extends ExamSessionCreator {
14
15     @Override
16     protected ExamSession createSession(Course c, LocalDate d) {
17         return new MixedSession(c, d);
18     }
19 }
20
21 ExamSessionCreator creator = new MixedSessionCreator();
22 ExamManagementService service =
23     new ExamManagementService(creator);
24 ExamSession s = service.createExamSession(course, date);
  
```

- Estensione concreta senza toccare ExamManagementService.
- La variabilità resta concentrata nel Creator.

Terzo esempio Factory Method: parser documenti

```

1 abstract class Documento {
2     abstract String tipo();
3 }
4
5 class DocumentoCsv extends Documento {
6     DocumentoCsv(String raw) { /* parse CSV */ }
7     String tipo() { return "csv"; }
8 }
9
10 class DocumentoJson extends Documento {
11     DocumentoJson(String raw) { /* parse JSON */ }
12     String tipo() { return "json"; }
13 }

```

```

1 abstract class DocumentParser {
2     protected abstract Documento createDocumento(String raw);
3 }
4
5 public Documento parse(String raw) {
6     Documento d = createDocumento(raw);
7     // validazioni e normalizzazione comuni
8     return d;
9 }
10
11 class CsvParser extends DocumentParser {
12     protected Documento createDocumento(String raw) {
13         return new DocumentoCsv(raw);
14     }
15 }
16
17 class JsonParser extends DocumentParser {
18     protected Documento createDocumento(String raw) {
19         return new DocumentoJson(raw);
20     }
21 }
22
23 DocumentParser parser = new JsonParser();
24 Documento doc = parser.parse(input);
25 String tipo = doc.tipo();

```

- Prodotto: Documento; concreti: DocumentoCsv/DocumentoJson.
- Flusso comune in parse(), varia solo la creazione del prodotto concreto.
- Nuovo formato (XmlParser) senza toccare il client.

Factory Method: vantaggi principali (collegamento a SOLID):

- **DIP (Dependency Inversion):**
 - il client lavora con ExamSession e ExamSessionCreator, non con WrittenSession/OralSession;
 - la scelta della classe concreta è confinata nel Factory Method.
- **OCF (Open/Closed):**
 - per aggiungere un nuovo tipo di sessione si estende con nuove sottoclassi/creator;
 - il codice che usa il prodotto non va modificato.
- **SRP (Single Responsibility):**
 - la creazione non è sparsa nel codice: è centralizzata nel Creator;
 - il servizio si concentra sulla logica applicativa, non sui dettagli costruttivi.
- **Coerenza del processo di creazione:**
 - eventuali passi comuni (validazioni, log, notifiche) stanno in un punto solo.

Factory Method: errori tipici (rischi su SOLID):

- **Creator che diventa un "God object":**
 - se nel Creator finiscono troppe responsabilità oltre alla creazione, si rischia di violare SRP.
- **Il client continua a conoscere le classi concrete:**
 - se compaiono comunque new WrittenSession(...) in giro, si perde il beneficio su DIP.
- **Esplosione di sottoclassi senza bisogno reale:**
 - troppi Creator "micro" possono aumentare complessità senza vero guadagno;
 - meglio introdurlo quando la creazione varia davvero.

Strategy vs Factory Method:

- **Strategy:**
 - incapsula un comportamento variabile;
 - il contesto delega l'esecuzione di un'operazione a una Strategy;
 - esempio UniManager: criterio di ordinamento degli appelli.
- **Factory Method:**
 - incapsula la creazione di oggetti (prodotti);
 - il client delega la scelta della classe concreta al Factory Method;
 - esempio UniManager: creazione di diverse tipologie di appello (ExamSession).
- **In comune:**

- favoriscono OCP: estendere (nuove strategie / nuovi prodotti) senza modificare i client;
- favoriscono DIP: dipendenza da astrazioni (SortStrategy, ExamSession, Creator);
- aiutano SRP: separano responsabilità (comportamenti / creazione) dal resto della logica.

Parte 12 - Design Patterns Essenziali: Adapter e Observer

Abbiamo visto:

- Strategy: rendere intercambiabili i comportamenti;
- Factory Method: centralizzare e strutturare la creazione degli oggetti.

Questi pattern agivano soprattutto:

- “dentro” al sistema;
- su logiche di dominio e creazione.

Ora affrontiamo due problemi diversi ma altrettanto comuni:

- come parlare con il mondo esterno senza contaminare il dominio interno (Adapter);
- come reagire a eventi interni senza creare dipendenze rigide tra classi (Observer).

Obiettivo: riconoscere quando questi pattern emergono “da soli” mentre il sistema cresce e si collega ad altri componenti.

Adapter: Il nostro sistema è costruito attorno a interfacce interne pulite. Le sorgenti esterne (CSV, JSON, API) espongono interfacce diverse. Effetti tipici:

- formati incompatibili;
- strutture dati grezze;
- gestione I/O o eccezioni non allineata al nostro dominio.

Obiettivo: integrare queste sorgenti senza “sporcare” il codice del dominio.

☰ Esempi dal caso UniManager:

Libreria LegacyCsvLibrary:

- restituisce solo List;
- formati di data non uniformi;
- API con metodi statici e dettagli I/O esposti.

Libreria o servizio JSON:

- restituisce strutture grezze (Map);
- formati temporali diversi (ISO, timestamp, timezone);
- gestione di errori di rete e codici HTTP.

Queste differenze non devono “trapelare” dentro il dominio.

Adapter: Definiamo una interfaccia Target interna (es. ExamSessionImporter). Per ogni sorgente (CSV, JSON, API,...) realizziamo un Adapter dedicato. Ogni Adapter:

- implementa l'interfaccia Target;
- incapsula la libreria esterna ('Adaptee');
- converte i dati grezzi in oggetti di dominio.

Il dominio usa sempre il Target e resta indipendente dalla tecnologia esterna.

UniManager: CSV e JSON come sorgenti esterne: UniManager vuole ottenere sempre una List. Le fonti possono essere diverse:

- file CSV letti da LegacyCsvLibrary;
- file JSON o API con LegacyJsonLibrary.

Problema: senza Adapter, il servizio dovrebbe conoscere formati, parsing, eccezioni, convenzioni di entrambe. Soluzione: due Adapter:

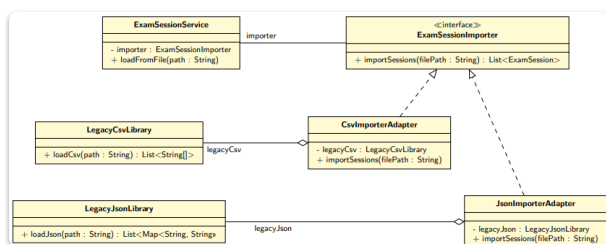
- CsvImporterAdapter;
- JsonImporterAdapter;

che implementano la stessa interfaccia ExamSessionImporter.

In UniManager il dominio vede sempre e solo:

- un'interfaccia (ExamSessionImporter),
- un tipo di ritorno (List).

Integrazioni diverse (CSV, JSON, API) sono intercambiabili. Possiamo aggiungere nuovi formati senza toccare il dominio. Tutta la logica di mapping e parsing resta confinata negli Adapter.



```
1 interface ExamSessionImporter {
2     List<ExamSession> importSessions(String filePath);
3 }
4 // Libreria esterna (CSV)
5 class LegacyCsvLibrary {
6     public List<String> loadCsv(String path) {
7         return new ArrayList<>(); // righe grezze
8     }
9 }
10
11 class CsvImporterAdapter implements ExamSessionImporter {
12     private final LegacyCsvLibrary legacyLib;
13     public CsvImporterAdapter(LegacyCsvLibrary legacyLib) {
14         this.legacyLib = legacyLib;
15     }
16     @Override
17     public List<ExamSession> importSessions(String filePath) {
18         List<String> rows = legacyLib.loadCsv(filePath);
19         List<ExamSession> sessions = new ArrayList<>();
20         for (String row : rows) {
21             Course c = findCourseByCode(row[0]);
22             LocalDateTime d = LocalDateTime.parse(row[1]);
23             String type = row[2];
24             sessions.add(createExamSession(c, d, type));
25         }
26         return sessions;
27     }
28 }
```

- Il servizio client dipende da ExamSessionImporter, non dalla libreria CSV concreta.
- CsvImporterAdapter traduce il formato grezzo (String[]) in oggetti di dominio (ExamSession).
- Se cambia la sorgente dati, si sostituisce l'Adapter senza modificare il codice applicativo.

<pre> 1 // Libreria esterna (200) 2 class LegacyJsonLibrary { 3 public List<Map<String,String>> loadFrom(String path) { 4 return new ArrayList<>(); // dati grezzi 5 } 6 } </pre>	<pre> 1 class JsonImporterAdapter implements ExamSessionImporter { 2 private final LegacyJsonLibrary legacyLib; 3 public JsonImporterAdapter(LegacyJsonLibrary legacyLib) { 4 this.legacyLib = legacyLib; 5 } 6 7 @Override 8 public List<ExamSession> importSessions(String filePath) { 9 List<Map<String,String>> items = legacyLib.loadFrom(filePath); 10 List<ExamSession> sessions = new ArrayList<>(); 11 12 for (Map<String,String> m : items) { 13 Course c = findCourseById(m.get("courseCode")); 14 LocalDateTime d = LocalDateTime.parse(m.get("date")); 15 String type = m.get("type"); 16 sessions.add(createExamSession(c, d, type)); 17 } 18 return sessions; 19 } 20 } </pre>
---	--

- L'Adapter JSON applica la stessa interfaccia del caso CSV: cambia la sorgente, non il contratto.
- La mappatura da Map<String,String> a ExamSession resta confinata nel layer di integrazione.
- Il dominio continua a ricevere oggetti coerenti, indipendenti dal formato esterno.

<pre> 1 // Servizio che dipende solo dall'interfaccia 2 class ExamSessionService { 3 private final ExamSessionImporter importer; 4 ExamSessionService(ExamSessionImporter importer) { 5 this.importer = importer; 6 } 7 8 public void importFromFile(String path) { 9 List<ExamSession> imported = importer.importSessions(path); 10 // salvataggio nel repository, controlli, ecc. 11 } 12 } </pre>	Configurazione CSV <pre> 1 LegacyJsonLibrary lib = new LegacyJsonLibrary(); 2 ExamSessionImporter importer = new CsvImporterAdapter(lib); 3 ExamSessionService service = new ExamSessionService(importer); 4 service.importFromFile("appelli.csv"); </pre> Configurazione JSON <pre> 1 LegacyJsonLibrary lib = new LegacyJsonLibrary(); 2 ExamSessionImporter importer = new JsonImporterAdapter(lib); 3 ExamSessionService service = new ExamSessionService(importer); 4 service.importFromFile("appelli.json"); </pre>
--	--

- ExamSessionService non sa se i dati provengono da CSV o JSON.
- Basta cambiare l'Adapter per cambiare la sorgente, senza toccare il servizio.

<pre> 1 interface PaymentPort { 2 EsitoPagamento pay(Ordine ordine, Carta carta); 3 } 4 5 // SDK esterna 6 class LegacyPaylib { 7 LegacyResult execute(String token, int centes) { 8 return new LegacyResult(); 9 } 10 } 11 12 // Secondo SDK legacy 13 class LegacyWireSdk { 14 LegacyResult send(String iban, int centes) { 15 return new LegacyResult(); 16 } 17 } </pre>	<pre> 1 class LegacyPayAdapter implements PaymentPort { 2 private final LegacyPaylib sdk; 3 LegacyPayAdapter(LegacyPaylib sdk) { this.sdk = sdk; } 4 public EsitoPagamento pay(Ordine o, Carta c) { 5 LegacyResult r = sdk.execute(tokenize(c), o.totaleCentes()); 6 return mapToEsito(r); 7 } 8 } 9 10 class LegacyWireAdapter implements PaymentPort { 11 // stesso idea del precedente adapter, SDK diversa 12 public EsitoPagamento pay(Ordine o, Carta c) { /* ... */ } 13 } </pre> <pre> 1 PaymentPort pay = new LegacyPayAdapter(new LegacyPaylib()); 2 EsitoPagamento esito = pay.pay(ordine, carta); 3 PaymentPort wire = new LegacyWireAdapter(new LegacyWireSdk()); 4 EsitoPagamento esito2 = wire.pay(ordine, carta); </pre>
--	--

- Il checkout dipende da PaymentPort, non dalla SDK esterna.
- Mapping e conversioni restano confinati nell'Adapter.

<pre> 1 interface IdentityProvider { 2 ProfiloUtente autentica(String username, String password); 3 } 4 5 // API esterna 6 class LegacyIdClient { 7 UserProfile login(String user, String pass) { 8 return new UserProfile(); 9 } 10 } </pre>	<pre> 1 class LegacyIdAdapter implements IdentityProvider { 2 private final LegacyIdClient api; 3 LegacyIdAdapter(LegacyIdClient api) { this.api = api; } 4 5 public ProfiloUtente autentica(String u, String p) { 6 UserProfile payload = api.login(u, p); 7 return toProfile(payload); 8 } 9 } </pre> <pre> 1 IdentityProvider idp = new LegacyIdAdapter(new LegacyIdClient()); 2 ProfiloUtente profilo = idp.autentica(user, pass); </pre>
---	---

- Il dominio usa IdentityProvider con modello interno stabile.
- Cambio provider SSO: si sostituisce l'Adapter, non il servizio di login.

Vantaggi:

- **DIP (Dependency Inversion):**
 - ExamSessionService dipende dall'interfaccia ExamSessionImporter;
 - le librerie esterne restano "dietro" gli Adapter.
- **OCP (Open/Closed):**
 - per supportare un nuovo formato (XML, API diversa,...) basta aggiungere un nuovo Adapter;
 - il servizio che importa non cambia.
- **SRP (Single Responsibility):**
 - parsing, conversione e mapping sono confinati nell'Adapter;
 - il dominio e i servizi non si riempiono di dettagli I/O e formati.
- **Testabilità e manutenzione:**
 - Adapter testabile in isolamento;
 - il servizio può usare importer fittizi nei test.

Errori tipici:

- Adapter che fa troppo: se oltre a convertire dati inizia a contenere logica di dominio, si viola SRP.
- Service che dipende comunque dalla libreria esterna: se il dominio usa tipi della libreria (es. String[] o Map ovunque), si indebolisce DIP.
- If/else sui formati dentro il servizio: scegliere CSV/JSON con switch nel servizio porta a modifiche frequenti: peggiora OCP.

Observer: Con Adapter abbiamo gestito l'integrazione con l'esterno. Ora il problema è interno: come reagire a un evento? Il contesto tipico è:

- una classe genera un evento (es. nuovo appello);
- più componenti vogliono essere informati.

Soluzione ingenua: la classe chiama direttamente tutti gli altri (sendEmail(), updateUi(), writeLog(),...). Conseguenze:

- la classe finisce per conoscere troppi dettagli su altri componenti;
- il codice diventa più difficile da mantenere e capire;
- aggiungere un nuovo comportamento richiede modifiche al codice esistente.

Il pattern Observer separa:

- il Subject, che produce gli eventi;
- gli Observer, che vogliono essere notificati.

Meccanismo:

- gli Observer si registrano presso il Subject;
- quando qualcosa cambia, il Subject chiama un unico metodo (notifyObservers()) che notifica tutti gli iscritti;
- il Subject non conosce i dettagli di cosa faranno gli Observer.

Effetto:

- possiamo aggiungere/rimuovere osservatori senza modificare il Subject;
- il sistema diventa più estendibile e testabile.

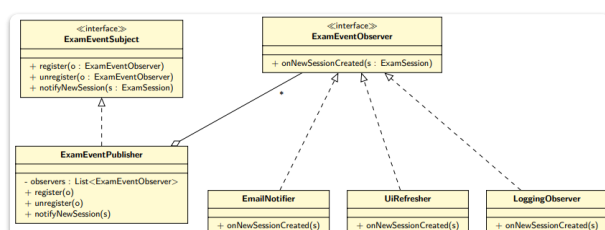
UniManager: notificare la creazione di un appello

In UniManager, la creazione di un nuovo ExamSession deve attivare più reazioni:

- aggiornare la vista,
- inviare notifiche agli studenti,
- scrivere un log.

Il servizio che crea l'appello non deve conoscere questi dettagli. Con Observer:

- ExamEventPublisher funge da Subject;
- EmailNotifier, UiRefresher, LoggingObserver sono Observer che reagiscono indipendentemente.



```

1 interface ExamEventObserver {
2     void subSessionCreated(ExamSession session);
3 }

1 interface ExamEventSubject {
2     void register(ExamEventObserver observer);
3     void unregister(ExamEventObserver observer);
4     void notifyNewSession(ExamSession session);
5 }

1 class ExamEventPublisher implements ExamEventSubject {
2     private final List<ExamEventObserver> observers = new ArrayList<>();
3
4     @Override
5     public void register(ExamEventObserver observer) {
6         observers.add(observer);
7     }
8
9     @Override
10    public void unregister(ExamEventObserver observer) {
11        observers.remove(observer);
12    }
13
14    @Override
15    public void notifyNewSession(ExamSession session) {
16        for (ExamEventObserver o : observers) {
17            o.subSessionCreated(session);
18        }
19    }
20 }

```

- ExamEventPublisher gestisce solo il ciclo di notifica (registro observer + dispatch evento).
- Il Subject dipende dall'interfaccia ExamEventObserver: nuove reazioni si aggiungono senza modificare il publisher.

```

1 class EmailNotifier implements ExamEventObserver {
2
3     @Override
4     public void subSessionCreated(ExamSession session) {
5         // invia email agli studenti
6         // iscritti al corso dell'appello
7         System.out.println("Invia email per " + session.getCourse().
8             getName());
9     }
10 }

1 class LoggingObserver implements ExamEventObserver {
2
3     @Override
4     public void subSessionCreated(ExamSession session) {
5         // scrive su un log applicativo
6         System.out.println("LOG: creato appello " + session.getName());
7     }
8 }

```

- Ogni observer implementa una reazione specifica allo stesso evento, senza accoppiamento diretto tra observer.
- Per aggiungere una nuova reazione basta introdurre una nuova classe observer e registrarla nel publisher.

```

1 class ExamManagementService {
2
3     private final ExamEventSubject eventPublisher;
4
5     ExamManagementService(ExamEventSubject eventPublisher) {
6         this.eventPublisher = eventPublisher;
7     }
8
9     public ExamSession createExamSession(Course course, LocalDate date) {
10
11         ExamSession session = new WrittenSession(course, date);
12         session = repository.save(session);
13
14         // La logica di dominio (creare l'appello) e' qui.
15         // La reazione all'evento e' delegata al publisher.
16         eventPublisher.notifyNewSession(session);
17
18         return session;
19     }
20 }
21
22 // Configurazione
23 ExamEventPublisher publisher = new ExamEventPublisher();
24 publisher.register(new EmailNotifier());
25 publisher.register(new LoggingObserver());
26
27 ExamManagementService service = new ExamManagementService(publisher);

```

- Per aggiungere un nuovo comportamento alla creazione di un appello:
 - basta creare un nuovo ExamEventObserver e registrarlo;
 - nessuna modifica al codice di ExamManagementService.

Vantaggi:

- **DIP (Dependency Inversion):**
 - il Subject dipende dall'interfaccia ExamEventObserver, non dagli observer concreti;
 - le reazioni (email, log, UI, . . .) restano dettagli sostituibili.
- **OCP (Open/Closed):**
 - nuove reazioni all'evento si aggiungono registrando un nuovo Observer;
 - il codice che genera l'evento non va modificato.
- **SRP (Single Responsibility):**
 - il servizio si concentra sulla logica principale (creare l'appello);
 - le reazioni all'evento stanno in classi separate, ognuna con un compito specifico.
- **Testabilità:**
 - nei test si possono usare observer fake e verificare che la notifica avvenga correttamente.

Errori tipici:

- Subject che conosce gli observer concreti: se istanzia direttamente new EmailNotifier(), perde DIP.
- Observer con responsabilità miste: se un singolo observer gestisce email + log + UI, si perde SRP.
- Notifiche “a cascata” difficili da controllare:
 - se gli observer generano altri eventi senza regole chiare, il sistema diventa fragile;
 - spesso si finisce a “patchare” codice esistente, andando contro OCP.

Adapter vs Observer:

- **Adapter:**
 - risolve problemi di incompatibilità di interfacce;
 - tipico per integrare librerie, servizi, formati dati esterni;
 - es. UniManager: import di appelli da CSV/JSON con un'interfaccia interna pulita.
- **Observer:**
 - risolve problemi di notifica di eventi;
 - tipico per disaccoppiare chi produce un evento da chi reagisce;
 - es. UniManager: notifiche agli studenti e aggiornamento viste quando viene creato un nuovo appello.
- **In comune:**
 - proteggono il codice da dipendenze rigide;
 - spostano la complessità nei confini del sistema (integrazione, eventi);
 - rendono il design più modulare e evolvibile.

Design Pattern essenziali:

Pattern	Problema che risolve	Esempi in UniManager
Strategy	Comportamenti che possono variare nel tempo; evita lunghi blocchi if/else.	Ordinamento degli appelli secondo criteri diversi (data, corso, iscritti).
Factory Method	Creazione di oggetti che può richiedere logiche diverse; centralizza la costruzione.	Creazione di diverse tipologie di ExamSession (Written, Oral).
Adapter	Integrazione di librerie/formati esterni incompatibili con il dominio interno.	Import di appelli da CSV/JSON tramite ExamSessionImporter.
Observer	Notificare cambiamenti senza accoppiamento rigido tra componenti.	Avvisare studenti, viste e log quando viene creato un nuovo appello.

I pattern affrontano quattro problemi distinti: comportamenti, creazione, integrazione ed eventi. Insieme costituiscono la base per un design evolvibile.

Sintesi:

- Adapter e Observer affrontano due problemi ricorrenti diversi: integrazione con esterni ed eventi interni tra componenti.
- Adapter protegge il dominio da formati e librerie incompatibili, spostando la complessità ai confini del sistema.
- Observer consente di reagire a cambiamenti senza legare direttamente chi produce l'evento a tutti i componenti interessati.
- Entrambi i pattern favoriscono estendibilità e riduzione dell'accoppiamento, in linea con OCP e DIP.
- Il beneficio dei pattern dipende da come vengono applicati: migliorano il design se rispondono a una variabilità reale.

- Gli esempi su UniManager mostrano come questi pattern preparino il sistema a integrazioni e notifiche future.

Parte 13 - Qualità del Design: Code Smells e Refactoring

Perché parlare di qualità interna? Un software può funzionare ed essere comunque difficile da mantenere e da far crescere nel tempo. La qualità interna determina in modo diretto:

- quanto è semplice correggere bug senza introdurre effetti collaterali;
- quanto è facile aggiungere nuove funzionalità senza "toccare tutto";
- quanto si capisce il codice dopo settimane/mesi (anche da parte di chi non lo ha scritto).

Nella pratica, la qualità interna è ciò che rende un progetto:

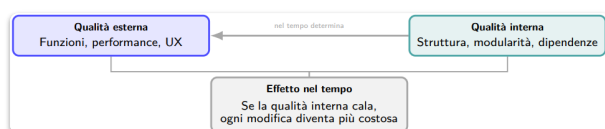
- evolvibile invece che fragile,
- leggibile invece che opaco,
- modulare invece che intrecciato,
- più testabile e quindi più sicuro da modificare.

Qualità esterna vs qualità interna:

- **Qualità esterna:** ciò che percepisce l'utente finale (funzionalità offerte, prestazioni, affidabilità, esperienza d'uso).
- **Qualità interna:** proprietà strutturali del codice (chiarezza, organizzazione, modularità, dipendenze sane).

Effetto tipico nei progetti reali: oggi funziona → domani costa troppo cambiarlo o estenderlo. In questa lezione ci concentriamo in particolare su:

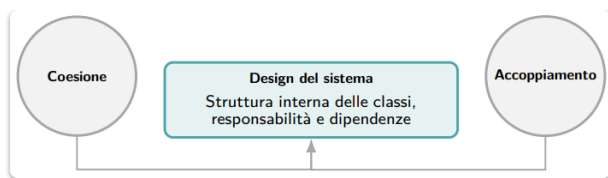
- segnali che indicano un design che sta lentamente peggiorando;
- strumenti concettuali per riconoscerli e intervenire in anticipo.



Come leggere il design:

- **Coesione:** quanto gli elementi di uno stesso modulo/classe stanno bene insieme e lavorano per un unico scopo. Se c'è alta coesione allora avrò una classe focalizzata, più facile da capire e testare.
- **Accoppiamento:** quanto un modulo/classe dipende dagli altri. Se ho basso accoppiamento allora ci saranno meno dipendenze rigide, cambiamenti più locali.

Regola pratica: alta coesione + basso accoppiamento.



Coesione: misura la “compattezza” delle responsabilità assegnate a un modulo o a una classe. Un modulo realmente coeso:

- ha un obiettivo chiaro e facilmente esplicitabile;
- contiene responsabilità dello stesso tipo e tra loro correlate;
- si può descrivere in modo naturale con una singola frase.

Segnali tipici di bassa coesione:

- una classe che offre molte funzionalità non correlate tra loro;
- metodi che lavorano su concetti diversi e che non condividono un filo logico comune;
- nomi generici (Manager, Utils, Helper) che cercano di “coprire tutto” senza un ruolo preciso.

Accoppiamento: misura quanto un modulo dipende da altri moduli e dalle loro decisioni interne. Un accoppiamento elevato significa:

- conoscere e sfruttare dettagli interni di altri componenti;
- vincolarsi a implementazioni concrete invece che a contratti astratti;
- amplificare l'effetto di una modifica anche lontano dal punto di intervento.

Segnali tipici di alto accoppiamento:

- accesso diretto a campi interni e strutture dati di altre classi;
- catene di chiamate lunghe (train wreck);
- dipendenze diffuse “ovunque” verso componenti instabili o tecnologici.

Nei progetti reali emergono spesso due problemi ricorrenti, che tendono a rafforzarsi a vicenda se non vengono controllati:

1. classi che crescono troppo nel tempo, accumulando responsabilità diverse → bassa coesione;
2. componenti sempre più intrecciati tra loro, con dipendenze diffuse → alto accoppiamento.

Una regola pratica utile per diagnosticare la situazione è la seguente:

- se per cambiare una parte A è necessario capire anche B, C, D, allora l'accoppiamento è probabilmente eccessivo;
- se una classe ha “troppe ragioni per cambiare” in contesti diversi, allora la sua coesione è probabilmente insufficiente.

L'obiettivo del buon design è mantenere questi due aspetti in equilibrio, così che ogni modulo possa evolvere in modo il più possibile indipendente.



Spesso il problema non è che il codice sia "sbagliato" o non funzionante, ma che risulti scomodo da leggere, capire e modificare. Situazioni tipiche in cui emerge questa sensazione:

- metodi troppo lunghi, con molti passaggi e livelli di dettaglio mescolati;
- classi che crescono a ogni nuova feature senza una struttura chiara;
- logiche di dominio mescolate con dettagli tecnici o di presentazione.

Questa sensazione di disagio è in realtà preziosa, non è una prova matematica di errore, ma è un segnale da prendere sul serio e verificare usando criteri progettuali e strumenti concettuali.

Code smells: sono indicatori di possibili problemi nella struttura e nell'organizzazione del design. Non sono bug né errori di funzionamento:

- il programma continua a funzionare correttamente;
- ma il design mostra segnali di tensione e perdita di chiarezza.

Sono particolarmente utili perché:

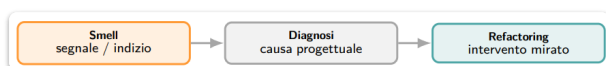
- aiutano a individuare il degrado del design prima che diventi evidente o ingestibile;
- suggeriscono punti del codice in cui il refactoring può portare il maggior beneficio.

Smells e refactoring:

- Smell → ipotesi di problema nel design: qualcosa suggerisce che la struttura non sia ottimale.
- Refactoring → intervento disciplinato per migliorare la struttura senza cambiare il comportamento osservabile.

L'idea chiave è che gli smells non dicono "cambia qui" in modo automatico, ma dicono "qui vale la pena guardare meglio e fare una verifica". In pratica, il flusso di lavoro tipico è:

- si osserva il codice e si riconoscono possibili segnali;
- si formula una diagnosi (quale problema progettuale c'è sotto?);
- si applicano refactoring mirati, piccoli e controllati.



Gli smell che vedremo sono:

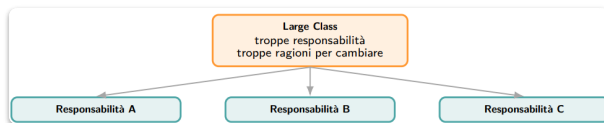
- **Large Class:** troppe responsabilità nella stessa classe.
- **Long Method:** metodo troppo lungo e difficile da mantenere.
- **Feature Envy:** metodo che usa più dati di un'altra classe che della propria.
- **Data Clumps:** gruppi di parametri che ricorrono sempre insieme.
- **God Object:** classe centrale che coordina, decide e conosce troppo.
- **Duplicated Code:** stessa logica copiata in più punti.
- **Primitive Obsession:** uso eccessivo di tipi primitivi al posto di concetti di dominio.
- **Divergent Change:** una classe cambia per motivi eterogenei.
- **Shotgun Surgery:** una modifica richiede piccoli cambi sparsi in molti file.

Large Class: Una Large Class è una classe che accumula troppe responsabilità e, di conseguenza, troppe ragioni diverse per cambiare nel tempo. Effetti tipici che emergono in questi casi:

- bassa coesione, perché le responsabilità non sono omogenee;
- maggiore difficoltà di test, poiché è complesso isolare i comportamenti;
- modifiche rischiose, perché molte funzionalità “passano tutte di lì”.

Questo smell si accompagna spesso a:

- nomi generici (`*Manager` , `*Service`) che non chiariscono il ruolo reale;
- una crescita “per accumulo” di metodi, senza una struttura progettuale chiara.



Example

```
1 class ExamManagementService {
2   // ERRORE: responsabilita' studenti nella stessa classe
3   void registerStudent(...) { ... }
4
5   // ERRORE: responsabilita' appelli nella stessa classe
6   void createSession(...) { ... }
7   void enrollStudent(...) { ... }
8   void publishResults(...) { ... }
9
10  // ERRORE: persistenza mescolata con dominio
11  void saveOnFile(...) { ... }
12  void loadFromFile(...) { ... }
13
14  // ERRORE: presentazione dentro il service
15  String formatCliMessage(...) { ... }
16 }
```

- Una sola classe copre dominio, persistenza e presentazione.
- Ogni nuova richiesta tende a finire nello stesso punto.

```
1 // FIX: solo regole di business sugli appelli
2 class ExamDomainService {
3   void createSession(...) { ... }
4   void enrollStudent(...) { ... }
5   void publishResults(...) { ... }
6 }
7
8 // FIX: persistenza isolata in un componente dedicato
9 class ExamRepository {
10  void save(ExamSession s) { ... }
11  ExamSession findById(String id) { ... }
12 }
13
14 // FIX: presentazione separata ai margini (CLI/UI)
15 class ExamPresenter {
16  String formatCliMessage(...) { ... }
17 }
```

Large class (refactoring)

- Obiettivo: separare per responsabilità e per motivo di cambiamento.
- Decomposizione esplicita: dominio, persistenza e presentazione.
- Effetto: classi più coese e modifiche più locali.

Long Method: Un Long Method è un metodo che cresce oltre misura e tende a concentrare troppa logica in un unico punto del codice. Tipicamente mescola tra

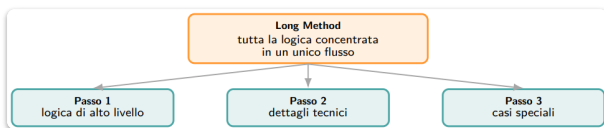
loro:

- livelli di astrazione diversi (alto livello e dettagli operativi);
- logica di business e dettagli tecnici o infrastrutturali;
- casi normali ed eccezioni gestiti nello stesso flusso lineare.

Effetti tipici di questo smell:

- è difficile capire “che cosa fa” davvero il metodo nel suo insieme;
- è difficile riusare o testare singole parti della logica;
- aumenta la probabilità di introdurre bug quando si modifica anche solo un passaggio.

Refactoring tipico e consigliato: **Extract Method**.



Example

```
1 void processEnrollment(String studentId, String examId) {
2     // ERRORE: metodo lungo con troppi dettagli interni
3     Student s = studentRepo.find(studentId);
4     ExamSession e = examRepo.find(examId);
5     if (s == null || e == null) throw new IllegalArgumentException();
6
7     // ERRORE: regole business mescolate nel metodo orchestratore
8     if (!e.isOpen()) throw new IllegalStateException();
9     if (!s.hasPrerequisitesFor(e)) throw new IllegalStateException();
10    if (s.hasScheduleConflict(e)) throw new IllegalStateException();
11
12    // ERRORE: persistenza nello stesso metodo
13    enrollmentRepo.save(new Enrollment(s, e));
14    // ERRORE: audit e notifica nello stesso flusso
15    audit.log("ENROLL", studentId, examId, e.getDate().toString());
16    notifier.notifyStudent(studentId, "Iscrizione completata");
17    // ERRORE: output UI/CLI dentro logica applicativa
18    System.out.println("OK " + studentId + " -> " + e.getCourse().getCode());
19 }
```

- Validazione, regole, persistenza, audit e output nello stesso metodo.

```
1 void processEnrollment(String studentId, String examId) {
2     // FIX: orchestrazione ad alto livello, leggibile
3     EnrollmentContext ctx = loadContext(studentId, examId);
4     // FIX: regole business isolate
5     validateEnrollment(ctx);
6     // FIX: persistenza isolata
7     persistEnrollment(ctx);
8     // FIX: side-effect isolati
9     logEnrollment(ctx);
10    notifyEnrollment(ctx);
11 }
```

Long method (refactoring)

- Refactoring applicato: **Extract method**
- Ogni step ha nome esplicito e responsabilità unica
- Il metodo principale torna leggibile a livello di processo

Feature Envy: Un Feature Envy si verifica quando un metodo utilizza più dati e comportamenti di un'altra classe che non quelli della classe in cui è definito. Indicazione progettuale tipica:

- la responsabilità è probabilmente posizionata nel posto sbagliato;
- il modello concettuale risulta sbilanciato o mal distribuito.

Sintomi comuni di questo smell:

- catene di accessi del tipo `a.getB().getC().getX()`, che rivelano una conoscenza eccessiva della struttura interna altrui;
- un metodo che “conosce troppo” lo stato e i dettagli di un altro oggetto.

Refactoring tipici: **Move Method** oppure **Extract Class**.



Example

```
1 class ExamSessionPrinter {
2     String formatForStudent(ExamSession s) {
3         // ERRORE: il metodo naviga troppo dentro altri oggetti
4         return s.getCourse().getCode() + " - "
5             + s.getCourse().getName() + " - "
6             + s.getDate().toString() + " - "
7             + s.getRoom().getName() + " - "
8             + s.getTeacher().getSurname();
9     }
10 }
```

- Il metodo usa quasi solo dati interni di ExamSession, Course, Room, Teacher.
- La responsabilità di formattazione è nel posto sbagliato.

```
1 class ExamSession {
2     // FIX: il comportamento vive vicino ai dati che usa
3     String summaryForStudent() {
4         return course.getCode() + " - "
5             + course.getName() + " - "
6             + date.toString() + " - "
7             + room.getName() + " - "
8             + teacher.getSurname();
9     }
10 }
11
12 class ExamSessionPrinter {
13     // FIX: il printer delega, non conosce dettagli interni
14     String formatForStudent(ExamSession s) {
15         return s.summaryForStudent();
16     }
17 }
```

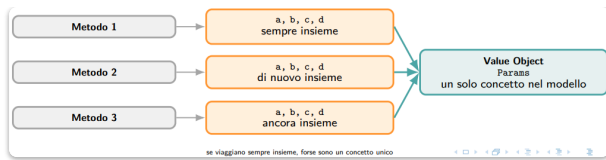
Long method (refactoring)

- Refactoring applicato: **Move method**
- Il comportamento torna vicino ai dati che usa davvero

Data Clumps: I Data Clumps sono gruppi di variabili che compaiono sempre insieme e vengono passati in giro come un unico “pacchetto”, anche se nel codice restano separati. Situazioni tipiche:

- gli stessi parametri ripetuti in molti metodi e costruttori;
- le stesse variabili che “viaggiano insieme” tra classi e servizi.

Questo smell spesso indica che nel modello manca un concetto: un'entità o, più spesso, un Value Object che rappresenti quel gruppo in modo esplicito. Il refactoring tipico è quello di introdurre un **Value Object** (es. StudentId, ExamInfo, ExamDescriptor,...).



Example

```

1 // ERRORE: stesso gruppo di parametri ripetuto in piu' metodi
2 void enroll(
3     String studentId,
4     String studentName,
5     String courseCode,
6     String courseName,
7     LocalDate examDate,
8     String roomName
9 ) { ... }
10
11 void publishResult(
12     String studentId,
13     String studentName,
14     String courseCode,
15     String courseName,
16     LocalDate examDate,
17     String roomName,
18     int grade
19 ) { ... }

```

- Lo stesso blocco di parametri si ripete in più metodi.
- Manca un concetto esplicito nel modello.

```

1 // FIX: i campi che viaggiano insieme diventano un Value Object
2 final class StudentDescriptor {
3     private final StudentId studentId;
4     private final StudentName studentName;
5     // costruttore + getter
6 }
7
8 final class ExamDescriptor {
9     private final CourseCode courseCode;
10    private final CourseName courseName;
11    private final LocalDate examDate;
12    private final String roomName;
13    // costruttore + getter
14 }
15
16 // FIX: firme piu' corte e semanticamente piu' chiare
17 void enroll(StudentDescriptor student, ExamDescriptor exam) { ... }
18 void publishResult(StudentDescriptor student, ExamDescriptor exam, int grade) { ... }

```

Data

clumps (refactoring)

- Refactoring applicato: **Introduce value object**
- Firma più corta, semantica più chiara, validazioni centralizzabili
- In questo esempio introduciamo anche StudentDescriptor per accorpare i dati studente che viaggiano insieme

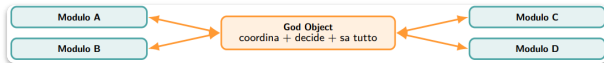
God Object: Un God Object è una classe “onnipotente” che finisce per diventare il centro operativo del sistema e concentra decisioni che dovrebbero essere distribuite. In pratica è una classe che:

- coordina tutto (orchestrazione di molte parti),
- decide tutto (regole e policy sparse),
- sa tutto (conoscenza di dettagli interni di molti moduli).

Tipicamente produce effetti molto negativi:

- accoppia tra loro parti del sistema che dovrebbero restare separate;
- diventa un collo di bottiglia cognitivo: per capire il sistema bisogna passare da lì;
- impedisce modularità e testabilità, perché è difficile isolarla e simulare dipendenze.

È spesso la forma estrema di Large Class + responsabilità non distribuite (tutto viene “assorbito” in un punto).



Example

```

1 class UniManagerService {
2     // ERRORE: dominio concentrato qui
3     void createSession(...) { ... }
4     void enrollStudent(...) { ... }
5     void publishGrades(...) { ... }
6     // ERRORE: integrazione/export nello stesso punto
7     void exportCsv(...) { ... }
8     void importCsv(...) { ... }
9     // ERRORE: persistenza nello stesso punto
10    void saveOnFile(...) { ... }
11    void loadFromFile(...) { ... }
12    // ERRORE: notifiche + presentazione nello stesso punto
13    void sendNotifications(...) { ... }
14    void buildDashboard(...) { ... }
15 }
  
```

- Una sola classe concentra dominio, I/O, reportistica e presentazione.
- Ogni modulo finisce per dipendere da questo punto centrale.

```

1 // FIX: separazione esplicita delle responsabilità
2 class ExamDomainService {
3     private final ExamRepository repo;
4
5     ExamDomainService(ExamRepository repo) { this.repo = repo; }
6     void closeSession(String examId) { ... }
7 }
8
9 class ExamRepository {
10     ExamSession findById(String examId) { /* query */ return null; }
11     void save(ExamSession session) { /* persistenza */ }
12 }
13
14 class NotificationService {
15     void notifyClosure(String examId) {
16         // invio email / evento verso UI
17     }
18 }
19
20 class ExamClosureUseCase {
21     private final ExamDomainService domain;
22     private final NotificationService notifications;
23
24     ExamClosureUseCase(ExamDomainService domain, NotificationService notifications) { ... }
25     void execute(String examId) { ... }
26 }
  
```

God object (refactoring)

- I ruoli sono separati in servizi distinti
- Il caso d'uso non passa più da una classe onnicomprensiva

Duplicated Code: Il Duplicated Code compare quando la stessa logica è copiata in più punti. Effetti tipici:

- ogni correzione va ripetuta in più file/metodi;
- alto rischio di inconsistenza tra copie quasi uguali.

Indizio operativo: “copia e incolla” usato per introdurre varianti minime.

Refactoring tipici: **Extract Method, Extract Class, Template Method**.

Example

```
1 class ExamService {
2     boolean canEnroll(Student s, ExamSession e) {
3         // ERRORE: questa sequenza sara' copiata anche altrove
4         if (!e.isOpen()) return false;
5         if (!s.hasPrerequisitesFor(e)) return false;
6         if (s.hasScheduleConflict(e)) return false;
7         return true;
8     }
9 }
10
11 class RetakeService {
12     boolean canEnrollRetake(Student s, ExamSession e) {
13         // ERRORE: duplicazione delle stesse tre regole
14         if (!e.isOpen()) return false;
15         if (!s.hasPrerequisitesFor(e)) return false;
16         if (s.hasScheduleConflict(e)) return false;
17         return s.hasPreviousFail(e.getCourse());
18     }
19 }
```

- Le prime tre regole sono duplicate.
- La variante reale è solo l'ultima condizione.

```
1 // FIX: regole comuni estratte in un metodo unico
2 class EnrollmentPolicy {
3     boolean baseChecks(Student s, ExamSession e) {
4         return e.isOpen()
5             && s.hasPrerequisitesFor(e)
6             && !s.hasScheduleConflict(e);
7     }
8 }
9
10 class RetakePolicy extends EnrollmentPolicy {
11     // FIX: qui resta solo la variante specifica
12     boolean canEnrollRetake(Student s, ExamSession e) {
13         return baseChecks(s, e) && s.hasPreviousFail(e.getCourse());
14     }
15 }
```

Duplicated code

(refactoring)

- Regole comuni in un punto solo
- Le variabili restano piccole e controllabili

Primitive Obsession: Riguarda l'uso eccessivo di tipi primitivi (String, int, double) al posto di concetti di dominio. Effetti tipici:

- validazioni sparse e duplicate;
- parametri poco autoesplicativi;
- errori di scambio tra campi semanticamente diversi.

Refactoring tipici: **Replace Data Value with Object, Introduce Value Object**.

Example

```

1 void registerGrade(
2     String studentId,
3     String courseCode,
4     int grade,
5     String examDate
6 ) {
7     // ERRORE: validazioni sparse su primitivi
8     if (studentId == null || studentId.isBlank()) throw new IllegalArgumentException();
9     if (courseCode == null || courseCode.isBlank()) throw new IllegalArgumentException();
10    if (grade < 18 || grade > 30) throw new IllegalArgumentException();
11    // ...
12 }

```

- I parametri sono primitivi, ma rappresentano concetti di dominio.
- Le regole di validazione non sono incapsulate.

```

1 // FIX: i vincoli entrano in tipi di dominio dedicati
2 final class StudentId { /* validazione nel costruttore */ }
3 final class CourseCode { /* validazione nel costruttore */ }
4 final class GradeValue { /* range 18..30 */ }
5 final class ExamDate { /* formato/data valida */ }
6
7 void registerGrade(
8     StudentId studentId,
9     CourseCode courseCode,
10    GradeValue grade,
11    ExamDate examDate
12 ) { /* ... */ }

```

Primitive obsession

(refactoring)

- I vincoli diventano parte del modello
- La firma del metodo è più espressiva e robusta

Divergent Change: Divergent Change si ha quando una singola classe viene modificata per motivi diversi e non correlati. Il segnale tipico è che ogni nuova richiesta (report, validazione, persistenza, UI) tocca sempre la stessa classe. L'effetto è quello di forte instabilità locale e regressioni frequenti. Il refactoring tipico: separare responsabilità per asse di cambiamento.

≡ Example

```

1 class ExamManagementService {
2     // ERRORE: cambia se cambiano regole business
3     void createSession(...) { ... }
4
5     // ERRORE: cambia se cambia formato export
6     String exportCsv(...) { ... }
7
8     // ERRORE: cambia se cambia storage
9     void saveOnFile(...) { ... }
10
11    // ERRORE: cambia se cambia interfaccia utente
12    String formatUiMessage(...) { ... }
13 }

```

- Quattro motivi di cambiamento nella stessa classe

```

1 // FIX: una classe per ciascun asse di cambiamento
2 class ExamDomainService { void createSession(...) { ... } }
3 class ExamExporter { String exportCsv(...) { ... } }
4 class ExamRepository { void saveOnFile(...) { ... } }
5 class ExamPresenter { String formatUiMessage(...) { ... } }

```

Divergent change

(refactoring)

- Ogni classe di cambiamento ha una classe dedicata
- Le modifiche eterogenee non ricadono più nello stesso file

Shotgun Surgery: Shotgun Surgery è il caso opposto del Divergent Change: una modifica concettualmente unica richiede tanti cambi minori in classi diverse. Segnali tipici:

- pull request con molti file toccati per una feature piccola;
- rischio elevato di dimenticare un punto.

Refactoring tipico: accorpare comportamento correlato e chiarire i confini.

Example

- Nuova regola: “uno studente non può iscriversi a due appelli nello stesso slot orario”.
- Implementazione dispersa:

```
1 // ERRORE: una sola regola richiede cambi sparsi
2 StudentService.java
3 ExamSessionService.java
4 EnrollmentFacade.java
5 CalendarService.java
6 CliController.java
7 CsvExporter.java
```

- Una sola regola di dominio impatta troppi punti diversi.

```
1 // FIX: centralizzare la regola in una policy unica
2 class SchedulingPolicy {
3     boolean canEnroll(Student s, ExamSession e) {
4         return e.isOpen() && !s.hasScheduleConflict(e);
5     }
6 }
7
8 class EnrollmentService {
9     private final SchedulingPolicy policy;
10    // FIX: il service delega la regola e resta stabile
11    boolean enroll(Student s, ExamSession e) {
12        if (!policy.canEnroll(s, e)) return false;
13        // save + notify
14        return true;
15    }
16 }
```

Shotgun surgery (refactoring)

- La regola è centralizzata in una policy unica
- I moduli delegano: una modifica tocca pochi punti

Riepilogo:

- La qualità interna del design incide direttamente sulla capacità di mantenere, testare ed evolvere il software nel tempo.
- Coesione e accoppiamento sono criteri pratici per leggere la “salute” del codice oltre il solo comportamento corretto.
- I code smells non sono prove definitive di errore, ma segnali utili per individuare zone a rischio del design.

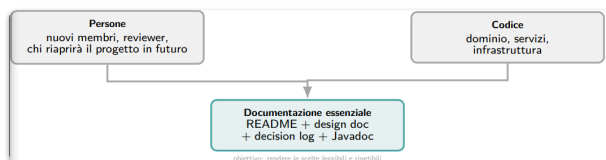
- Riconoscere presto gli smells aiuta a intervenire con refactoring prima che il degrado diventi architetturale.
- I miglioramenti di design spesso non cambiano il comportamento esterno, ma riducono fragilità e costo delle modifiche future.
- Gli esempi su UniManager mostrano come collegare osservazione degli smells e refactoring concreto.

Parte 14 - Documentazione Essenziale

La documentazione viene talvolta trattata come attività accessoria. In realtà è uno strumento di ingegneria:

- riduce incomprensioni;
- accelera onboarding e passaggi di consegne;
- rende il progetto manutenibile e accogliente.

Obiettivo pratico: meno attrito tra persone e codice $\Rightarrow$ più qualità e meno errori.



A chi serve davvero la documentazione?

1. A chi ha già lavorato sul progetto, quando:
 - lo riapre dopo settimane o mesi;
 - deve intervenire rapidamente su un bug o una modifica senza ricordare tutti i vincoli progettuali.
2. A chi entra successivamente nel progetto:
 - un onboarding più rapido riduce domande ricorrenti, incomprensioni e interventi correttivi.
3. A chi valuta, estende o integra il progetto:
 - consente di comprendere scelte architetturali e trade-off senza dover analizzare l'intero codice sorgente.

Documentazione essenziale: Non è necessario "documentare tutto" né descrivere ogni dettaglio del sistema. È invece fondamentale mantenere un insieme ridotto ma significativo di documenti sempre coerenti con il codice:

- **README:** punto di ingresso al progetto, guida all'onboarding e all'avvio rapido per chi lo utilizza o lo esplora per la prima volta.
- **Design document:** descrizione delle principali scelte architetturali, dei vincoli e delle motivazioni che le hanno guidate.
- **Decision log:** tracciamento delle decisioni tecniche rilevanti, utile per comprendere perché è stata scelta una soluzione rispetto ad altre.
- **Documentazione API mirata (Javadoc, docstring, godoc,...):** descrive i contratti delle API pubbliche e i comportamenti rilevanti per l'uso corretto del codice.

Regola pratica di riferimento:

- poco, chiaro e aggiornato → più utile di una documentazione ampia ma obsoleta.
- **Terminologia: ADR** (Architecture Decision Record) indica una nota breve che registra una singola decisione architeturale.

Documentazione come codice (docs-as-code): La documentazione essenziale non vive “fuori” dal progetto, ma sta nel repository, versionata insieme al codice. Una scelta pratica ricade su:

- Markdown (.md) per documenti operativi brevi (README, ADR, note tecniche);
- LaTeX per documenti più strutturati e versionati (relazioni, report tecnici, slide, documenti di progetto).

Quando il codice cambia, la documentazione “si vede” che va aggiornata (stesso commit, stessa responsabilità). Un anti-pattern ricorrente è: documento Word / Drive separato ⇒ divergenza rapida ⇒ perdita di fiducia. L'effetto pratico è che la documentazione si aggiorna insieme al codice e resta allineata nel tempo.

README, il primo documento che si legge: Rappresenta la porta di ingresso al repository ed è spesso il primo punto di contatto con il progetto. Si tratta di un documento operativo orientato all'uso (uno strumento di onboarding tecnico). Consente di ottenere rapidamente un primo risultato funzionante e verificabile. Un README ben scritto contribuisce a ridurre:

- tempo perso nella fase di setup e configurazione;
- errori di esecuzione dovuti a istruzioni incomplete;
- domande ripetitive e interventi di supporto non necessari.

Le 4 domande a cui deve rispondere:

1. *Che cos'è il progetto?*

- fornisce il contesto generale e l'ambito applicativo;
- permette di capire subito se il progetto è rilevante per chi lo sta esplorando.

2. *Cosa fa il progetto?*

- describe le funzionalità principali in modo sintetico;
- evita descrizioni prolisse: poche righe sono più efficaci di una spiegazione lunga e dispersiva.

3. *Come si esegue?*

- spiega come ottenere un'installazione funzionante;
- include un quickstart riproducibile, verificabile passo dopo passo.

4. *Come si contribuisce?*

- chiarisce regole minime di collaborazione;
- riduce errori dovuti a stili incoerenti o test mancanti.



Struttura consigliata (template mentale):

- **Overview:**
 - introduce il progetto in un paragrafo chiaro;
 - evidenzia obiettivi e funzionalità principali.
- **Quickstart:**
 - fornisce comandi reali e immediatamente eseguibili;

- porta rapidamente a un primo risultato funzionante.
- **Usage:**
 - mostra 1-2 esempi rappresentativi di utilizzo;
 - chiarisce il comportamento senza aprire il codice.
- **Configurazione:**
 - documenta variabili, file e dipendenze rilevanti;
 - chiarisce cosa è personalizzabile e come.
- **Test:**
 - spiega come eseguire la suite di test;
 - rende verificabile il corretto funzionamento.
- **Contributing:**
 - definisce regole minime per contribuire in modo coerente;
 - riduce attriti su branch, review e stile del codice.

README anti-pattern (cose che lo rovinano):

- **Wall of text:** assenza di sezioni chiare e di un quickstart immediato, che rende difficile individuare le informazioni essenziali.
- **Istruzioni vaghe:** indicazioni generiche come “installare le dipendenze” senza specificare strumenti, versioni o comandi concreti.
- **Comandi non verificati o incompleti:** esempi che non funzionano o che producono errori non documentati.
- **Duplicazioni e incoerenze:** stesse informazioni ripetute in più punti del README con contenuti non allineati tra loro.
- **README usato come contenitore indistinto:** accumulo di informazioni eterogenee che lo rende lungo, confuso e poco leggibile.

README leggibile, Markdown essenziale: Il README è quasi sempre scritto in Markdown (non serve conoscerlo tutto, basta un sottoinsieme essenziale). Elementi fondamentali per la leggibilità sono:

- **# Titolo principale:** identifica chiaramente il progetto.
- **## Sezioni:** struttura logica (Quickstart, Usage, Contributing).
- **### Sottosezioni:** dettagli senza appesantire.

Enfasi testuale:

- ****grassetto**** per concetti chiave e parole guida;
- ***corsivo*** per chiarimenti o termini secondari;
- **`codice`** per comandi, file, classi, opzioni CLI.

Regola pratica: se una sezione è lunga più di una schermata, probabilmente va spezzata con un titolo **##**.

<pre> 1 # UniManager 2 3 **UniManager** is a small teaching project used to discuss 4 software design, architecture boundaries, and documentation practices. 5 6 ## Overview 7 - Manage courses, exam sessions, enrollments, and grades. 8 - Keep 'domain' independent from CLI and persistence. 9 - Provide a simple baseline for discussing architecture in class. 10 11 ## Main goals 12 1. clear architecture boundaries 13 2. testability-first 14 3. extensibility over time 15 16 ## Quickstart 17 Requirements: JDK 17+, Maven 3.8+ 18 19 \$ git clone https://github.com/corp/unimanager.git 20 \$ cd unimanager 21 \$ mvn test 22 \$ mvn exec:java -Dexec.mainClass="unimanager.cli.Main" 23 24 ## Project structure 25 - 'src/main/java/unimanager/domain' 26 - 'src/main/java/unimanager/application' 27 - 'src/main/java/unimanager/interfaces' </pre>	<pre> 1 2 ## Usage (CLI) 3 ## Record an exam grade 4 \$ unimanager exam record \ 5 --student #123 \ 6 --course INF101 \ 7 --date 2020-10-10 \ 8 --room A1 \ 9 --grade 20 10 11 ## Help 12 \$ unimanager --help 13 14 ## Docs 15 - [Design doc](docs/design/unimanager-architecture.md) 16 - [API index](docs/decisions/README.md) 17 - [API notes](docs/api/README.md) 18 19 ## Contributing 20 - Run 'mvn test' before pushing. 21 - Keep 'domain' independent from CLI and persistence. 22 - Add/update API for non-trivial decisions. 23 - Update README if setup or commands change. </pre>
---	---

Regola pratica per il README: Se una persona che entra nel progetto non riesce a:

- clonare il repository;
- eseguire il progetto senza ambiguità;
- ottenere un output minimo e verificabile;

allora il README non sta svolgendo correttamente la funzione di onboarding. L'obiettivo concreto del README è di ridurre al minimo il time-to-first-success, ovvero il tempo necessario per arrivare a un primo risultato funzionante.

Design document: È un documento breve (indicativamente 1-3 pagine) che descrive:

- le principali scelte architetturali del sistema;
- le alternative progettuali considerate durante la progettazione;
- i vincoli e i trade-off che hanno guidato le decisioni.

Non è una descrizione esaustiva di ogni classe, metodo o dettaglio implementativo. Il valore principale del design document è di preservare il ragionamento progettuale alla base delle scelte, rendendolo comprensibile anche a chi arriverà sul progetto in un secondo momento.

Quando serve davvero un design doc? Un design document è particolarmente utile quando una scelta progettuale è:

- difficile o costosa da modificare in seguito;
- rilevante per più moduli o parti del sistema;
- soggetta a discussioni ricorrenti nel tempo.

Esempi tipici di decisioni da documentare:

- organizzazione dei layer e definizione dei boundary;
- strategie di persistenza (in-memory vs database);
- formati di input e output e loro stabilità nel tempo;
- policy di gestione degli errori ed eccezioni.

Struttura di un design doc leggero:

- **Contesto e obiettivi:** descrivono il problema da risolvere e i risultati che il progetto intende ottenere.
- **Non-obiettivi:** chiariscono esplicitamente cosa il design document non intende affrontare o risolvere.
- **Proposta:** presenta la soluzione architetturale adottata, idealmente supportata da una singola figura riassuntiva.
- **Alternative considerate:** elencano le opzioni valutate e i motivi per cui non sono state scelte.
- **Conseguenze:** analizzano vantaggi, svantaggi e rischi introdotti dalla soluzione proposta.
- **Piano di adozione:** descrive come e quando la soluzione viene introdotta, se è prevista una migrazione o un'evoluzione graduale.

Esempio design doc:

Estratto sintetico di un design document applicato a UniManager.

- **Contesto:** progetto didattico utilizzato per discutere aspetti di design, organizzazione dei servizi applicativi e definizione dei boundary

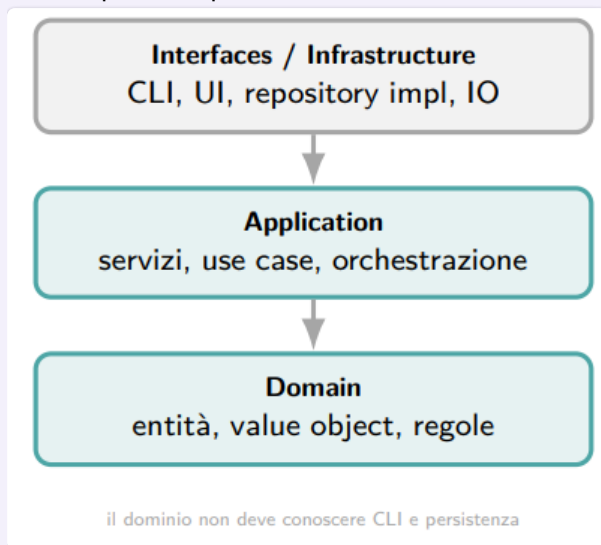
architetturali.

- **Obiettivo principale:** mantenere il dominio applicativo indipendente sia dall'interfaccia utente (CLI) sia dai meccanismi di persistenza.
- **Vincoli progettuali:**
 - **semplicità:** struttura leggibile e comprensibile dagli studenti;
 - **testabilità:** utilizzo di repository in-memory per facilitare i test automatici;
 - **estendibilità:** supporto a interfacce diverse nel tempo (CLI iniziale, GUI o API in una fase successiva).

Proposta architetturale: separazione a livello di responsabilità:

- **Domain:** entità e regole del dominio.
- **Application:** servizi (use case) e orchestrazione.
- **Infrastructure/Interfaces:** repository concreti, CLI/UI.

Principio: dipendenze verso l'interno (dominio stabile).



UML minimale: Un diagramma UML inserito in un design document deve essere:

- minimale, con un numero limitato di elementi (circa 6-10);
- leggibile a colpo d'occhio, senza sovraccaricare la vista;
- focalizzato su relazioni, responsabilità e dipendenze principali.

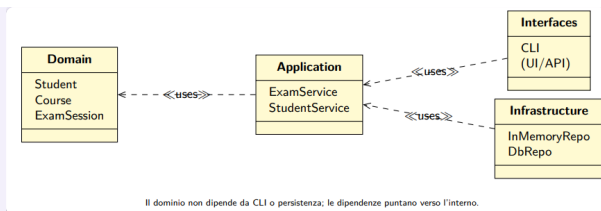
Non è necessario rappresentare:

- ogni campo, getter o dettaglio implementativo;
- aspetti che non contribuiscono a chiarire la struttura complessiva.

≡ **Continuo dell'esempio precedente con UML minimale:**

Rappresentazione essenziale per chiarire:

- boundary tra Domain, Application e Interfaces/Infrastructure;
- dipendenze consentite (verso l'interno).



Decision log: Il codice descrive che cosa è stato realizzato e come il sistema funziona oggi. Spesso però non esplicita perché sono state prese certe decisioni:

- quali alternative erano disponibili al momento della scelta?
- quali vincoli (tecnici o organizzativi) hanno guidato la decisione?
- quali conseguenze e compromessi vengono accettati nel tempo?

Il decision log affronta il problema con note brevi, tracciabili e cronologiche, utili per ricostruire il contesto anche a distanza di mesi.

Formato semplice (stile ADR): Ogni decisione viene registrata in un file autonomo, che include:

- **ID e data** (per riferimenti e ordine cronologico);
- **decisione** presa, formulata in modo chiaro;
- **contesto** che ha portato alla scelta;
- **alternative considerate** e scartate;
- **conseguenze** accettate (pro e contro).

Un buon ADR dovrebbe essere:

- **compatto** (mezza pagina o una pagina al massimo);
- facilmente **linkabile** da task e revisioni tecniche;
- semplice da **aggiornare** se la decisione evolve.

≡ Esempio ADR:

```

1  ADR-001 (2025-10-01)
2  Title: Repository interfaces + InMemory implementation
3
4  Decision
5  Define StudentRepository and ExamRepository as interfaces.
6  Provide an InMemoryRepository implementation for the course.
7
8  Context
9  The system requires testability and a clear separation between
10 domain/application logic and infrastructure concerns.
11
12 Alternatives
13 A) Use a database immediately (e.g., JDBC / ORM).
14 B) Use file-based persistence (CSV / JSON).
15 C) Keep everything in memory.
16
17 Consequences
18 Fast tests, simple setup, clean architectural boundaries.
19 Not persistent across runs.
20 Future work: introduce a database adapter.
```

Regola pratica: mettere l'informazione dove serve a chi la userà. Per questo è bene usare una gerarchia (dal più "vicino al codice" al più "panoramico"):

- **Javadoc** accanto a classi e metodi: descrive contratti e comportamento osservabile delle API.

- **README** alla radice del repository: è il punto di ingresso (quickstart, usage, contributing).
- **Design doc** in /docs/design/: chiarisce architettura, vincoli e trade-off.
- **Decision log (ADR)** in /docs/decisions/ADR-001.md, ADR-002.md,...: conserva la memoria storica delle decisioni non banali.

Documentazione del codice, Javadoc mirata: Documentare il codice non significa commentare ogni riga né spiegare ciò che è già evidente dalla lettura. Una Javadoc realmente utile si concentra su:

- **uso:** come e quando chiamare l'API correttamente;
- **contratto:** che cosa il metodo o la classe garantisce;
- **effetti:** side effects rilevanti e interazioni con la persistenza;
- **casi di errore:** condizioni anomale ed eccezioni significative.

L'obiettivo pratico è quello di comprendere il comportamento dell'API senza dover aprire o analizzare l'implementazione.

Priorità nella documentazione:

1. **API pubbliche e stabili:** punti di accesso pensati per essere usati da altri moduli o da codice esterno.
2. **Classi con responsabilità non banale:** componenti che incarnano regole di dominio, coordinano più oggetti o svolgono ruoli centrali nel sistema.
3. **Metodi che presentano almeno una delle seguenti caratteristiche:**
 - effetti collaterali importanti (persistenza, I/O, modifiche di stato);
 - invarianti o precondizioni che devono essere rispettate;
 - eccezioni significative che il chiamante deve gestire;
 - regole di business non ovvie o non immediatamente deducibili dal nome.

Cosa evitare:

- Commenti che ripetono il codice senza aggiungere informazione, ad esempio spiegazioni ovvie come "incrementa i" sopra i++.
- Commenti di tipo procedurale, che descrivono ogni singolo passo quindi che diventano rapidamente falsi o fuorvianti non appena l'implementazione cambia.
- La "bugia della documentazione", un commento non aggiornato è un bug informativo e per questo induce chi legge a fare scelte sbagliate o a perdere tempo. La Javadoc deve quindi privilegiare i contratti (più stabili), non i dettagli interni (più volatili).
- Javadoc eccessivamente lunga o verbosa. Se entra nei dettagli interni del codice, smette di descrivere un contratto.

 **Javadoc su ExamService (mirata all'uso)**

```

1 package unimanager.service;
2
3 import unimanager.domain.ExamDescriptor;
4
5 /**
6  * Application service for exam registration and grade recording.
7  * <p>
8  * This service implements a single use case: ensure a student exists,
9  * register them to an exam session, and optionally record a passing grade.
10  * </p>
11  */
12 public class ExamService {
13
14     // repositories injected (interfaces)
15
16     /**
17      * Registers a student to the exam described by {exam} and records
18      * the grade if it is passing.
19      *
20      * @param studentId unique student identifier (non-null, non-empty)
21      * @param studentName student name (required if the student does not exist yet)
22      * @param exam exam descriptor (non-null)
23      * @param grade grade to record; passing threshold is 18
24      * @return a human-readable outcome message for the caller (CLI/UI)
25      * @throws IllegalArgumentException if input parameters are invalid
26      */
27     public String registerAndRecord(String studentId, String studentName, ExamDescriptor exam, int grade) {
28
29         // implementation
30         return "OK";
31     }
32 }

```

La documentazione dell'API deve rendere esplicito che il servizio:

- ritorna un esito (valore o risultato strutturato),
- non si occupa della presentazione dell'output.

La Javadoc dovrebbe chiarire:

- che cosa viene restituito e con quale significato;
- perché la formattazione e la stampa sono responsabilità della CLI/UI.

Benefici progettuali:

- separazione chiara tra logica applicativa e interfaccia;
- maggiore testabilità (nessun output nel mezzo del caso d'uso);
- riuso della stessa logica in contesti diversi: CLI oggi, GUI o API domani.

Come si integra nel ciclo di vita del progetto:

- **README:** viene aggiornato quando cambiano il setup, i comandi di esecuzione o le modalità di utilizzo.
- **Design document:** viene aggiornato quando cambiano i boundary, l'architettura complessiva o i vincoli principali.
- **Decision log:** viene aggiornato ogni volta che viene presa una decisione non banale e difficilmente reversibile.
- **Javadoc:** viene aggiornata quando cambia il contratto delle API pubbliche o il loro comportamento osservabile.

Tutti questi elementi non sono documentazione "aggiuntiva" o burocratica, ma fanno parte integrante del processo di sviluppo del software.

Definition of Done della documentazione: Una feature non è davvero completata se il codice cambia ma la documentazione resta indietro. Regola operativa per ogni modifica rilevante:

- se cambia setup/uso → aggiornare **README**;
- se cambiano confini o scelte architettureali → aggiornare **design doc**;
- se c'è una decisione non banale → aggiungere/aggiornare **ADR**;
- se cambia il contratto API → aggiornare **Javadoc**.

DoD minima: codice, test e documentazione coerenti nello stesso merge.

Ownership, chi mantiene cosa:

- **README:** ownership del team, revisione obbligatoria quando cambiano uso/setup.
- **Design doc:** ownership dei responsabili architetturali del modulo.
- **ADR:** ownership di chi propone la decisione, con review tecnica.
- **Javadoc:** ownership di chi modifica API pubbliche e servizi applicativi.

Principio: ogni documento deve avere un proprietario esplicito, non "di nessuno".

Errori frequenti in fase di consegna:

- README con quickstart non riproducibile (comandi mancanti o versioni assenti).
- Design doc descrittivo ma senza alternative e trade-off.
- ADR che registra solo la decisione finale senza contesto.
- Javadoc che copia l'implementazione invece di esplicitare il contratto.
- Documenti aggiornati in momenti diversi: codice e testo divergono.

Riepilogo:

- La documentazione essenziale riduce attrito operativo: aiuta onboarding, manutenzione e passaggi di consegne.
- README, design document, decision log e Javadoc rispondono a bisogni diversi e complementari di comunicazione tecnica.
- Il valore della documentazione dipende dalla sua utilità pratica e dall'allineamento con il codice, non dalla quantità di pagine.
- README e quickstart supportano l'uso del progetto; design document e decision log spiegano perché il sistema è fatto così.
- La Javadoc è più utile quando chiarisce contratto e comportamento osservabile delle API, non i dettagli interni.
- Tenere la documentazione aggiornata è parte del lavoro di sviluppo e prepara meglio testing, CI e manutenzione.

Parte 15 - Testing: Principi e Piramide del Testing

Una convinzione molto diffusa nello sviluppo software è: "Prima scrivo il codice e alla fine faccio un test per controllare se funziona". In un progetto reale questo approccio tende a produrre effetti sistematici:

- il feedback arriva tardi, quando il codice è già cambiato più volte e il contesto iniziale si è perso;
- gli errori diventano più difficili da isolare, perché entrano in gioco più componenti e più ipotesi non esplicitate;
- la correzione richiede più tempo, perché si deve ricostruire a posteriori cosa ci si aspettava davvero.

Il risultato più dannoso spesso non è il bug in sé, ma la conseguenza organizzativa:

- ogni modifica viene percepita come rischiosa,
- il refactoring viene rimandato,

- si sviluppa una reale paura di cambiare il codice.

Nel contesto dello sviluppo moderno, il testing non può più essere considerato un semplice “controllo qualità finale” eseguito a valle dell'implementazione. Il testing diventa invece una pratica continua, integrata nel processo di sviluppo, che accompagna le scelte progettuali e contribuisce a orientarle nel tempo. Un insieme ben costruito di test automatici svolge il ruolo di rete di sicurezza:

- riduce il rischio associato alle modifiche e agli interventi correttivi;
- rende praticabili refactoring frequenti e una progressiva pulizia del design;
- facilita il lavoro collaborativo, perché rende esplicito il comportamento atteso del sistema.

In assenza di test automatici, il software tende progressivamente a irrigidirsi: ogni cambiamento diventa una scommessa e l'evoluzione del sistema rallenta.

Due obiettivi del testing: I test possono essere letti attraverso due obiettivi principali, entrambi necessari:

- *Dimostrare che il sistema soddisfa i requisiti (validazione):* verificare che i comportamenti richiesti siano presenti e coerenti con quanto promesso.
- *Scoprire difetti e comportamenti indesiderati (defect testing):* far emergere errori, incoerenze e casi non gestiti prima che arrivino in produzione.

Non sono due mondi separati: molto spesso un test nato per validare un requisito finisce per rivelare un difetto nascosto, oppure un caso ambiguo nella specifica. Cambia però lo “spirito” con cui si scelgono i casi di test:

- test con input realistici e attesi, che rappresentano l'uso normale del sistema;
- test con input anomali o ai limiti, che esplorano condizioni rare, errori e vincoli.

“Pasted image 20260430095903.png” could not be found.

Perché oggi le regressioni sono inevitabili? Un sistema software reale non è statico: cambia continuamente nel tempo, per motivi diversi:

- nuove funzionalità e miglioramenti richiesti dagli utenti,
- correzioni di bug e interventi di manutenzione,
- refactoring per mantenere il design leggibile e sostenibile,
- aggiornamenti di librerie, framework e dipendenze esterne.

Ogni cambiamento, anche piccolo, può rompere un comportamento che prima era corretto: questa situazione prende il nome di **regressione**. Il punto centrale non è chiedersi “se succederà”, ma accettare che “prima o poi succederà”, perché la complessità cresce e le interazioni tra componenti non sono sempre prevedibili. La difesa più efficace è una suite di test automatica:

- ripetibile,
- eseguita spesso,
- capace di segnalare rapidamente quando un comportamento atteso non è più garantito.

Manuale vs automatico: Il testing manuale rimane necessario in alcune situazioni:

- valutazione dell'esperienza d'uso e dell'interfaccia grafica,
- flussi complessi che coinvolgono più passaggi e decisioni,
- verifiche che richiedono giudizio umano o interpretazione contestuale.

Tuttavia, il testing manuale non scala bene: richiede tempo, è costoso da ripetere e tende a essere applicato solo a una parte dei casi possibili.

Il testing automatico costituisce la base della verifica delle regressioni:

- è ripetibile in modo affidabile,
- è veloce da eseguire,
- può essere lanciato ad ogni modifica del codice senza costi aggiuntivi significativi.

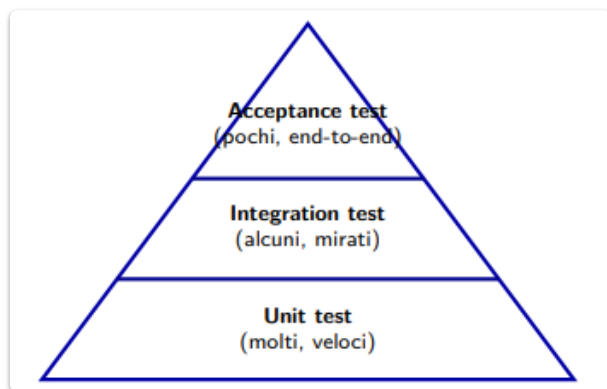
L'obiettivo pratico non è scegliere uno solo dei due approcci, ma automatizzare tutto ciò che è stabile e verificabile, lasciando al testing manuale gli aspetti che richiedono valutazioni qualitative.

La piramide del testing: La piramide del testing è un modello semplice per distribuire bene i test:

- molti test piccoli e veloci alla base,
- alcuni test di collaborazione a metà,
- pochi test end-to-end in cima.

L'idea non è “fare tanti test a caso”, ma ottenere:

- feedback veloce,
- diagnosi semplice,
- fiducia sul comportamento complessivo.



Unit test: Gli unit test verificano una singola unità di comportamento del sistema, considerata in modo isolato rispetto al resto dell'applicazione. Tipicamente si concentrano su:

- un metodo specifico,
- una classe con una responsabilità ben definita,
- una singola regola di business o di validazione.

Per essere davvero efficaci, gli unit test condividono alcune caratteristiche fondamentali:

- sono **rapidi**, perché si eseguono in pochi millisecondi e possono essere lanciati molto spesso;

- sono **isolati**, cioè non dipendono da database, rete, filesystem o tempo reale;
- sono **deterministici**, nel senso che a parità di input producono sempre lo stesso risultato.

Queste proprietà rendono gli unit test la base della piramide del testing, perché permettono un feedback continuo e affidabile durante lo sviluppo quotidiano.

I candidati migliori per gli unit test sono le regole di business, perché rappresentano il cuore logico del sistema e concentrano molte decisioni importanti. In particolare, vale la pena testare con attenzione:

- validazioni di input e di stato,
- vincoli che devono essere sempre rispettati,
- calcoli e trasformazioni dei dati,
- transizioni di stato tra fasi diverse del ciclo di vita di un oggetto.

Questi test sono in genere facili da scrivere e offrono un grande valore, perché intercettano regressioni "costose", che altrimenti emergerebbero solo molto più avanti. Un buon unit test è anche leggibile:

- descrive chiaramente cosa deve succedere in una certa situazione,
- evita di raccontare come il codice realizza internamente quel comportamento.

Integration test: Gli integration test verificano la collaborazione tra più componenti del sistema, osservando come parti diverse funzionano insieme in uno scenario realistico. Tipicamente riguardano situazioni come:

- servizi applicativi che interagiscono con repository o componenti di persistenza;
- flussi che attraversano più layer dell'architettura (ad esempio dominio, applicazione, infrastruttura);
- integrazioni con risorse reali o semi-reali, come database, file system o servizi esterni.

Rispetto agli unit test, questi test sono più costosi da gestire perché richiedono:

- un setup più articolato dell'ambiente di esecuzione,
- la presenza di dipendenze esterne correttamente configurate,
- dati coerenti e consistenti tra i diversi componenti coinvolti.

Per questo motivo gli integration test non dovrebbero essere numerosi, ma selezionati con attenzione e mantenuti pochi e mirati sui punti di collaborazione più critici.

Gli integration test risultano particolarmente utili in quei punti del sistema in cui il comportamento corretto dipende dall'interazione tra più componenti. In pratica, i casi più critici sono spesso legati a:

- il mapping tra oggetti del dominio e il livello di persistenza;
- la serializzazione e il parsing dei dati, insieme alla gestione dei formati;
- le transazioni e la garanzia di consistenza dei dati tra operazioni successive;
- le configurazioni dell'applicazione e il corretto wiring delle dipendenze.

In questi punti gli unit test non sono sufficienti, perché l'errore non nasce da una singola unità, ma dal modo in cui più parti del sistema collaborano. Un integration

test ben scelto e mirato su uno di questi punti critici vale spesso più di molti test generici che coprono solo casi superficiali.

Acceptance test: Gli acceptance test verificano il comportamento del sistema dal punto di vista di chi lo utilizza, senza entrare nei dettagli interni dell'implementazione. Si basano su descrizioni ad alto livello, come:

- scenari completi di utilizzo,
- user stories rappresentative,
- criteri di accettazione che definiscono quando una funzionalità può dirsi soddisfacente.

Il loro valore principale è la validazione funzionale: permettono di verificare che il sistema, nel suo insieme, faccia davvero ciò che serve rispetto alle esigenze espresse. Allo stesso tempo presentano alcune limitazioni strutturali:

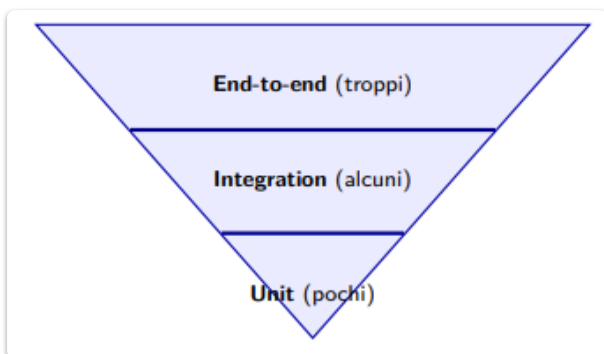
- sono più lenti da eseguire rispetto agli altri tipi di test,
- risultano più fragili, perché sensibili a cambiamenti anche minimi,
- rendono più complessa l'analisi delle cause quando qualcosa va storto.

Per questi motivi gli acceptance test sono collocati in cima alla piramide: sono pochi, selezionati con cura e rappresentano solo i flussi principali e più significativi del sistema.

Un anti-pattern, la piramide rovesciata: Errore comune: basarsi quasi solo su test end-to-end. Effetti tipici:

- suite lenta,
- fallimenti difficili da interpretare,
- test che si rompono per cambiamenti minimi,
- persone che smettono di eseguire i test.

Questa forma viene spesso chiamata "ice-cream cone".



Criteri pratici per una suite sana: Una suite di test "sana" tende a mostrare alcune proprietà ricorrenti:

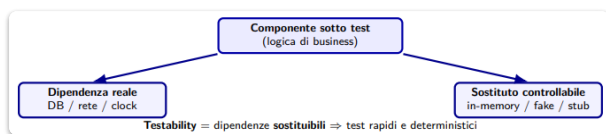
- è abbastanza veloce da poter essere eseguita spesso, idealmente durante lo sviluppo quotidiano;
- quando fallisce, produce errori comprensibili e facili da collegare alla causa;
- fallisce in modo deterministico, cioè non "a caso" e non dipendendo dall'ambiente;
- copre soprattutto le regole di business e i flussi principali, cioè ciò che dà valore al sistema.

Non esiste una percentuale magica o una ricetta universale: l'obiettivo pratico è ottenere feedback continuo e mantenere alta la fiducia mentre il software evolve. Se la suite diventa lenta, la conseguenza è quasi sempre la stessa: viene eseguita sempre meno, fino a essere ignorata nei momenti in cui servirebbe di più.

Testability: Non tutto il codice è ugualmente facile da testare: alcune scelte progettuali rendono la verifica semplice, altre la complicano inutilmente. Dire che un codice è "testabile" significa, in pratica, che è possibile:

- verificare il comportamento di una parte del sistema senza dover costruire o avviare mezzo sistema intorno;
- controllare l'ambiente del test, incluse dipendenze esterne, dati di input e fattori come il tempo;
- ottenere test piccoli, veloci da eseguire e stabili nel tempo.

La testabilità è quindi un indicatore importante della qualità del design: spesso coincide con alta coesione, basso accoppiamento e con una struttura che favorisce l'evoluzione del software.



Segnali di bassa testability: Alcune caratteristiche del codice rendono la scrittura dei test più difficile del necessario e sono spesso segnali di problemi di design. Tra i segnali più comuni rientrano:

- dipendenze nascoste, come la creazione interna di client per database, servizi HTTP o altre risorse esterne;
- side effect diffusi, ad esempio scritture su file o rete mescolate direttamente alla logica di business;
- classi troppo grandi, che accumulano responsabilità diverse e diventano difficili da comprendere e isolare.

Un'altra fonte frequente di problemi è l'uso diretto di elementi globali o non controllabili:

- tempo di sistema,
- generatori di numeri casuali,
- singleton o stati globali condivisi.

Un sintomo pratico molto chiaro è questo: per testare anche una singola regola di business diventa necessario costruire un setup enorme e complesso.

SRP: Il principio di **Single Responsibility** (SRP) afferma che una classe dovrebbe avere una sola responsabilità, cioè un solo motivo valido per cambiare. Quando questo principio è rispettato, anche i test diventano più naturali e diretti:

- ogni test verifica un comportamento preciso e ben delimitato;
- il setup necessario è minimo, perché ci sono poche dipendenze coinvolte;
- in caso di errore, la causa è più facile da individuare e isolare.

Al contrario, classi "tuttofare", che mescolano logica diversa, costringono a scrivere test grandi, complessi e fragili, difficili da capire e costosi da mantenere. Una regola empirica utile per riconoscere il problema è la seguente: se

un singolo test deve controllare troppe cose insieme, molto probabilmente la classe sotto test fa troppe cose.

DIP: Il **Dependency Inversion Principle (DIP)** afferma che il codice dovrebbe dipendere da astrazioni e non da dettagli concreti di implementazione. Questo principio ha un impatto diretto sulla testabilità, perché rende possibile sostituire facilmente le dipendenze esterne durante i test. In pratica, nei test diventa naturale usare alternative controllabili:

- un repository in memoria al posto di un database reale,
- un clock controllabile invece del tempo di sistema,
- un gateway finto o simulato invece di una comunicazione di rete.

Quando le dipendenze sono progettate per essere sostituibili, i test tornano ad essere rapidi, deterministici e focalizzati sul comportamento, senza effetti collaterali indesiderati.

UniManager: due casi ideali da testare:

All'interno di UniManager sono presenti alcune regole di business particolarmente chiare e ben delimitate, che si prestano molto bene alla scrittura di unit test. Due esempi rappresentativi sono:

- la logica di iscrizione a un appello d'esame,
- la validazione di un voto inserito dal docente.

Questi casi sono buoni esempi dal punto di vista didattico perché:

- presentano input e output ben definiti, facilmente verificabili;
- includono diversi casi limite e condizioni di errore interessanti;
- non dovrebbero dipendere direttamente dall'interfaccia utente o dal database per essere validate correttamente.

Iscrizione a un appello: Scenario di riferimento: un utente con ruolo di studente tenta di iscriversi a un appello d'esame disponibile nel sistema. Questo scenario è governato da alcune regole di business tipiche, realistiche e facilmente comprensibili anche a fini didattici:

- lo studente deve risultare regolarmente iscritto al corso associato all'appello;
- non è consentita una doppia iscrizione allo stesso appello;
- l'iscrizione non è possibile oltre una certa data di scadenza;
- l'appello può prevedere un numero massimo di posti disponibili.

In un contesto del genere, gli unit test risultano particolarmente efficaci perché permettono di verificare in modo diretto:

- i casi validi, in cui l'iscrizione deve andare a buon fine;
- i casi invalidi, in cui il sistema deve impedire l'operazione;
- la coerenza dei messaggi di errore o delle eccezioni prodotte.

Validazione del voto: Scenario di riferimento: un utente con ruolo di docente registra il voto di uno studente al termine di un appello d'esame. Anche in

questo caso il comportamento del sistema è regolato da un insieme di regole di business ben definite:

- il voto deve rientrare nell'intervallo previsto dal regolamento (ad esempio 18-30), con l'eventuale gestione della lode;
- il voto può essere registrato solo per studenti effettivamente iscritti all'appello;
- non è consentito registrare o modificare un voto se il verbale dell'appello risulta chiuso;
- il processo può prevedere stati distinti, come "inserito", "pubblicato" e "verbalizzato".

Anche in questo scenario il nucleo logico è facilmente testabile senza coinvolgere infrastrutture esterne: la validazione del voto dovrebbe risiedere in un punto chiaro e ben isolato del dominio, rendendo possibile la verifica tramite unit test semplici e mirati.

Come scegliere casi di test: La scelta dei casi di test non è casuale, ma segue un ragionamento preciso. Un approccio semplice ed efficace consiste nel considerare:

- un caso "tipico", che rappresenta l'uso normale e che deve funzionare senza problemi;
- casi ai limiti, come inizio e fine di un intervallo, scadenze temporali, valori massimi e minimi ammessi;
- casi di errore, legati a input non validi, stati non consentiti o tentativi di operazioni duplicate.

L'obiettivo non è testare ogni possibile combinazione, ma concentrare gli sforzi su ciò che statisticamente tende a rompersi:

- i confini delle regole,
- le condizioni eccezionali,
- le combinazioni meno ovvie ma più rischiose.

Test-first: L'approccio test-first non va inteso come l'adozione rigida di un rituale formale o di una metodologia prescrittiva. In pratica, significa *usare i test come strumento per chiarire il comportamento atteso prima di scrivere l'implementazione*:

- cosa deve succedere in condizioni corrette;
- cosa non deve succedere mai;
- quali situazioni di errore devono essere intercettate e gestite.

L'effetto pratico di questo modo di lavorare è spesso immediato:

- interfacce e API più pulite e coerenti;
- meno ambiguità nelle responsabilità del codice;
- implementazioni più semplici, perché guidate dal comportamento atteso.

Un modo pragmatico e leggero di applicare il test-first può essere visto come un micro-processo iterativo:

1. si sceglie una regola di business piccola e ben delimitata;

2. si scrive un test che ne descrive il comportamento atteso;
3. si esegue il test e si osserva il fallimento, che è il risultato previsto (Red);
4. si implementa il minimo indispensabile per far passare il test (Green);
5. si refactorizza il codice mantenendo i test sempre verdi (Refactor).

Il valore principale non è semplicemente “scrivere il test prima del codice”, ma il fatto che questo approccio costringe a ragionare sull'uso, sul comportamento e sui confini della regola prima di concentrarsi sull'implementazione.

Takeaway, come leggere la piramide: La piramide non è una moda: è un modo pratico per bilanciare costo e feedback. Salendo nella piramide, in generale:

- il **costo** aumenta (setup, dipendenze, manutenzione),
- la **velocità** diminuisce (da millisecondi a secondi/minuti),
- l'**isolamento** diminuisce (da unità isolate a flussi end-to-end).

Obiettivo operativo: tanti unit test per feedback veloce, integration test mirati sui punti critici, pochi acceptance sui flussi principali.



Riepilogo:

- Il testing è parte del processo di progettazione: rende esplicito il comportamento atteso e riduce il rischio delle modifiche.
- La piramide del testing aiuta a bilanciare velocità di feedback, costo e granularità dei controlli automatici.
- Unit, integration e acceptance test rispondono a domande diverse e vanno combinati, non sostituiti l'uno con l'altro.
- La testabilità dipende fortemente dal design: buon isolamento e dipendenze sane rendono i test più semplici e utili.
- La rete di test automatici è ciò che rende più praticabili refactoring ed evoluzione continua del sistema.
- Il test-first è utile soprattutto perché costringe a chiarire comportamento e confini prima dell'implementazione.

Parte 16 - JUnit 5: Test Automatici in Java

Nel ciclo di sviluppo moderno, implementazione e verifica non sono fasi separate, ma attività che procedono in modo intrecciato. Il codice viene scritto insieme ai test che ne descrivono il comportamento atteso, rendendo esplicite ipotesi e decisioni che altrimenti rimarrebbero implicite. L'esecuzione frequente dei test fornisce un feedback immediato sugli effetti delle modifiche introdotte. Questo feedback consente di individuare rapidamente errori e regressioni, prima che il contesto diventi troppo complesso o difficile da ricostruire. Nel tempo, la presenza dei test

influenza il design del sistema, favorendo soluzioni più semplici, modulari e facilmente evolvibili.

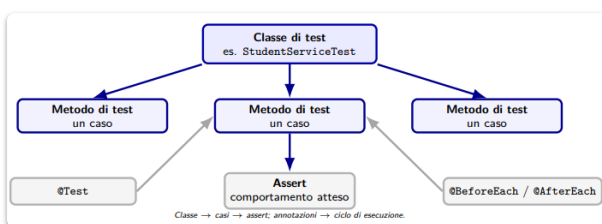
JUnit 5: è il framework di riferimento per la scrittura ed esecuzione di test automatici in Java, progettato per supportare in modo esplicito il testing di unità. Fornisce un insieme di annotazioni, asserzioni e meccanismi di esecuzione che permettono di descrivere in modo preciso il comportamento atteso del codice. L'obiettivo principale non è "verificare che il codice funzioni" in senso generico, ma rendere eseguibili e verificabili le regole e le ipotesi su cui il sistema è costruito. L'integrazione nativa con gli ambienti di sviluppo consente l'esecuzione frequente dei test come parte naturale del lavoro quotidiano. In questo senso, JUnit 5 rappresenta lo strumento operativo che rende praticabili i principi di testing discussi finora.

L'obiettivo non è esplorare tutte le funzionalità del framework, ma mostrare come tradurre in pratica i concetti visti nelle lezioni precedenti. L'attenzione sarà quindi concentrata su un sottoinsieme essenziale:

- struttura di una classe di test,
- gestione delle fixture,
- uso consapevole degli assert,
- leggibilità e naming dei test.

Questo è sufficiente per scrivere test di unità chiari, efficaci e coerenti con gli obiettivi del corso.

Struttura di base di un test JUnit: Un test JUnit è organizzato in una **classe di test**, separata dal codice di produzione. Ogni **metodo di test** rappresenta un singolo caso o una singola regola da verificare. Le annotazioni definiscono ruolo e momento di esecuzione dei metodi (**@Test**, **@BeforeEach**, **@AfterEach**). La struttura favorisce test piccoli, leggibili e indipendenti, con risultati facilmente interpretabili.



La classe di test: Una classe di test è una normale classe Java, collocata nel codice di test e dedicata esclusivamente alla verifica del comportamento. Per convenzione, il nome della classe di test riflette quello della classe sotto test, rendendo immediato il collegamento tra codice di produzione e verifica:

- *StudentServiceTest*
- *ExamServiceTest*

Ogni classe di test si concentra su una singola classe reale del sistema, evitando di mescolare responsabilità e comportamenti non correlati. Questa corrispondenza uno-a-uno favorisce la leggibilità, semplifica la navigazione del progetto e rende più chiaro cosa viene verificato e dove.

Il metodo di test: Un metodo di test è un normale metodo Java, annotato con **@Test**, che descrive un singolo comportamento atteso. Ogni metodo dovrebbe concentrarsi su una sola regola o condizione, evitando di mescolare più verifiche non correlate. Un buon metodo di test è tipicamente:

- piccolo, perché verifica un solo aspetto;
- focalizzato, perché ha uno scopo chiaro;
- indipendente, perché non dipende da altri test.

fail: test rosso → correggere il codice

```

1  @Test
2  void shouldNotEnrollStudentTwice() {
3      examService.enroll(student, exam);
4
5      try {
6          examService.enroll(student, exam);
7          fail("Second enrollment should not be allowed");
8      } catch (IllegalStateException e) {
9          // expected behavior
10     }
11 }

```

Fixture: Una fixture rappresenta lo stato iniziale in cui un test viene eseguito, costruito in modo controllato e intenzionale. Il suo scopo principale è garantire che ogni test parta da condizioni note e ben definite. L'uso di fixture permette di ottenere test che risultano:

- **puliti**, perché non dipendono da esecuzioni precedenti;
- **ripetibili**, perché producono sempre lo stesso risultato a parità di condizioni;
- **prevedibili**, perché falliscono solo quando il comportamento è realmente errato.

In assenza di una fixture esplicita, i test tendono a dipendere implicitamente dallo stato lasciato da altri test, diventando fragili e difficili da interpretare.

@BeforeEach: @BeforeEach identifica un metodo eseguito automaticamente prima di ogni metodo di test. Il suo ruolo è costruire la fixture, inizializzando lo stato necessario all'esecuzione dei test. In questo modo ogni test parte da condizioni identiche, senza dipendere dall'ordine di esecuzione o da altri test.

```

1  class ExamServiceTest {
2
3      private ExamService examService;
4      private Student student;
5      private Exam exam;
6
7      @BeforeEach
8      void setUp() {
9          examService = new ExamService();
10         student = new Student("12345");
11         exam = new Exam("Software Engineering");
12     }
13 }

```

@AfterEach: @AfterEach identifica un metodo eseguito automaticamente dopo ogni metodo di test. Serve soprattutto a ripristinare lo stato esterno modificato dal test, quando questo è necessario. Esempi tipici sono:

- cancellare file temporanei creati dal test;
- ripulire directory di lavoro;
- riportare il database di test a uno stato noto.

Nei test di unità semplici, il suo utilizzo è spesso limitato, perché la fixture viene ricostruita da zero per ogni test.

```

1 class ImportJobTest {
2
3     private Path tempCsv;
4     private TestDatabase db;
5
6     @AfterEach
7     void tearDown() throws IOException {
8         Files.deleteIfExists(tempCsv); // cleanup file temporaneo
9         db.reset();                  // reset stato DB di test
10    }
11 }

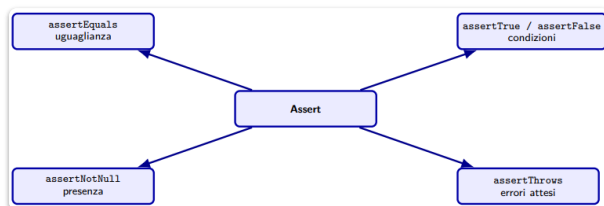
```

Perché i test devono essere indipendenti? L'ordine di esecuzione dei test non è garantito e non deve influenzarne il risultato. Ogni test dovrebbe costruire e usare il proprio stato, senza dipendere da effetti prodotti da altri test. Un test ben progettato fallisce solo quando il comportamento che verifica è effettivamente errato.



Assert: Un **assert** esprime in forma eseguibile una proprietà che deve risultare vera al termine dell'azione verificata dal test. La sua funzione non è "controllare un valore a caso", ma rendere esplicito il comportamento atteso del sistema in una certa condizione. Un test è quindi, in larga parte, una sequenza di scelte che portano a un punto in cui l'assert stabilisce cosa costituisce correttezza e cosa costituisce errore. Quando un test fallisce, la qualità dell'assert determina la qualità del feedback: un buon assert rende il fallimento comprensibile e direttamente collegabile alla causa.

JUnit mette a disposizione diverse famiglie di assert, ciascuna pensata per esprimere un tipo specifico di proprietà. La scelta dell'assert corretto contribuisce in modo diretto alla leggibilità del test e alla chiarezza del fallimento.



assertEquals: assertEquals verifica che il valore prodotto dal sistema coincida con il valore atteso in una determinata condizione. Un assert di questo tipo va letto come una frase dichiarativa: "dato questo scenario, il risultato deve essere questo". Quando è ben scelto, assertEquals rende il test immediatamente comprensibile e facilita l'interpretazione dei fallimenti.

Esempio 1	Esempio 2
<pre> 1 @Test 2 void totalPriceShouldBeAllItems() { 3 Order order = new Order(); 4 order.addItem(new Item(10)); 5 order.addItem(new Item(15)); 6 assertEquals(25, order.totalPrice()); 7 } </pre>	<pre> 1 @Test 2 void shouldStartWithSevenRollments() { 3 Exam exam = new Exam("M"); 4 assertEquals(0, exam.rollmentCount()); 5 } </pre>

assertTrue: assertTrue verifica che una condizione booleana sia vera. È utile quando si vuole controllare un vincolo espresso naturalmente come predicato (*isOpen()*, *isValid()*, *contains(...)*).

Esempio 1	Esempio 2
<pre> 1 @Test 2 void examShouldBeOpenForRollment() { 3 Exam exam = new Exam("M"); 4 exam.openRollment(); 5 assertTrue(exam.isRollmentOpen()); 6 } </pre>	<pre> 1 @Test 2 void enrolledListShouldContainStudent() { 3 exam.enrollStudent(student, exam); 4 assertTrue(exam.enrolledStudents() 5 .contains(student)); 6 } </pre>

assertFalse: assertFalse verifica che una condizione booleana sia falsa. È il complemento naturale di assertTrue per esprimere in modo esplicito ciò che il

sistema non deve permettere.

Esempio 1	Esempio 2
<pre>1 @test 2 void shouldNotEnrollTwice() { 3 Exam exam = new Exam("00"); 4 exam.openEnrollment(); 5 exam.closeEnrollment(); 6 assertTrue(exam.isEnrollmentOpen()); 7 }</pre>	<pre>1 @test 2 void shouldRejectEnrollmentOutside() { 3 Exam exam = new Exam("00"); 4 assertTrue(exam.canEnroll(today)); 5 assertFalse(exam.canEnroll(tomorrow)); 6 }</pre>

assertNotNull: assertNotNull verifica che un oggetto sia stato creato o restituito correttamente. È utile per controllare precondizioni minime prima di assert più specifici.

Esempio 1	Esempio 2
<pre>1 @test 2 void shouldCreateStudentWithValidId() { 3 Student student = studentService.register("12345"); 4 assertNotNull(student); 5 }</pre>	<pre>1 @test 2 void shouldFindStudentById() { 3 Exam exam = examRepository.findById("00-2026"); 4 assertNotNull(exam); 5 }</pre>

assertThrows: Le eccezioni non rappresentano solo errori tecnici, ma spesso esprimono regole di business violate. Testare che un'eccezione venga sollevata significa verificare che il sistema impedisca comportamenti non consentiti. In questo senso, un test può specificare in modo esplicito anche ciò che il sistema non deve permettere. Nella chiamata a assertThrows, () -> ... è una **funzione lambda**: rappresenta il blocco di codice che ci aspettiamo lanci l'eccezione (una funzione anonima senza argomenti).

Esempio 1	Esempio 2
<pre>1 @test 2 void shouldRejectInvalidId() { 3 ExamService enroll(student, exam); 4 assertThrows(IllegalArgumentException.class, 5 () -> enroll(student, exam)); 6 }</pre>	<pre>1 @test 2 void shouldRejectInvalidId() { 3 assertThrows(IllegalArgumentException.class, 4 () -> examService.recordTrade(student, 21)); 5 }</pre>

Framework di testing in altri linguaggi: JUnit 5 è il riferimento in Java, ma il concetto è trasversale: quasi ogni linguaggio ha framework analoghi per unit test, fixture e assert. Esempi comuni sono:

- Python: pytest, unittest;
- C++: GoogleTest, Catch2;
- JavaScript/TypeScript: Jest, Vitest.

Cambia la sintassi, ma resta uguale l'obiettivo: verificare in modo automatico il comportamento atteso.

≡ Python: esempio con pytest:

```
1 import pytest
2
3 def test_should_not_enroll_twice(exam_service, student, exam):
4     exam_service.enroll(student, exam)
5
6     with pytest.raises(ValueError):
7         exam_service.enroll(student, exam)
```

- pytest.raises(...) è l'analogo pratico di assertThrows(...).

≡ C++: esempio con GoogleTest:

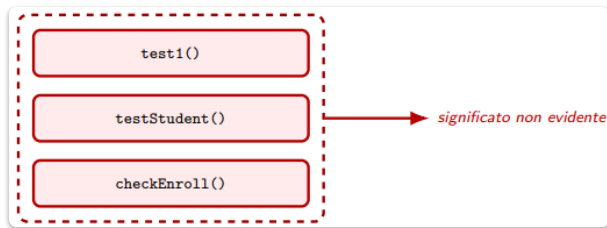
```
1 #include <gtest/gtest.h>
2
3 TEST(ExamServiceTest, ShouldNotEnrollStudentTwice) {
4     ExamService service;
5     service.enroll("s1", "se-2026");
6
7     EXPECT_THROW(service.enroll("s1", "se-2026"),
8                  std::logic_error);
9 }
```

- TEST(...) è una **macro** di GoogleTest che definisce un caso di test.
- EXPECT_THROW(...) esprime l'attesa di eccezione in stile C++.

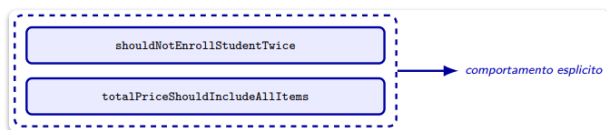
I test sono documentazione: I test automatici vengono spesso letti più frequentemente del codice di produzione, soprattutto quando si deve capire come il sistema dovrebbe comportarsi. Un buon test non si limita a verificare un risultato, ma racconta una storia: descrive uno scenario, un'azione e un comportamento atteso.

In questo contesto, il nome del test diventa parte integrante della documentazione del sistema, perché fornisce una descrizione sintetica e verificabile del comportamento.

Naming sbagliato: Alcuni nomi di test sono formalmente corretti, ma non comunicano quale comportamento venga verificato. Il risultato è che, quando un test fallisce, diventa necessario aprire e leggere il codice per capirne il senso.



Naming corretto: Un buon nome di test descrive in modo sintetico lo scenario e il comportamento atteso. Questo rende la suite più leggibile e riduce la necessità di aprire il codice per capire cosa viene verificato.



Naming e responsabilità: Ogni test dovrebbe essere responsabile della verifica di un singolo comportamento ben definito. Un nome vago è spesso il sintomo di un test che verifica troppe cose insieme o non ha uno scopo chiaro. Al contrario, un nome esplicito e anche relativamente lungo indica che il comportamento sotto test è stato chiarito prima ancora di guardare l'implementazione.

Struttura del progetto: Nei progetti Java moderni, il codice di produzione e il codice di test sono mantenuti separati in modo esplicito. Questa separazione riflette una distinzione concettuale:

- il codice in `src/main/java` definisce cosa fa il sistema;
- il codice in `src/test/java` definisce come quel comportamento viene verificato.

I test non sono codice "di supporto" o temporaneo, ma una parte strutturale del progetto, con pari dignità rispetto al codice di produzione. Mantenere questa distinzione rende più chiaro cosa appartiene al dominio del sistema e cosa appartiene alla sua specifica verificabile.

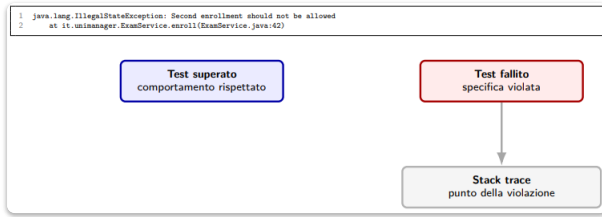
Eseguire i test dall'IDE: Gli ambienti di sviluppo moderni, come IntelliJ IDEA, integrano l'esecuzione dei test direttamente nel flusso di lavoro quotidiano. I test possono essere eseguiti a diversi livelli di granularità, a seconda del tipo di feedback che si vuole ottenere:

- un singolo metodo di test, per verificare rapidamente una regola specifica;
- un'intera classe di test, per controllare un componente nel suo insieme;
- l'intera suite, per avere fiducia globale prima di un cambiamento più ampio.

Questa integrazione rende naturale l'uso frequente dei test, riducendo l'attrito tra scrittura del codice e verifica del comportamento.

Interpretare i risultati: L'esito di un test fornisce un feedback immediato sul rispetto o meno del comportamento specificato. Un fallimento indica una violazione della specifica, non un errore generico del programma. Lo stack trace è uno

strumento per risalire al punto in cui il comportamento atteso non è stato rispettato.



Debug guidato dai test: Quando un test fallisce, il debug non parte “a caso”: parte da una specifica violata. Flusso pratico consigliato:

1. leggere nome del test e messaggio di fallimento;
2. individuare il punto utile nello stack trace;
3. riprodurre il caso in debug con breakpoint mirato;
4. osservare stato e variabili nel punto della violazione;
5. correggere, rilanciare il test, poi la suite.

Obiettivo: correggere la causa del fallimento, non solo il sintomo.

☰ Test rosso -> debug -> fix

```
1 org.opentest4j.AssertionFailedError:
2 Expected :2
3 Actual   :1
```

- Il test attende 2 iscritti, ma il servizio ne restituisce 1.
- Si imposta un breakpoint nel metodo `enroll(...)` e si verifica:
 - se il secondo inserimento viene davvero eseguito;
 - se una condizione blocca in anticipo il flusso;
 - se la collection usata per tracciare gli iscritti viene aggiornata.
- Correzione tipica: correzione della condizione o dell'aggiornamento stato.
- Verifica finale: test rosso diventa verde, poi riesecuzione della suite.

UniManager: I test vengono scritti su classi reali del sistema, non su esempi artificiali o semplificati. In particolare, vengono testati servizi applicativi come:

- `StudentService`, per la gestione degli studenti;
- `ExamService`, per la gestione di appelli, iscrizioni e voti.

Ogni classe di test verifica regole di business precise, come vincoli di iscrizione o condizioni di validità dei dati. Questo consente di osservare come JUnit venga usato su codice che potrebbe realmente evolvere nel tempo.

☰ Test completo:

```

1  class ExamServiceTest {
2
3      private ExamService examService;
4      private Student student;
5      private Exam exam;
6
7      @BeforeEach
8      void setUp() {
9          examService = new ExamService();
10         student = new Student("12345");
11         exam = new Exam("Software Engineering");
12     }
13
14     @Test
15     void shouldNotEnrollStudentTwice() {
16         examService.enroll(student, exam);
17         assertThrows(IllegalStateException.class,
18             () -> examService.enroll(student, exam));
19     }
20 }

```

Come rendiamo i test semplici: Per mantenere i test di unità rapidi e focalizzati, le dipendenze infrastrutturali vengono eliminate o sostituite. In particolare:

- i repository sono implementati in memoria;
- non viene utilizzato alcun database reale;
- non vengono coinvolti framework o componenti esterni.

Questo permette di concentrarsi esclusivamente sulla logica di dominio e sui comportamenti da verificare. Il risultato è una suite di test eseguibile rapidamente, adatta al feedback continuo durante lo sviluppo.

Cosa stiamo davvero testando: I test non verificano come il codice è scritto, ma cosa il sistema deve fare in determinate condizioni. In pratica, vengono testate:

- regole di dominio (vincoli, validazioni, divieti);
- comportamenti attesi in scenari leciti e non leciti;
- reazioni del sistema a stati non validi.

I dettagli implementativi possono cambiare nel tempo senza invalidare i test, se il comportamento rimane corretto. Questo rende i test uno strumento di protezione durante refactoring ed evoluzione del codice.

Verso il mocking: Finora i test hanno riguardato classi con dipendenze semplici e facilmente controllabili. Nei sistemi reali, però, le classi collaborano con componenti esterni come repository, servizi o gateway. Queste dipendenze rendono i test più complessi, se non vengono gestite in modo esplicito.

Messaggio finale: Scrivere test significa rendere esplicite le regole e le aspettative sul comportamento del sistema. Test ben strutturati migliorano il design, perché rendono evidenti responsabilità e dipendenze. Una buona suite di test riduce il rischio del cambiamento e rende il codice più facile da mantenere ed evolvere.

Riepilogo:

- JUnit 5 fornisce il supporto pratico per trasformare principi di unit testing in test automatici eseguibili con continuità.
- Struttura dei test, annotazioni e fixture servono soprattutto a ottenere test indipendenti, ripetibili e leggibili.

- Assert mirati e naming esplicito migliorano la qualità del feedback e rendono i test più utili anche come documentazione eseguibile.
- I test dovrebbero descrivere il comportamento atteso del sistema, non i dettagli accidentali dell'implementazione.
- Suite rapide e isolate aumentano la frequenza di esecuzione e rendono il testing parte naturale del workflow quotidiano.
- La base costruita qui prepara il passaggio al mocking per gestire collaboratori esterni e dipendenze più complesse

Parte 17 - Mocking con Mockito

Finora i test che abbiamo visto verificano soprattutto:

- input → output,
- regole di business in una singola classe o metodo.

Ma in un sistema reale le classi raramente sono autosufficienti:

- un service delega a un repository,
- un manager usa un gateway verso un servizio esterno,
- un componente notifica un altro componente.

In questi casi il comportamento corretto non è solo “calcolare bene”, ma anche parlare correttamente con i collaboratori. Consideriamo un esempio tipico:

- ExamService gestisce la logica di iscrizione a un appello;
- salva una registrazione tramite ExamRepository.

Se nel test usiamo un repository reale, spesso succede che:

- dobbiamo configurare un database o una persistenza,
- il setup diventa lungo e fragile,
- il test diventa più lento e meno deterministico.

La domanda corretta non è “posso farlo?”, ma “vale la pena portarsi dietro tutto questo per testare una regola di business?”. La soluzione è sostituire le dipendenze. Nel test vogliamo controllare l'ambiente, quindi:

- sostituiamo la dipendenza reale con un oggetto controllabile,
- decidiamo noi cosa restituisce e cosa registra.

Questo approccio produce tre benefici immediati:

- **isolamento:** testiamo davvero solo la classe sotto test;
- **velocità:** niente I/O, niente setup pesante;
- **precisione:** quando fallisce sappiamo dove guardare.

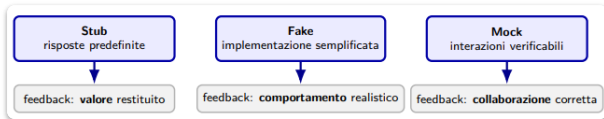
Qui entrano in gioco i sostituti di test.

Sostituti di test: Nei test usiamo spesso oggetti “sostituti” dei reali, ma con scopi diversi:

- ridurre il costo del setup (niente DB, rete, filesystem),
- rendere il test deterministico (stesso input $\Rightarrow$ stesso risultato),
- isolare il comportamento della classe sotto test dai dettagli esterni.

In pratica, tre categorie ricorrono continuamente:

- **Stub**: restituisce valori predefiniti per far proseguire il flusso (ci interessa il risultato, non come quel valore è stato ottenuto).
- **Fake**: implementazione semplificata ma funzionante (es. repository in-memory al posto di un database reale).
- **Mock**: oggetto controllato che non solo restituisce valori, ma permette di verificare le chiamate (chi ha chiamato cosa, con quali parametri).



Stub: risposte predefinite: Uno stub serve quando la classe sotto test:

- ha bisogno di un valore per poter proseguire l'esecuzione,
- ma non è importante capire come quel valore venga calcolato o prodotto.

In altre parole, lo stub viene usato per controllare lo scenario del test, fornendo risposte coerenti e prevedibili alle richieste della classe sotto test.

≡ Un repository che deve rispondere alla domanda "esiste già questa iscrizione?"

- nel test decidiamo esplicitamente la risposta, ad esempio true oppure false,
- così possiamo verificare il comportamento del service nei diversi casi.

Lo stub non registra chiamate e non impone vincoli sulle interazioni: il suo unico scopo è restituire valori utili al test. In breve: lo stub è "un distributore di risposte", semplice e mirato.

≡ Esempio di stub Java manuale:

Una classe minimale che fornisce risposte predefinite. L'obiettivo è quello di controllare in modo esplicito lo scenario del test attraverso i valori restituiti dal collaboratore.

```
1 class ExamRepositoryStub implements ExamRepository {
2     @Override
3     public boolean isEnrolled(Student s, Exam e) {
4         return true; // risposta predefinita per il test
5     }
6
7     @Override
8     public void saveRegistration(Student s, Exam e) {
9         // no-op: nello stub non ci interessa questa parte
10    }
11 }
```

Fake: implementazione semplificata ma reale: Un fake è un oggetto che funziona davvero, cioè esegue una logica coerente con quella reale, ma in una forma più leggera e più facile da controllare. A differenza di uno stub, un fake mantiene uno

stato interno e può essere utilizzato per far “girare” un flusso completo. Tipici esempi industriali includono:

- database in memoria (basati su liste o mappe) al posto di un DB vero,
- file system finto usato solo all'interno dei test,
- servizi locali o embedded invece di chiamate di rete reali.

Il fake è utile quando vogliamo un comportamento realistico e coerente con il sistema finale, ma non vogliamo pagare il costo dell'infrastruttura reale in termini di configurazione, tempo di esecuzione e fragilità dei test.

Esempio Java:

Un fake non restituisce solo valori fissi: mantiene uno stato interno e si comporta in modo realistico. Qui il repository salva davvero le iscrizioni, ma in memoria.

```
1 class InMemoryExamRepository implements ExamRepository {
2     private final Set<String> enrolled = new HashSet<>();
3
4     @Override
5     public boolean isEnrolled(Student s, Exam e) {
6         return enrolled.contains(key(s, e));
7     }
8
9     @Override
10    public void saveRegistration(Student s, Exam e) {
11        enrolled.add(key(s, e));
12    }
13
14    private String key(Student s, Exam e) {
15        return s.id() + ":" + e.code();
16    }
17 }
```

```
1 InMemoryExamRepository repo = new InMemoryExamRepository();
2 assertFalse(repo.isEnrolled(student, exam)); // prima del salvataggio
3 repo.saveRegistration(student, exam);
4 assertTrue(repo.isEnrolled(student, exam)); // dopo il salvataggio
```

Mock: verificare le interazioni: Un mock è pensato per testare comportamenti collaborativi, cioè situazioni in cui una classe non lavora da sola ma delega parte del lavoro:

- la classe sotto test deve chiamare un collaboratore,
- con certi parametri specifici,
- un certo numero di volte (o mai),
- eventualmente rispettando un certo ordine.

In questi casi l'output finale non è sempre sufficiente: anche se il metodo non restituisce nulla, vogliamo controllare che la collaborazione sia avvenuta correttamente. In breve: il mock è un “collaboratore controllato” che permette anche di osservare le interazioni tra oggetti.

Riassunto:

Stub risposte predefinite	Fake implementazione semplificata	Mock verifica interazioni
mi serve un valore	mi serve far girare il flusso	mi serve controllare la collaborazione

- Stub, fake e mock sono concetti di testing, non strumenti specifici.

- Nella pratica, per creare e configurare mock in Java possiamo usare una libreria di supporto: qui entra in gioco **Mockito**.

Mockito: Nei test collaborativi vogliamo sostituire alcuni oggetti reali con collaboratori controllabili. Mockito è una libreria Java per il testing. Mockito permette di creare questi collaboratori sotto forma di mock:

- possiamo configurarli come stub, quando servono risposte predefinite;
- possiamo usarli come mock, quando vogliamo verificare le interazioni.

Questo ci consente di testare la logica applicativa senza trascinarci dietro database, rete o infrastruttura reale. Mockito però non sostituisce il design:

- funziona bene solo se le dipendenze sono **sostituibili**;
- cioè se il codice rispetta il **Dependency Inversion Principle** (DIP).

Per usare Mockito in modo ordinato, conviene ragionare con una struttura fissa:

- **Given:** preparo scenario e collaboratori;
- **When:** eseguo l'azione da testare;
- **Then:** verifico risultato e/o collaborazione.

≡ Iscrizione a un appello:

Nel test vogliamo simulare uno scenario semplice: lo studente non è ancora iscritto e quindi il service può procedere.

```
1 // Given
2 // il repository indica che lo studente non e' gia' iscritto
3
4 // When
5 service.enroll(student, exam);
6
7 // Then
8 // il service richiede il salvataggio della registrazione
```

Il punto chiave è separare bene i tre momenti:

- prima preparo il comportamento dei collaboratori;
- poi invoco solo il metodo che voglio testare;
- infine controllo che il service abbia collaborato correttamente.

≡ Caso d'uso in UniManager:

Uno studente prova a iscriversi a un appello d'esame disponibile nel sistema. ExamService applica le regole di business principali e poi delega alcune operazioni a componenti collaboratori:

- controlli di pre-condizione, ad esempio la doppia iscrizione,
- salvataggio della registrazione tramite ExamRepository.

Il nostro obiettivo nel test è verificare il comportamento del service in isolamento:

- in condizioni valide, il service deve salvare correttamente la registrazione;
- in condizioni non valide, il service deve non invocare alcun salvataggio.

Costruzione testabile: DIP in pratica: Prima di entrare nei dettagli operativi di Mockito, è fondamentale guardare come è costruita la classe. Per poter scrivere test isolati e usare sostituti di test, la classe deve essere progettata in modo testabile:

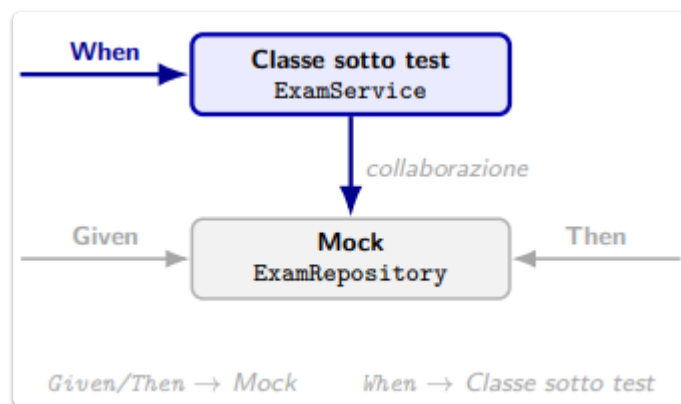
- deve dipendere da un'astrazione (ExamRepository),
- deve ricevere la dipendenza dall'esterno (costruttore o setter), invece di crearla internamente.

Questo è un esempio di applicazione pratica del Dependency Inversion Principle (DIP).

```

1 class ExamService {
2     private final ExamRepository repo;
3
4     ExamService(ExamRepository repo) {
5         this.repo = repo;
6     }
7
8     void enroll(Student s, Exam e) {
9         if (repo.isEnrolled(s, e)) {
10             throw new IllegalStateException("Student already enrolled");
11         }
12         repo.saveRegistration(s, e);
13     }
14 }
15

```



Schema mentale Mockito: Nei test collaborativi, il flusso tipico segue Given / When / Then:

- **Given:** si crea il mock e si definisce lo scenario (when);
- **When:** si invoca il metodo del service (azione sotto test);
- **Then:** si verifica la collaborazione (verify). Regola pratica: when prepara il comportamento atteso, verify controlla quello avvenuto.

Come creare un mock: A questo punto, nel test, vogliamo sostituire la dipendenza reale con un oggetto controllabile. Mockito fornisce il metodo `mock(...)` per creare un mock:

```

1 ExamRepository repo = mock(ExamRepository.class);

```

`mock(...)` non è una parola chiave di Java, ma è un metodo statico della libreria Mockito. Tipicamente importato con `import static org.mockito.Mockito.mock;` Questo oggetto:

- prende il posto del collaboratore reale nel test,
- non ha un'implementazione reale,

- registra le chiamate che riceve.

Mockito: definire comportamento (stub): Un mock, appena creato, non fa nulla di utile: restituisce valori di default (null, false, 0). Per questo, nel test, dobbiamo dirgli come deve rispondere quando viene interrogato. Mockito usa il costrutto `when(...).thenReturn(...)`:

```
1 when(repo.isEnrolled(student, exam)).thenReturn(false);
```

Questo codice non esegue il metodo:

- sta programmando il comportamento del mock,
- dice in modo esplicito: “se viene chiamato `repo.isEnrolled(student, exam)`, allora il mock restituisce false”.

In questo modo controlliamo lo scenario del test:

- iscrizione inesistente $\Rightarrow$ il service può procedere;
- se restituiamo true $\Rightarrow$ il service deve bloccare l'operazione.

Mockito: verify per controllare la collaborazione: Dopo aver eseguito l'azione sotto test, vogliamo controllare come la classe ha collaborato con i suoi dipendenti. Mockito fornisce il metodo `verify(...)` per osservare le chiamate già avvenute:

```
1 verify(repo).saveRegistration(student, exam);
```

Questo codice non esegue il metodo:

- non salva nulla,
- non modifica lo stato del sistema.

Sta invece affermando che:

- durante l'esecuzione del test,
- il service ha invocato quel metodo su quel collaboratore, con quei parametri.

Se quella chiamata non è avvenuta, il test fallisce. Questo test non dice “il DB contiene una riga”, ma una cosa più adatta a un unit test: il service ha chiesto al repository di salvare.

```

1  class ExamServiceMockitoTest {
2
3      private ExamRepository repo;
4      private ExamService service;
5
6      private Student student;
7      private Exam exam;
8
9      @BeforeEach
10     void setUp() {
11         // Given
12         repo = mock(ExamRepository.class);
13         service = new ExamService(repo);
14
15         student = new Student("12345");
16         exam = new Exam("Software Engineering");
17     }
18
19     @Test
20     void shouldSaveRegistrationWhenValid() {
21         // Given
22         when(repo.isEnrolled(student, exam)).thenReturn(false);
23
24         // When
25         service.enroll(student, exam);
26
27         // Then
28         verify(repo).saveRegistration(student, exam);
29     }
30 }

```

Esempio completo di un caso valido: Il test verifica il comportamento di ExamService in una situazione valida di iscrizione. La struttura segue una sequenza standard:

- Preparazione dell'ambiente e dei sostituti di test (**Given**);
- Esecuzione dell'azione sotto test (**When**);
- Verifica del comportamento collaborativo (**Then**).

Flusso concreto del caso valido:

- **Given:** isEnrolled(...) restituisce false (studente non ancora iscritto);
- **When:** il service esegue enroll(student, exam);
- **Then:** il repository deve ricevere saveRegistration(student, exam).

L'obiettivo del test è verificare che:

- l'iscrizione risulti consentita,
- il repository venga correttamente invocato.

```

1  @Test
2  void shouldNotSaveIfAlreadyEnrolled() {
3      // Given
4      when(repo.isEnrolled(student, exam)).thenReturn(true);
5
6      // When
7      assertThrows(IllegalStateException.class,
8          () -> service.enroll(student, exam));
9
10     // Then
11     verify(repo, never()).saveRegistration(student, exam);
12 }

```

Caso non valido: Il test verifica il comportamento di ExamService in una situazione NON valida di iscrizione. Scenario considerato: lo studente risulta già iscritto all'appello. L'aspettativa è duplice:

- l'operazione deve essere rifiutata,
- il repository non deve essere invocato per il salvataggio.

Flusso concreto del caso non valido:

- **Given:** isEnrolled(...) restituisce true (studente già iscritto);
- **When:** il service esegue enroll(student, exam)

- **Then:** viene lanciata `IllegalStateException` e `saveRegistration(...)` non deve essere chiamato.

Il test verifica quindi la segnalazione dell'errore e l'assenza di collaborazioni indesiderate.

Varianti di verify: times / never: Con `verify(...)` non si controlla solo che un metodo sia stato chiamato, ma anche quante volte è stato invocato. Mockito consente di esprimere vincoli quantitativi sulle interazioni:

```
1 verify(repo, times(1)).saveRegistration(student, exam);
2 verify(repo, never()).deleteRegistration(student, exam);
```

Nota didattica:

- `times(1)` è il comportamento di default di `verify`, quindi spesso è ridondante;
- diventa utile quando il numero di chiamate è parte esplicita del comportamento atteso;
- `never()` è invece utile per affermare che una collaborazione non deve avvenire.

Mocking e buon design: DIP come prerequisito: Il mocking funziona bene solo quando il design lo permette:

- dipendenze esplicite e visibili,
- costruttore (o setter) che riceve i collaboratori,
- dipendenza da interfacce e astrazioni, non da implementazioni concrete.

Se una classe crea internamente le proprie dipendenze (ad esempio usando `new` su un repository):

- la dipendenza non è sostituibile,
- il test deve portarsi dietro l'infrastruttura reale,
- il codice risulta più accoppiato e meno evolvibile.

I mock quindi non sono solo uno strumento di testing: rappresentano anche un feedback diretto sul design e sul rispetto del Dependency Inversion Principle (DIP).

Mocking come feedback: I test con mock possono essere letti anche come uno strumento di diagnosi del design. Se per testare una singola classe è necessario creare:

- molti mock,
- molte configurazioni `when/thenReturn`,
- numerosi `verify` molto dettagliati,

allora spesso il problema non è Mockito:

- la classe sotto test probabilmente ha troppe responsabilità (violazione di SRP);
- oppure dipende da dettagli concreti invece che da astrazioni (violazione di DIP);
- oppure espone un comportamento poco coeso e difficile da isolare.

In questi casi il test sta facendo emergere la necessità di un refactoring del design.

Cattive pratiche: Alcuni errori ricorrenti nell'uso dei mock includono:

- mockare tutto, inclusi oggetti di dominio semplici e stabili;

- verificare interazioni superflue (test troppo “pettegoli”);
- scrivere test che replicano l'implementazione interna (“prima chiama A, poi B, poi C”);
- abuso di verifiche troppo restrittive su interazioni secondarie.

L'effetto tipico di queste pratiche è:

- test fragili, che falliscono a ogni refactoring,
- perdita del beneficio principale dei test automatici: la possibilità di cambiare codice con fiducia.

Regole pratiche per usare bene i mock: Alcune regole empiriche utili per un uso corretto dei mock:

- mockare i collaboratori, non l'oggetto sotto test;
- verificare solo ciò che conta per il comportamento osservabile;
- preferire test che descrivono cosa deve accadere, non come accade internamente;
- se un test richiede troppi mock, considerare una riduzione di responsabilità o dipendenze.

L'obiettivo finale è avere test che rimangano stabili nel tempo, anche quando cambia l'implementazione, purché il comportamento esterno resti coerente.

Riepilogo:

- Stub, fake e mock (sostituti di test) servono a scopi diversi e vanno scelti in base al tipo di verifica.
- Mockito è utile soprattutto per test collaborativi: configurare risposte dei mock e verificare interazioni.
- Il mocking funziona bene quando il design è testabile (DIP, dipendenze sostituibili).
- Troppi mock o verifiche troppo dettagliate sono un segnale di design da rifattorizzare. 28

Parte 18 - Git: Controllo di Versione Distribuito

Senza un sistema di controllo di versione, la gestione delle modifiche tende a degenerare in:

- copie multiple dello stesso progetto (progetto_finale_v 3_definitivo 2.zip),
- file sovrascritti e versioni perse,
- difficoltà a ricostruire uno stato funzionante dopo un errore.

In questo scenario, due domande diventano rapidamente costose (tempo, stress, rischio):

- Chi ha cambiato cosa?
- Quando e perché è stato cambiato?

Git introduce una risposta strutturata:

- ogni modifica significativa viene registrata come commit,
- la storia risulta tracciabile, consultabile e verificabile.

Git è un sistema di controllo di versione distribuito:

- ogni copia locale del repository contiene l'intera storia del progetto,
- il lavoro quotidiano non dipende dalla disponibilità continua di un server centrale.

Il concetto fondamentale è il commit: ogni commit rappresenta una fotografia coerente e consistente dello stato del progetto in un dato momento.

Contesto storico e nome:

- Git è stato creato da Linus Torvalds nel 2005;
- il nome è stato usato in modo ironico; compare anche il backronym "Global Information Tracker".

Il valore pratico di questo modello è operativo:

- è possibile sperimentare in modo controllato,
- correggere errori senza perdere informazioni,
- recuperare stati precedenti grazie alla storia registrata.

Git non è GitHub:

- Git:
 - strumento locale di controllo di versione,
 - responsabile della gestione della storia del progetto,
 - supporta operazioni come commit, merge e recupero degli errori.
- GitHub (così come GitLab, Bitbucket,...):
 - servizio di hosting per repository remoti,
 - consente la condivisione del codice tra più sviluppatori,
 - aggiunge funzionalità di collaborazione e automazione sopra Git.

Setup minimo: init, clone, status: L'inizializzazione di un progetto Git può avvenire in due modi principali:

- creare un nuovo repository all'interno di una cartella esistente tramite git init,
- ottenere una copia locale di un repository già esistente tramite git clone.

In entrambi i casi, è fondamentale verificare lo stato del repository:

- il comando git status fornisce una fotografia immediata della situazione corrente del progetto,
- permette di capire quali file sono modificati, quali sono già tracciati e quali no.

```
1 git init
2 git status
3
4 git clone https://github.com/<org>/<repo>.git
5 git status
```

Lettura rapida di git status: Il comando git status fornisce una sintesi immediata dello stato corrente del repository. In particolare, indica:

- il branch attualmente attivo (tipicamente main),
- i file che risultano modificati ma non ancora selezionati,
- i file che si trovano in staging e sono pronti per il commit.

Dal punto di vista operativo, git status è il comando di riferimento:

- chiarisce sempre “a che punto si è”,
- riduce il rischio di operazioni non intenzionali.

Regola pratica: in caso di dubbio sullo stato del progetto, eseguire git status prima di qualsiasi altra operazione.

```
1 git status
2 # modified:   src/...
3 # untracked:  notes.txt
4 # changes to be committed: ...
```

Scegliere cosa entra nel commit: Git non include automaticamente tutte le modifiche in un commit: la selezione è un'operazione esplicita e controllata. Il comando `git add` sposta una modifica dall'area di lavoro alla staging area. Questo passaggio permette di:

- rendere esplicito cosa si intende salvare nella storia,
- costruire commit coerenti anche in presenza di più modifiche,
- evitare l'inclusione involontaria di file o cambiamenti non rilevanti.

```
1 git status
2 # modified:   src/main/java/unimanager/ExamService.java
3 # modified:   src/test/java/unimanager/ExamServiceTest.java
4 # untracked:  notes.txt
5
6 git add src/main/java/unimanager/ExamService.java
7 git add src/test/java/unimanager/ExamServiceTest.java
8
9 git status
10 # changes to be committed:
11 #   modified:   src/main/java/unimanager/ExamService.java
12 #   modified:   src/test/java/unimanager/ExamServiceTest.java
13 #
14 # untracked files:
15 #   notes.txt
```

Diff: Prima di creare un commit è importante comprendere quali modifiche verranno effettivamente salvate. Git mette a disposizione due comandi distinti per questo scopo:

1. `git diff`: mostra le differenze presenti nel working tree ma non ancora aggiunte alla staging area;
2. `git diff --staged` (o `--cached`): mostra invece le modifiche già selezionate e pronte per il commit.

Questa distinzione consente di:

- verificare che ogni commit contenga solo ciò che è intenzionale,

- individuare rapidamente modifiche accidentali o incomplete,
- mantenere una storia del progetto più chiara e leggibile.

```
1 git diff          # modifiche non ancora in staging
2 git diff --staged # modifiche pronte per il commit
```

Regola operativa: prima di eseguire `git commit`, controllare sempre l'output di almeno uno dei due comandi.

Commit: Un commit rappresenta una fotografia coerente dello stato del progetto in un preciso momento. Ogni commit include metadati essenziali:

- l'autore della modifica,
- il timestamp,
- un messaggio descrittivo.

Il messaggio di commit non è un dettaglio formale:

- costituisce una forma di documentazione tecnica,
- serve a spiegare cosa è cambiato e, soprattutto, perché.

```
1 git commit -m "Prevent double enrollment in ExanService"
```

Messaggi di commit: I messaggi di commit svolgono un ruolo centrale nella comprensione e nella manutenzione della storia del progetto.

- Un buon messaggio di commit:
 - descrive chiaramente cosa è stato modificato,
 - esplicita, quando rilevante, perché la modifica è stata introdotta,
 - rimane sintetico, ma sufficientemente informativo.
- Al contrario, messaggi poco informativi:
 - non aggiungono contesto tecnico,
 - non aiutano a ricostruire le motivazioni delle scelte,
 - rendono la storia del progetto difficile da leggere nel tempo.

≡ Esempi tipici di messaggi da evitare:

- "fix"
- "changes"
- "final version"

Esempio: Add validation to prevent double enrollment
(cosa) prevenzione della doppia iscrizione (perché) applicazione di una regola di business

Log: Il comando `git log` consente di esplorare la storia delle modifiche registrate nel repository. In particolare, è utile per:

- comprendere l'evoluzione del codice nel tempo,
- individuare il commit che ha introdotto un errore o una regressione,
- ricostruire le motivazioni tecniche dietro determinate scelte progettuali.

Per una visualizzazione più compatta e leggibile, Git mette a disposizione alcune opzioni di formattazione:

```
1 git log --oneline --decorate --graph --all
```

Questa vista aiuta a cogliere rapidamente:

- la sequenza dei commit,
- la presenza di rami e merge,
- lo stato dei puntatori principali (branch e tag).

Granularità: La dimensione di un commit influenza direttamente leggibilità, manutenzione e capacità di recupero nel tempo.

- Commit piccoli (coerenti e focalizzati):
 - risultano più semplici da comprendere e discutere,
 - possono essere annullati con minore impatto (revert più mirato),
 - producono una storia del progetto più chiara e navigabile.
- Commit molto grandi (eterogenei):
 - nascondono cambiamenti non intenzionali o non correlati,
 - aumentano la probabilità di conflitti durante l'integrazione,
 - rendono più difficile isolare la causa di un problema e tornare indietro.

Regola operativa: un commit dovrebbe rappresentare una singola idea o modifica logica.

.gitignore: In un progetto Git non tutti i file presenti nella cartella di lavoro devono essere tracciati e salvati nella storia. In particolare, è buona pratica escludere:

- file temporanei o generati automaticamente (ad esempio tmp, log),
- output del processo di build (come target/ o build/),
- file specifici dell'ambiente di sviluppo (configurazioni di IDE come .idea/, *.iml).

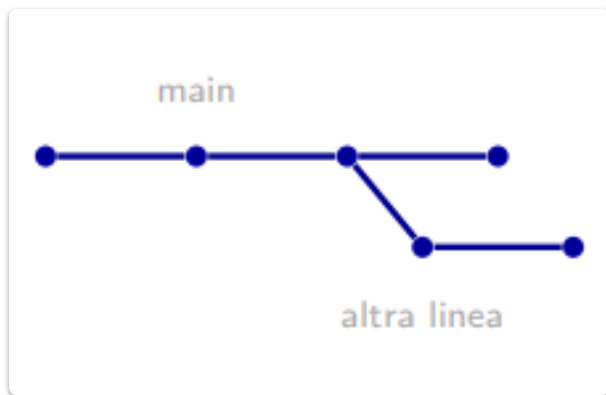
Il file .gitignore permette di dichiarare in modo esplicito quali file o directory Git deve ignorare:

- tali elementi restano locali,
- non vengono inclusi nei commit,
- non “inquinano” la storia del progetto.

```
1 # Maven / Gradle
2 target/
3 build/
4
5 # IntelliJ
6 .idea/
7 *.iml
8
9 # OS / temp
10 .DS_Store
11 *.log
```

Branch: Un branch rappresenta una linea alternativa della storia, che nasce da un commit esistente. `main` è la linea principale, ma da un punto preciso la storia può biforcarsi. È sufficiente sapere che:

- le linee di sviluppo nascono da un commit comune,
- quando si ricongiungono è necessaria un'operazione di merge.

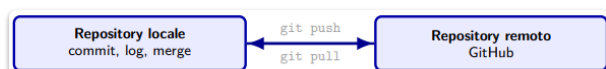


Locale vs remoto: In un contesto di sviluppo, è comune distinguere tra:

- **repository locale:** copia presente sul computer, usata per lavorare e creare commit;
- **repository remoto:** copia condivisa, usata come punto di riferimento per il team (ad esempio su GitHub).

Il lavoro quotidiano avviene principalmente in locale, mentre la condivisione avviene tramite lo scambio di commit. In questo modello:

- `git push` pubblica i commit locali sul remoto,
- `git pull` scarica e integra i commit presenti sul remoto.



Remote: In un repository Git, il remoto principale è convenzionalmente chiamato `origin`. Il remoto rappresenta il punto di sincronizzazione tra il lavoro locale e la copia condivisa del progetto. Alcuni comandi di base permettono di gestire questa relazione:

- `git remote -v`: consente di visualizzare i remoti configurati e i relativi URL;
- `git push`: invia i commit locali al repository remoto, rendendoli disponibili agli altri;
- `git pull`: recupera i commit dal remoto e li integra nel repository locale.

L'uso consapevole di questi comandi è essenziale per mantenere allineate le diverse copie del progetto.

```
1  git remote -v
2  git push
3  git pull
```

Push: Il comando `git push` serve a inviare i commit locali al repository remoto. Con il push, le modifiche:

- smettono di essere solo locali,
- diventano visibili e accessibili agli altri membri del progetto.

Un modello mentale utile distingue chiaramente due momenti:

- `commit`: la storia viene salvata localmente,
- `push`: la storia viene resa pubblica.

Buona pratica operativa:

- eseguire push frequenti,
- ma solo su commit coerenti, compilabili e significativi.



Pull: Il comando `git pull` serve a recuperare i commit presenti sul remoto e ad aggiornarne la copia locale. In pratica, il pull combina due operazioni:

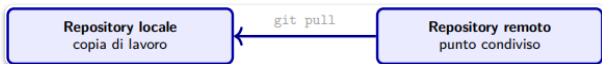
- scarica i nuovi commit dal repository remoto,
- integra tali commit nella storia locale.

Possono emergere problemi quando:

- nel frattempo altri hanno modificato gli stessi file o le stesse righe,
- la storia locale e quella remota devono essere integrate tramite merge.

Regola operativa:

- prima di iniziare a lavorare su un progetto condiviso: eseguire `git pull`,
- prima di pubblicare nuove modifiche: eseguire `git pull` per ridurre conflitti e sorprese.



Merge: Un merge è l'operazione che consente di integrare due linee di sviluppo in una storia comune. Il merge si verifica tipicamente quando:

- il repository locale viene aggiornato con commit remoti,
- più sviluppatori hanno lavorato in parallelo.

Se le modifiche non si sovrappongono, Git integra automaticamente. In caso contrario, è richiesto un intervento esplicito: si genera un conflitto di merge.



Conflitti: Un conflitto indica semplicemente che Git non è in grado di decidere automaticamente quale versione di una modifica mantenere. La presenza di un conflitto:

- non segnala un errore concettuale,
- non è “colpa” di qualcuno,
- è una conseguenza normale del lavoro in parallelo sullo stesso codice.

La gestione corretta di un conflitto segue alcuni passi chiave:

1. comprendere l'obiettivo delle due modifiche in conflitto;
2. scegliere la versione corretta o combinarle in modo coerente;
3. ricompilare il progetto ed eseguire i test per verificare il risultato.

L'obiettivo non è evitare ogni conflitto, ma saperli risolvere in modo consapevole e controllato.

Come appare un conflitto: Quando Git non riesce a risolvere automaticamente un merge, inserisce dei marcatori di conflitto direttamente nel file coinvolto. Questi marcatori delimitano le due versioni concorrenti della modifica:

```
1 <<<<<< HEAD
2 return enrollIfValid(student, exam);
3 =====
4 return enroll(student, exam);
5 >>>>>> origin/main
```

La risoluzione del conflitto richiede un intervento manuale:

- analizzare le due versioni e comprenderne l'intento,
- rimuovere i marcatori (<<<, ==, >>>),
- lasciare nel file una sola versione coerente del codice.

Una volta risolto il conflitto:

- il file va aggiunto nuovamente allo staging con `git add` ,
- il merge si conclude con un `git commit` dedicato.

Example

- Contesto: due sviluppatori lavorano sullo stesso file, uno ha già fatto push sul remoto, l'altro prova a fare push con una storia non aggiornata.
- Tentativo di push:

```
1 git push
2 # rejected: non-fast-forward
3 # error: failed to push some refs
```

- Soluzione: aggiornare e integrare le modifiche remote

```
1 git pull
2 # Git prova il merge
3
4 # se c'e' conflitto:
5 # modificare i file, risolvere manualmente
6
7 git add file.java
8 git commit # completa il merge
9
10 git push    # ora funziona
```

- Idea chiave: il push fallisce perché la storia remota è più avanti, il pull integra le modifiche (merge), poi il push può andare a buon fine.

Undo: Git consente di recuperare situazioni non desiderate in modo controllato, senza interventi improvvisati. Il comando corretto dipende da dove si trovano le modifiche.



La distinzione tra “privato” e “pubblico” è determinante: alcuni comandi riscrivono la storia, altri la compensano.

Ripristinare file non committati: Quando sono presenti modifiche nel working tree che non devono essere mantenute, è possibile tornare allo stato dell'ultimo commit. In questo caso, il comando `git restore` ripristina il contenuto del file come registrato nella storia:

```
1 git restore src/main/java/unimanager/ExamService.java
```

È anche possibile intervenire sullo staging:

- se un file è stato aggiunto per errore con `git add`,
- lo si può rimuovere dallo staging senza perdere le modifiche locali.

```
1 git restore --staged src/main/java/unimanager/ExamService.java
```

In entrambi i casi, il ripristino agisce solo sullo stato locale e non modifica la storia dei commit.

Modificare l'ultimo commit locale: Un caso frequente durante lo sviluppo locale è:

- un file dimenticato nel commit precedente,
- un messaggio di commit impreciso o errato.

Altro caso tipico: aver committato per errore un file molto grande (ad esempio 1 GB) che non doveva entrare nella storia. L'opzione `--amend` consente di modificare l'ultimo commit, aggiornandone il contenuto e/o il messaggio. Questa operazione riscrive la storia e deve essere usata solo su commit che non sono ancora stati pubblicati.

```
1 git add src/test/java/unimanager/ExamServiceTest.java
2 git commit --amend
```

Regola operativa:

- usare `--amend` solo su commit locali,
- evitare sempre `--amend` su commit già pushati.

Reset: Il comando `git reset` sposta il puntatore della storia locale ad un commit precedente. È uno strumento potente ma delicato:

- utile per correggere errori in locale,
- potenzialmente pericoloso se applicato a commit già pubblici.

Il comportamento di `git reset` dipende dall'opzione utilizzata:

```
1 git reset --soft <commit> # mantiene le modifiche nello staging
2 git reset --mixed <commit> # mantiene le modifiche nel working tree
3 git reset --hard <commit> # elimina le modifiche locali (attenzione!)
```

Regola operativa fondamentale:

- usare `git reset` solo su commit non pushati,
- evitare `--hard` se non si è certi di voler perdere il lavoro locale.

`git reset --soft: fondere commit`

- Situazione:
 - C1: struttura progetto
 - C2: aggiungo funzione
 - C3: fix piccolo bug
- Problema: C2 e C3 dovevano essere un unico commit.
- Soluzione:

```
1 git reset --soft HEAD-2
2 git commit -m "aggiunta funzione + fix"
```

- Effetto:
 - torno a C1
 - modifiche di C2 e C3 nello staging
 - creo un nuovo commit unico

`git reset --mixed: riorganizzare commit`

- Situazione:
 - C1: base
 - C2: modifica A + modifica B (troppo insieme)
- Problema: voglio separare A e B in due commit distinti.
- Soluzione:

```
1 git reset --mixed HEAD-1
2
3 # ora scelgo cosa aggiungere
4 git add fileA
5 git commit -m "modifica A"
6
7 git add fileB
8 git commit -m "modifica B"
```

- Effetto:
 - modifiche restano nei file
 - staging vuoto → scelgo io cosa includere

`git reset --hard: tornare indietro e scartare tutto`

- Situazione:
 - ho creato un commit locale sbagliato
 - voglio eliminarlo insieme alle sue modifiche

- Obiettivo: tornare allo stato del commit precedente.
- Soluzione:

```
1 git reset --hard HEAD~1
```

- Effetto:
 - l'ultimo commit locale viene eliminato dalla storia
 - staging e working tree tornano allo stato del commit precedente
 - eventuali modifiche locali non salvate vengono perse

Revert: Quando un commit è già stato condiviso con altri, la strategia più sicura per annullarne gli effetti è `git revert`. Il comando `git revert`:

- crea un nuovo commit che annulla logicamente le modifiche introdotte da un commit precedente,
- preserva la storia esistente senza eliminarla o riscriverla.

Questo approccio è particolarmente adatto a repository condivisi, perché mantiene una cronologia completa e tracciabile.

```
1 git revert <commit>
```

Messaggio operativo da ricordare:

- reset riscrive la storia,
- revert la compensa in modo sicuro.

Pubblico vs privato. Prima di utilizzare comandi potenzialmente invasivi, è fondamentale chiarire il contesto in cui si sta operando. Due domande guida aiutano a scegliere l'azione corretta:

- il commit è già stato pubblicato su un repository remoto?
- altri sviluppatori potrebbero basarsi su quel commit?

In termini generali, vale la seguente distinzione:

- in locale è possibile adottare strategie più aggressive (ad esempio `reset`, `--amend`);
- su repository condivisi è preferibile usare operazioni conservative che preservano la storia (ad esempio `revert`).

In caso di dubbio: consolidare il lavoro con un commit, rimandare il push e confrontarsi prima di procedere.

Tag: I tag permettono di associare un nome stabile a un commit specifico della storia. Sono tipicamente usati per identificare:

- versioni rilasciate (v 1.0, v 1.1),
- snapshot significativi del progetto,
- punti di riferimento per build, deploy o documentazione.
- indica esattamente quel commit e nient'altro.

I tag non vengono pushati automaticamente:

```
1 git tag -a v1.0 -m "Prima versione stabile"
2 git push origin v1.0
```



Il tag v1.0 rimane fissato su un commit anche se HEAD avanza

Errori comuni (e come evitarli):

- Aggiungere tutto indiscriminatamente al commit (`git add .`) senza verificare cosa viene incluso.
- Usare messaggi di commit vaghi o generici: "fix" non fornisce alcuna informazione utile (nemmeno a distanza di poche settimane).
- Creare commit senza aver controllato le differenze con `git diff` o `git diff --staged`.
- Aggiornarsi raramente dal remoto, accumulando conflitti che emergono solo all'ultimo momento.
- Utilizzare comandi potenzialmente distruttivi senza aver chiarito se il lavoro è già pubblico o condiviso.
- Dimenticare che il file `.gitignore` va condiviso:
 - funziona localmente, ma
 - per avere effetto su tutto il team deve essere committato.

Un flusso minimo e sano:

1. `git status` (verifica dello stato del repository)
2. sviluppo di una modifica piccola e circoscritta
3. `git diff` (analisi delle modifiche introdotte)
4. `git add` (selezione esplicita dei contenuti del commit)
5. `git commit -m "..."` (una singola modifica logica)
6. `git pull` (sincronizzazione con il remoto, se presente)
7. `git push` (pubblicazione dei commit)

Questo flusso non è pensato per essere sofisticato, ma per essere robusto, ripetibile e facilmente verificabile. L'adozione sistematica di questa sequenza riduce conflitti, errori evitabili e interventi correttivi non pianificati.

Riepilogo:

- Git è utile soprattutto perché rende tracciabili le modifiche e riduce il caos della collaborazione sul codice.
- Il modello `working tree` / `staging area` / `repository` è la chiave per capire cosa sta succedendo prima di usare i comandi.
- Un flusso quotidiano semplice e ripetibile (`status`, `diff`, `add`, `commit`, `pull`, `push`) previene molti errori evitabili.
- Merge e conflitti fanno parte del lavoro normale: gestirli presto e spesso è meglio che rimandarli.
- Le operazioni di undo richiedono attenzione al contesto (locale vs condiviso), non solo memoria dei comandi.

- Buone pratiche di commit e versionamento migliorano review, tracciabilità e manutenzione del progetto.

Parte 19 - Continuous Integration (CI) e DevOps

Nelle lezioni precedenti sono stati introdotti diversi strumenti fondamentali per migliorare la qualità del software sviluppato:

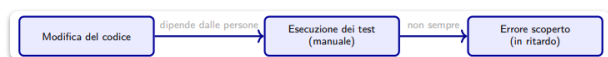
- i test automatici, utilizzati per verificare in modo sistematico il comportamento atteso del codice;
- il mocking, impiegato per isolare le dipendenze e rendere i test più mirati e controllabili;
- Git, come sistema di controllo di versione per tenere traccia dell'evoluzione del progetto nel tempo.

L'uso combinato di questi strumenti contribuisce a rendere il codice più affidabile e più semplice da mantenere. Rimane tuttavia una questione aperta: come garantire che tali verifiche vengano eseguite in modo regolare, sistematico e affidabile nel tempo?

I test automatici rappresentano uno strumento efficace di verifica solo nella misura in cui vengono effettivamente eseguiti. In un processo basato esclusivamente su attività manuali:

- l'esecuzione dei test dipende dall'attenzione, dall'esperienza e dalla disciplina delle persone coinvolte;
- è possibile dimenticare di lanciare i test prima di condividere una modifica o di proseguire con altro lavoro;
- gli errori possono emergere quando il codice è già stato modificato più volte, rendendo più difficile individuarne la causa.

Questo tipo di approccio tende a diventare fragile e poco affidabile man mano che il progetto cresce in dimensione e complessità.



Il problema delle regressioni: Una regressione si verifica quando una modifica introduce un errore in una funzionalità che in precedenza funzionava correttamente. Le regressioni sono spesso non intenzionali:

- una modifica locale può avere effetti indiretti su altre parti del sistema;
- il problema può non essere immediatamente evidente.

Senza verifiche sistematiche, le regressioni tendono ad accumularsi nel tempo.

Controllo manuale e controllo automatico: Il controllo manuale del codice presenta alcuni limiti strutturali, legati alla sua dipendenza dalle persone che lo eseguono:

- non è uniforme, poiché sviluppatori diversi possono adottare criteri o livelli di attenzione differenti;
- non è sempre ripetibile nello stesso modo, specialmente in momenti diversi o sotto pressione;
- può essere influenzato da fattori esterni, come fretta, distrazione o carichi di lavoro elevati.

Il controllo automatico, al contrario, introduce un meccanismo più stabile:

- segue regole esplicite e definite in modo formale;
- viene eseguito sempre nelle stesse condizioni operative;
- produce risultati oggettivi, ripetibili e verificabili nel tempo.

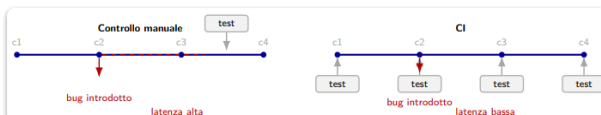
L'idea alla base della CI: La CI nasce con l'obiettivo di rendere automatiche le verifiche sul codice durante il suo sviluppo. L'idea di fondo è che ogni modifica introdotta nel progetto debba essere controllata in modo sistematico, e non soltanto in maniera occasionale o informale. In questo modo, la verifica diventa una parte integrante del processo di sviluppo del software, e non un'attività opzionale affidata alla sola buona volontà.



CI: CI è una pratica di sviluppo software che rende l'integrazione delle modifiche un'attività frequente e controllata. In un processo basato sulla CI:

- le modifiche vengono integrate frequentemente nel progetto;
- il sistema viene verificato automaticamente ad ogni integrazione.

L'obiettivo principale è ridurre il tempo che intercorre tra l'introduzione di un problema e la sua individuazione.



Cosa non è la CI: La CI non sostituisce le attività di progettazione del software né la progettazione dei test automatici:

- richiede comunque decisioni progettuali corrette;
- si basa sulla presenza di test adeguati.

La CI non garantisce, da sola, l'assenza di errori nel software:

- verifica solo ciò che viene effettivamente controllato;
- non intercetta problemi non coperti dai test.

La CI non è legata a una tecnologia o a una piattaforma specifica, ma rappresenta una pratica di processo indipendente dagli strumenti utilizzati.

CI come estensione del testing: I test automatici definiscono cosa deve essere verificato nel sistema:

- descrivono il comportamento atteso del codice;
- formalizzano le condizioni di correttezza.

La CI stabilisce quando tali verifiche devono avvenire:

- ad ogni integrazione di una modifica;
- senza dipendere da decisioni manuali.

Il contenuto dei test rimane invariato:

- ciò che cambia è la frequenza di esecuzione;
- le verifiche diventano sistematiche e ripetibili nel tempo.

Quando avviene la verifica: La verifica viene eseguita ogni volta che il codice del progetto viene modificato:

- in seguito all'introduzione di nuove funzionalità;
- dopo correzioni o refactoring del codice esistente.

In pratica, ciò avviene tipicamente quando una modifica viene condivisa nel repository remoto (ad esempio con push):

- il sistema intercetta l'aggiornamento del codice;
- la verifica viene avviata senza azioni esplicite da parte degli sviluppatori.

L'intero processo avviene in modo automatico:

- l'esecuzione segue sempre le stesse regole;
- i risultati sono ripetibili e indipendenti dalle persone.

Collegamento con Git: Nel flusso di lavoro visto con Git, una modifica diventa “parte della storia” quando viene registrata con un commit. In un contesto collaborativo, i commit vengono poi condivisi su un repository remoto, rendendo la modifica visibile anche ad altri. La CI si inserisce esattamente in questo passaggio:

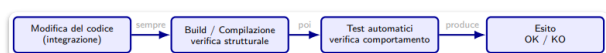
- ogni modifica condivisa può attivare automaticamente una verifica;
- build e test vengono eseguiti con regole sempre uguali per tutti;
- il risultato rende immediatamente chiaro se l'integrazione è “sana”.

In questo modo, la verifica non dipende più da scelte individuali, ma diventa una proprietà del processo.

Verifiche minime: Una configurazione di CI minimale esegue un insieme di verifiche fondamentali per controllare la correttezza del progetto. In particolare, il sistema verifica che:

- il progetto possa essere compilato correttamente, senza errori strutturali o di dipendenze;
- i test automatici definiti vengano eseguiti e passino senza segnalare anomalie.

Queste verifiche di base permettono di intercettare molti problemi comuni già nelle prime fasi di integrazione.



Errori di compilazione: Un errore di compilazione segnala la presenza di un problema strutturale nel codice del progetto. Questo tipo di errore può dipendere, ad esempio, da:

- errori sintattici introdotti durante la scrittura del codice;
- incoerenze tra interfacce o firme dei metodi;
- dipendenze mancanti, errate o incompatibili.

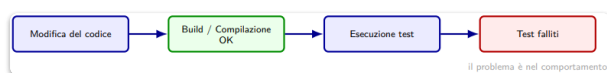
La CI segnala questi problemi in modo immediato, prima che possano propagarsi o influenzare altre parti del sistema.



Fallimento dei test: Un test che fallisce indica che il comportamento del sistema non soddisfa più le condizioni attese o dichiarate dai test. Questo può segnalare, ad esempio:

- una regressione, cioè un comportamento corretto in passato che non lo è più dopo una modifica;
- un errore logico introdotto durante lo sviluppo;
- un cambiamento voluto che richiede un aggiornamento dei test per riflettere il nuovo comportamento atteso.

La CI rende questi fallimenti immediatamente visibili, permettendo di intervenire prima che il problema si diffonda.



≡ Esempio (UniManager): individuare una regressione subito

Si consideri una modifica al progetto UniManager, ad esempio sul calcolo della media dei voti di uno studente. La modifica viene integrata nel repository e attiva automaticamente la pipeline di CI:

- la compilazione verifica che la struttura del progetto sia consistente;
- i test controllano che il comportamento atteso sia ancora valido.

In assenza di CI:

- l'errore può emergere molto più tardi, ad esempio durante l'uso di quella funzionalità in un altro modulo;
- individuare la causa risulta più difficile, perché il codice è già cambiato più volte.

Con la CI:

- il test fallisce subito dopo l'integrazione della modifica;
- la regressione viene associata a una modifica recente;
- il problema diventa visibile nell'arco di pochi secondi.

L'obiettivo non è evitare ogni errore, ma ridurre drasticamente il tempo che intercorre tra l'introduzione di un problema e la sua individuazione.

Esito della verifica: Ogni esecuzione della CI produce un esito chiaro e non ambiguo. In particolare, la verifica può risultare:

- **superata**, quando tutte le condizioni definite (build e test) sono soddisfatte;
- **fallita**, quando almeno una di tali condizioni non viene rispettata.

L'esito della verifica è oggettivo e riproducibile nel tempo, poiché deriva dall'applicazione automatica delle stesse regole.



La CI come guardiano automatico: La CI non prende decisioni progettuali e non valuta la qualità architeturale di una scelta. Non interpreta le intenzioni degli sviluppatori e non stabilisce se una modifica sia “giusta” dal punto di vista del dominio. Si limita a confrontare lo stato del progetto con un insieme di regole di verifica definite in anticipo (ad esempio compilazione e test), producendo un esito oggettivo.



Impatto della CI sul processo: La CI rende la verifica una proprietà del processo, non una scelta individuale. Effetto principale: feedback rapido sulle modifiche condivise.

- i problemi emergono subito dopo l'introduzione di una modifica;
- diventa più semplice risalire alla causa del problema.

Riduce il rischio di regressioni che si accumulano nel tempo:

- i comportamenti già verificati vengono controllati continuamente;
- le modifiche non corrette vengono intercettate prima.

Migliora l'affidabilità complessiva del progetto:

- le verifiche vengono eseguite sempre nello stesso modo;
- la qualità diventa più osservabile e meno soggettiva.

Limiti e responsabilità: La CI verifica solo ciò che è stato esplicitamente definito (build, test, altri check). Una pipeline “verde” non dimostra l'assenza totale di difetti:

- test incompleti possono nascondere problemi reali;
- alcune anomalie emergono solo in esercizio.

La CI non sostituisce la progettazione software:

- non corregge scelte architetturelle sbagliate;
- non elimina la necessità di ragionare sul dominio applicativo.

Responsabilità condivisa del team:

- mantenere test significativi;
- migliorare progressivamente le regole di verifica.

Dai principi agli strumenti: La CI nasce come insieme di principi, indipendenti da qualunque tecnologia specifica. Nella pratica, tali principi vengono realizzati

tramite strumenti che automatizzano le verifiche viste finora. Il ruolo di questi strumenti è operativo:

- eseguire automaticamente build e test;
- fornire un esito chiaro e immediato;
- rendere il processo ripetibile e sistematico.

Esempi di piattaforme di CI

Nel tempo sono nate diverse piattaforme per supportare la CI. Alcuni esempi diffusi sono:

- Jenkins;
- GitLab CI;
- GitHub Actions.

Queste piattaforme svolgono lo stesso ruolo concettuale:

- automatizzano le verifiche;
- segnalano l'esito delle integrazioni.

Un esempio concreto di utilizzo

Si consideri un progetto software con:

- una fase di compilazione;
- una suite di test automatici.

A ogni modifica del codice, la piattaforma di CI:

- compila automaticamente il progetto;
- esegue i test definiti;
- produce un esito complessivo.

Lo sviluppatore non deve eseguire manualmente queste operazioni ogni volta.

GitHub Actions

GitHub Actions è una delle piattaforme che permettono di realizzare la CI. In questo contesto, viene considerata semplicemente come un esempio concreto di strumento. I dettagli tecnici di configurazione non rientrano negli obiettivi di questa lezione:

- ciò che conta è il ruolo svolto;
- non il modo specifico in cui viene configurata.

Stato della verifica: Ogni esecuzione della CI produce uno stato di verifica associato alla modifica effettuata. Questo stato riassume l'esito delle verifiche automatiche:

- build e test superati;
- oppure presenza di errori.

Nella pratica, questo stato viene mostrato attraverso indicatori visivi immediati:

- ✓ Verde – verifica superata;
- × Rosso – verifica fallita;
- ⚠ Giallo – verifica in corso.

Questi segnali forniscono un colpo d'occhio sullo stato di salute del progetto in quel momento, senza garantire l'assenza totale di problemi.

Riepilogo:

- La CI rende sistematiche verifiche che altrimenti dipenderebbero dalla disciplina manuale del team.
- Il valore principale della CI è il feedback rapido su modifiche condivise.
- Build e test automatici sono i primi controlli da integrare; altri check possono essere aggiunti in modo progressivo.
- La CI collega in modo operativo Git, testing automatico e collaborazione.
- Gli strumenti (come GitHub Actions) sono mezzi per realizzare la pratica, non il centro concettuale della lezione.

Parte 20 - Manutenzione ed Evoluzione del Software

Una convinzione tipica (soprattutto all'inizio) è che il progetto “finisca” quando il software funziona. In realtà, il rilascio è spesso il momento in cui il sistema inizia davvero a vivere:

- arrivano utenti reali con bisogni reali,
- emergono casi limite non previsti,
- cambiano vincoli tecnici e contesti d'uso.

Per questo motivo, una parte rilevante dei costi e del lavoro si sposta su:

- correzioni,
- adattamenti,
- miglioramenti qualitativi,
- nuove funzionalità.

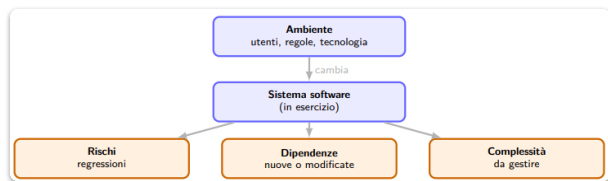
L'ingegneria del software serve a rendere questi cambiamenti gestibili, non “eroici”.

Il software non è un oggetto statico: è un sistema che opera in un ambiente che cambia. Cambiano le persone (utenti e team), cambiano le regole, cambiano gli strumenti. Ogni cambiamento introduce inevitabilmente:

- nuovi rischi di regressione,

- nuove dipendenze,
- nuove complessità da controllare.

Una buona progettazione non elimina il cambiamento, ma riduce il costo di “pagare” quel cambiamento nel tempo.



Che cos'è la manutenzione del software: Per manutenzione intendiamo le attività svolte dopo il rilascio per mantenere il sistema:

- corretto rispetto ai requisiti,
- compatibile con il contesto,
- utilizzabile e affidabile per gli utenti,
- sostenibile per chi lo mantiene.

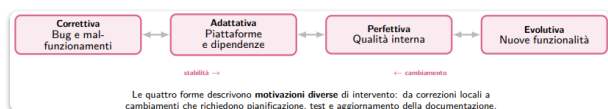
Non è solo “fixare bug”: spesso significa anche migliorare il design, ridurre debito tecnico, rafforzare i test e rendere la pipeline più robusta. Il punto chiave è che la manutenzione è parte naturale del ciclo di vita: se la ignoriamo, il progetto tende a deteriorarsi.

Le quattro forme di manutenzione: Tradizionalmente distinguiamo quattro forme, utili per capire perché stiamo intervenendo:

- **Correttiva:** eliminare malfunzionamenti (bug).
- **Adattativa:** adeguarsi a cambiamenti di piattaforme, librerie, dipendenze.
- **Perfettiva:** migliorare qualità interna (prestazioni, leggibilità, struttura) senza nuove funzionalità.
- **Evolutiva:** introdurre nuove funzionalità o estensioni del sistema.

Questa classificazione è pratica perché aiuta a prevedere:

- impatto su test e architettura,
- rischio di regressioni,
- priorità e pianificazione.



Manutenzione correttiva: La manutenzione correttiva interviene quando il sistema non si comporta come atteso:

- bug funzionali,
- errori di validazione,
- crash, eccezioni non gestite,
- comportamenti incoerenti in casi limite.

Spesso nasce da:

- segnalazioni degli utenti,
- ticket e issue interne,

- test automatici che iniziano a fallire,
- problemi in produzione.

Anche un fix “piccolo” può essere rischioso se il sistema è accoppiato: per questo i test sono cruciali.

Manutenzione adattativa: La manutenzione adattativa si verifica quando il contesto cambia, non (necessariamente) i requisiti. Esempi tipici:

- aggiornamento di una libreria con API diversa,
- migrazione a una nuova versione del database,
- cambiamento di sistema operativo o ambiente di deploy,
- nuovi vincoli di sicurezza o autenticazione.

Qui diventa evidente il valore di una buona architettura: separare dominio e infrastruttura riduce quanto “si propaga” l'impatto delle dipendenze.

Manutenzione perfettiva: La manutenzione perfettiva migliora la qualità del sistema senza cambiare le funzionalità “visibili”. È spesso associata a:

- refactoring e semplificazione del codice,
- riduzione del debito tecnico accumulato,
- miglioramenti prestazionali mirati,
- riorganizzazione di moduli per aumentare coesione e ridurre accoppiamento.

È facile rimandarla perché “non aggiunge feature”, ma nel lungo periodo è ciò che rende il progetto modificabile e quindi più economico.

Manutenzione evolutiva: La manutenzione evolutiva introduce nuove funzionalità o estende il comportamento del sistema. È la forma che più coinvolge tutto il ciclo:

- aggiornamento requisiti,
- aggiornamento modelli e design,
- nuovi test e revisione di quelli esistenti,
- aggiornamento documentazione e release notes.

È anche quella dove il rischio di “rompere qualcosa” è più alto, perché cambiano casi d'uso e flussi. Per questo serve un processo disciplinato e una base di test affidabile.

Architettura e costo del cambiamento: Il costo di un cambiamento non dipende solo dalla quantità di codice da scrivere, ma da quanto lontano si propaga l'impatto. Se le responsabilità sono mescolate e le dipendenze sono “ovunque”:

- un fix locale può richiedere modifiche in molti punti,
- diventa difficile capire cosa si sta rompendo,
- aumenta la paura di cambiare.

I principi visti nel corso (SRP, DIP, separazione dei livelli, confini architetturali) servono proprio a rendere i cambiamenti localizzati e quindi più gestibili.



Dal problema al codice: Quando emerge un problema (da un utente, da un log, da un test che fallisce), la tentazione naturale è “aprire l'IDE e correggere subito”. In un progetto serio, invece, conviene fermarsi un attimo e seguire un ordine:

- tracciare il problema in modo esplicito (issue), così da renderlo visibile e discutibile dal team;
- riprodurre il difetto in modo controllato, per evitare di lavorare su ipotesi vaghe o non verificabili;
- capire la causa (root cause), distinguendo chiaramente il sintomo osservato da ciò che lo genera.

Questo passaggio iniziale sembra “tempo perso”, ma in realtà riduce due rischi classici:

- correggere la cosa sbagliata (fix che maschera il sintomo ma non risolve il problema reale);
- introdurre regressioni non previste, perché si interviene senza sapere quali parti del sistema sono coinvolte.

In altre parole: prima comprendiamo e delimitiamo il problema, poi lo risolviamo in modo verificabile.

Il ciclo disciplinato di modifica: Un flusso robusto e ripetibile per introdurre un cambiamento è: issue → branch → fix → test → PR → merge. Dopo aver sviluppato e verificato la modifica su un branch dedicato, il cambiamento non viene integrato direttamente, ma passa attraverso una richiesta di integrazione (Pull Request). La Pull Request rende il cambiamento:

- visibile agli altri membri del team,
- revisionabile prima dell'integrazione,
- verificabile automaticamente tramite test e CI.



Il ruolo centrale dei test: I test non servono solo a “controllare” il codice alla fine: sono parte del lavoro di manutenzione. Un modello pratico è:

- un test che fallisce riproduce il bug in modo oggettivo,
- il fix rende il test verde,
- il test rimane come garanzia contro future regressioni.

Se correggiamo un bug senza aggiungere o aggiornare test, stiamo lasciando aperta la porta al ritorno dello stesso problema. Nel lungo periodo, una buona suite di test è ciò che rende possibile cambiare con fiducia.

CI come rete di sicurezza: La Continuous Integration automatizza controlli fondamentali:

- build,
- test,

- eventualmente linting e analisi statica.

Il valore della CI non è “fare i test più velocemente”, ma garantire che ogni modifica passi da una verifica ripetibile e indipendente. Collegamento con le lezioni precedenti:

- i test riducono regressioni,
- la CI impedisce che regressioni entrino nel ramo principale,
- la review riduce errori di design e di implementazione.

Versioning semantico: Per comunicare chiaramente il tipo di cambiamento introdotto si usa spesso il Semantic Versioning. Formato: MAJOR.MINOR.PATCH

- **MAJOR:** cambiamenti incompatibili (breaking changes).
- **MINOR:** nuove funzionalità retrocompatibili.
- **PATCH:** correzioni e piccoli fix senza modifiche di API/comportamento atteso.

È una convenzione semplice, ma estremamente utile quando:

- più team dipendono dallo stesso componente,
- dobbiamo pianificare aggiornamenti,
- vogliamo evitare “sorprese” agli utenti.



Perché il versioning conta davvero: Il versioning non è solo un dettaglio formale, ma un vero e proprio linguaggio comune tra chi sviluppa software e chi lo utilizza o lo integra. Attraverso il numero di versione, il progetto comunica in modo sintetico il tipo di cambiamento introdotto e il livello di rischio associato. In pratica, il versioning aiuta a rispondere rapidamente a domande molto concrete:

- Questo aggiornamento è sicuro da applicare senza effetti collaterali?
- Potrebbe rompere la compatibilità con codice o servizi esistenti?
- Introduce nuove funzionalità oppure solo correzioni e miglioramenti interni?

Questo diventa ancora più importante quando un progetto dipende da altri pacchetti o servizi: una gestione chiara e coerente delle versioni riduce conflitti, ambiguità e tempi di diagnosi in caso di problemi. In altre parole, un buon versioning rende i cambiamenti prevedibili e aiuta team e utenti a prendere decisioni informate sugli aggiornamenti.

Gestione delle release: Una release non è semplicemente “fare merge” sul ramo principale, ma un momento preciso in cui un insieme di cambiamenti viene reso adottabile da utenti e altri sistemi. Preparare una release significa decidere consapevolmente che cosa entra e che cosa resta fuori in quel rilascio. In pratica, una gestione corretta delle release include:

- la selezione delle issue incluse, spesso organizzate in milestone o sprint target;
- una fase di verifica della qualità, basata su test automatici, CI e controlli aggiuntivi;
- la documentazione dei cambiamenti, così che gli utenti possano capire cosa è cambiato.

A supporto di questo processo si usano strumenti e pratiche consolidate:

- un changelog per tenere traccia delle modifiche nel tempo;
- delle release notes orientate all'utente e all'impatto pratico del rilascio;
- automazioni per creare tag di versione, generare artefatti e pubblicare il rilascio.

Una buona gestione delle release riduce sorprese e rende il ciclo di evoluzione più prevedibile.

Caso di studio: UniManager – una feature nuova

Per rendere concreto il concetto di evoluzione del software, consideriamo l'introduzione di una nuova funzionalità in UniManager, ad esempio:

- il rifiuto del voto da parte dello studente,
- oppure la visualizzazione della media ponderata degli esami sostenuti.

Si tratta di funzionalità realistiche e plausibili, che riflettono esigenze comuni nei sistemi informativi universitari. Il punto centrale, però, non è la feature in sé, ma osservare come anche un cambiamento apparentemente “semplice” abbia un impatto su più livelli del sistema. In particolare, l'evoluzione coinvolge:

- i requisiti, che devono essere chiariti e aggiornati;
- il modello e il design, che possono richiedere nuove responsabilità;
- l'architettura, per verificare che i confini restino coerenti;
- i test, che devono proteggere il nuovo comportamento;
- la versione e la release, per comunicare correttamente il cambiamento.

Impatto sui requisiti: Aggiungere una funzionalità significa prima di tutto chiarire:

- cosa deve essere possibile fare,
- in quali condizioni,
- con quali vincoli e casi limite.

Esempio (rifiuto del voto):

- quando lo studente può rifiutare?
- quante volte è consentito?
- cosa succede allo stato del verbale?

Requisiti chiari diventano la base per:

- progettare correttamente,
- scrivere test verificabili,
- evitare ambiguità tra team e utenti.

Impatto su UML e architettura: La nuova feature spesso richiede di aggiornare:

- diagrammi (stati, sequenze, classi) se li stiamo usando per documentare comportamento e responsabilità,

- confini architetturali: chi decide cosa? dove sta la regola?

Obiettivo: evitare di “spargere” logica di dominio ovunque. Una buona domanda guida è: questa regola è dominio (stabile) o infrastruttura (variabile)? Se i confini sono chiari, l'evoluzione diventa più locale e meno rischiosa.

Impatto su test e CI: Una feature nuova implica:

- nuovi test (unità e/o integrazione),
- aggiornamento di test esistenti se cambiano regole o flussi.

I test vanno pensati anche come protezione futura: la feature di oggi diventa la regressione di domani se non viene coperta. La CI deve continuare a garantire che ogni PR:

- compili,
- passi i test,
- non introduca rotture evidenti.

Versione e rilascio della nuova funzionalità: Se la nuova funzionalità è compatibile con quanto esisteva prima, tipicamente incrementiamo la MINOR: ad esempio 1.0.0 → 1.1.0. A quel punto una release “fatta bene” include:

- changelog con le issue chiuse,
- note chiare su eventuali modifiche di comportamento,
- indicazioni di migrazione se serve.

Il valore aggiunto è la prevedibilità: gli utenti sanno cosa aspettarsi e quando adottare l'aggiornamento.

Evoluzione continua: La manutenzione non è un tema separato: è il punto in cui tutto ciò che abbiamo visto converge. Requisiti chiari riducono ambiguità; architettura pulita riduce propagazione dei cambiamenti. Test e CI permettono di cambiare senza paura, documentazione riduce attrito e onboarding. Un progetto che cura questi aspetti:

- dura più a lungo,
- cresce in modo controllato,
- rimane comprensibile anche dopo anni.

Messaggio finale: Costruire buon software non significa solo farlo funzionare oggi. Significa progettarlo e mantenerlo in modo che sia ancora:

- comprensibile,
- modificabile,
- affidabile, domani, quando i requisiti, il contesto e le persone saranno cambiati.

L'evoluzione non è un incidente del percorso, ma la naturale prosecuzione di ogni progetto reale. Messaggio finale: il buon software si progetta pensando al cambiamento.

- Il software va pensato come un sistema vivo: il rilascio iniziale apre una fase di manutenzione ed evoluzione continua.
- I diversi tipi di manutenzione hanno obiettivi e rischi diversi, quindi richiedono priorità e strategie operative differenti.
- Test automatici, CI e workflow disciplinato sono ciò che rende i cambiamenti più frequenti e meno rischiosi.
- Architettura, refactoring e documentazione incidono direttamente sul costo futuro della manutenzione, non solo sulla qualità immediata.
- Il versioning semantico aiuta a comunicare l'impatto dei rilasci e a rendere l'evoluzione più prevedibile per chi usa il software.
- La visione finale del corso è che il buon software si progetta, testa e documenta pensando esplicitamente al cambiamento.



Navigazione Lezioni

- **Index Corso** : [00 Index Ingegneria Software](#)